微网购电的机理建模与滚动调控

摘　要

微网把光伏就地消纳，用储能把低价时段的电量搬到高价时段，购电费用取决于每个时段买多少电、储能什么时候充什么时候放。本文针对含光伏与储能的非孤岛式微网，从母线上的电量平衡出发，逐层建立了确定性日前规划、含预测误差的滚动计划、带偏差费用的日内调整与波动电价下的重算四个模型，并为电量换算、误差抵御、预报融合、偏差计价这些关键机理逐处配了示意图。

针对问题一，题目要求在电价、负载与光伏预测已知、储能首尾储电量相同时制定当天的计划购电策略。首先把一天切成 144 个 10 分钟时段，把功率乘时段长换成电量，由母线的三个来源两个去向写出供需不等式，由充放电各折一次效率写出储电量递推式并累加消元；然后以全天购电费最小为目标，建立**基于线性规划的日前计划购电模型**，并证明最优解自动满足同一时段不同时充放电，模型无需 0-1 变量；最后用**对偶单纯形法**求解，与内点法、动态规划、遗传算法、粒子群算法、模拟退火算法和谷充峰放规则法对比，精确解法用时 0.026 s 且命中最优比例 100%，启发式最好也差 2.663%；供需、边界与首尾三项检验全部成立，能量守恒两端的吻合度为 100%。全天购电量 59482.6990 kWh，购电费 35126.9486 元，比不用储能节省 26.90%。

针对问题二，负载与光伏逐日变化，当天计划一经确定不可更改，缺口按 5 倍电价紧急购电。首先发现负载有极强的周内效应，据此建立**同类日形状乘近期水平的负载预测模型**，与谐波回归、同星期几均值等七种方法对比，相对误差最低，为 2.82%；然后把误差代价写进日前规划，用光伏按预测的 95% 计入与储能分段留备用两条途径抵御误差，建立**基于光伏折减与分段备用的滚动计划购电模型**；最后用**滚动时域控制算法**逐段仿真全年。2025 年 2 月 1 日至 12 月 31 日的总购电费为 13492783.13 元，比不做任何抵御节省 7.35%，与逐日完美预测相比只高 10.19%。

针对问题三，每天增加 6:00、12:00 和 18:00 三次滚动预报，调整量相对计划量另计偏差费用。首先把整点预报按时段末刻插值并以发布时刻的实测值作左端点，权重随提前小时数线性衰减，建立**按提前量衰减的预报融合模型**，12:00 发布的预报误差由按中点取值的 4.70% 降到 3.24%，融合之后再降到 3.10%；然后把违约与超额两段费用写成绝对值并线性化，建立**基于偏差费用的日内滚动调整模型**；最后逐档对照，只用 0:00 预报为 13496984.66 元，依次加入三次调整降到 13353137.48、13114289.88 与 13068766.24 元，**需要引入其他时刻的预报，且三个时刻都要用上**，调整使总费用再降 3.17%。

针对问题四，固定电价换成实时波动电价。首先统计附件 4，日内极差均值为 1.1073 元/kWh，是固定电价的 1.08 倍；然后指出电价只以系数向量进入目标函数，不改变任何约束，可行域完全不变，据此建立**波动电价下的滚动调控模型**；最后重新求解，波动电价下问题二为 14130588.56 元，问题三为 13689275.77 元，调整带来的节省由固定电价下的 3.17% 升到 3.22%，把电价换成全年逐段均值后重算得 13068764.79 元，与问题三相差 1.45 元，吻合度接近 100%。

最后对安全裕度、备用容量与两级备用配比做了灵敏度分析，费用曲面在最优点附近平坦，参数取值不必精调；四问的完整策略见支撑材料中的 result1.xlsx 至 result4-3.xlsx。

关键词：线性规划　滚动时域控制　同类日形状预测　储能备用　偏差费用线性化

1、 问题重述

1.1 问题背景

微网是为解决光伏发电等分布式能源本地消纳、减少并网问题而发展起来的小型发用电系统[1,2]。非孤岛式微网可以在外网电价较低或分布式能源有剩余电量的时段为储能设备充电，在电价较高的时段释放电能[3]。调控的目的是利用小区负载、光伏发电量与储能当前储电量等数据，尽可能精准地给出购电量与充放电量，使微网提供的电力既满足负载，又尽可能节省购电费用。

本题的微网由外部电网、光伏电源、储能设备和小区负载四部分组成，四者挂在同一条母线上，如图 1 所示。

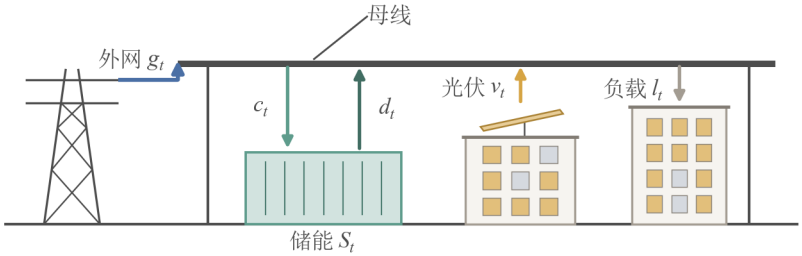


图 1 微网的构成与母线上的四条支路

图 1 中只有外网这一路要花钱，只有小区这一路是刚性的，只有储能这一路可以双向流动，光伏这一路不计成本但不可调度。这三句话决定了后面四个模型的全部结构：目标函数只对购电量计价，供需约束把负载放在不等式右端，储能带来跨时段的耦合。

1.2 问题要求

题目按信息条件由易到难给出四个问题。

- 问题一：电价与小区负载每天相同，并给出光伏发电功率一天的预测数据。要求微网提供的电能不低于小区负载，储能设备在 0:00 与 24:00 的储电量相同，在每天 0:00 制定当天的计划购电策略。
- 问题二：电价每天相同，负载与光伏发电功率随时间变化。微网提供的电能低于负载时需紧急购电，电价为交易时刻电价的 5 倍，其余时段按计划购电量计费。要求在每天 0:00 制定当天的计划购电策略，给出 2025 年 2 月 1 日至 12 月 31 日的结果。
- 问题三：每天 0:00、6:00、12:00 和 18:00 可获得未来 24 小时整点的光伏发电功率预报。计划购电量高于调整购电量的部分按交易时刻电价的 50% 计违约费用，调整购电量高于计划购电量的部分按 1.5 倍计费。要求建立当天购电策略的模型，并分析是否需要引入其他时刻的预报。
- 问题四：外网电价实时波动。要求针对波动电价建立购电策略模型，并在波动电价下重新计算问题二与问题三。

2、 问题分析

2.1 问题一的分析

题目要求在电价、负载与光伏预测全部已知的条件下，制定使全天购电费最低的计划购电策略。

首先，把一天等分成若干个 10 分钟时段，把附件给出的功率乘时段长换算成电量，使供需关系与储电量变化写在同一量纲下。母线上只有购电、光伏与储能放电三个来源，

去向是负载与储能充电，据此写出每个时段的供需不等式，再由充放电效率写出储电量的递推式。

然后，注意到光伏不计成本、储能不产生电量，全部费用只来自购电，因此目标函数是电价与购电量的逐段乘积之和。目标与约束都对决策变量线性，把储电量按下三角矩阵累加消元后，问题化为一个标准线性规划。

最后，采用对偶单纯形法求解，与另外六种算法对比，并从供需、储电量边界、首尾储电量与能量守恒四个方面检验解的正确性。

2.2 问题二的分析

题目要求在负载与光伏逐日变化、当天计划不可更改、缺口按 5 倍电价补齐的条件下，在每天 0:00 制定计划购电策略。

首先，与问题一相比，本问的核心变化是 0:00 无法知道当天的真实负载与光伏，必须先做预测。对附件 2 统计发现负载存在极强的周内效应，周五与周六的日总负载只有其余各天的六成左右，因此把同类日的形状与近期的水平拆开预测，光伏则取近若干天的同段均值。

然后，预测有误差，计划买少了就要按 5 倍电价补齐。抵御误差有两条途径：把光伏按预测的一个折扣计入，相当于多买一点电；在储能里留出一层不动用的备用容量，相当于把套利收益让出一部分。两条途径的配比要通过全年对照确定。

最后，用滚动时域控制算法逐段仿真整年的实际运行，购电计划在 0:00 定死，储能按实测数据实时调度，缺口计入紧急购电，以仿真得到的真实费用作为唯一的评价口径。

2.3 问题三的分析

题目要求在可获得四个时刻滚动预报、调整量相对计划量需另计偏差费用的条件下制定购电与调整策略，并回答是否需要引入其他时刻的预报。

首先，附件 3 给的是整点预报，而模型按 10 分钟时段算，必须先把预报插值到各时段，插值取在时段末刻，发布时刻已经实测到的功率作为插值的左端点。官方预报越靠近发布时刻越准，历史均值则不随提前量变化，因此融合权重随提前小时数线性衰减。

然后，把偏差费用写成计划量与调整量之差的绝对值形式，引入超额量与违约量两组非负变量将其线性化，模型仍是线性规划。

最后，按调整时刻的开启档位逐档求解，用总费用的下降幅度回答是否需要引入其他时刻的预报，并用关掉全部调整的算例做退化检验。

2.4 问题四的分析

题目要求在实时波动电价下重新计算问题二与问题三。

首先，统计附件 4 的电价，与附件 1 的固定电价比较日内极差与取值范围，判断波动对套利空间与误差代价的影响方向。

然后，注意到电价在前两问的模型中只以系数向量的形式出现在目标函数与偏差费用中，不改变任何约束的结构与右端项，因此只需把每天的电价向量替换为附件 4 对应日期的电价，模型结构完全沿用。

最后，用与问题二、问题三相同的滚动仿真求解，比较两种电价下调整带来的节省幅度，并把波动电价换成全年逐段均值做退化检验。

2.5 全文的求解思路

四问的模型与算法之间的关系如图 2 所示。

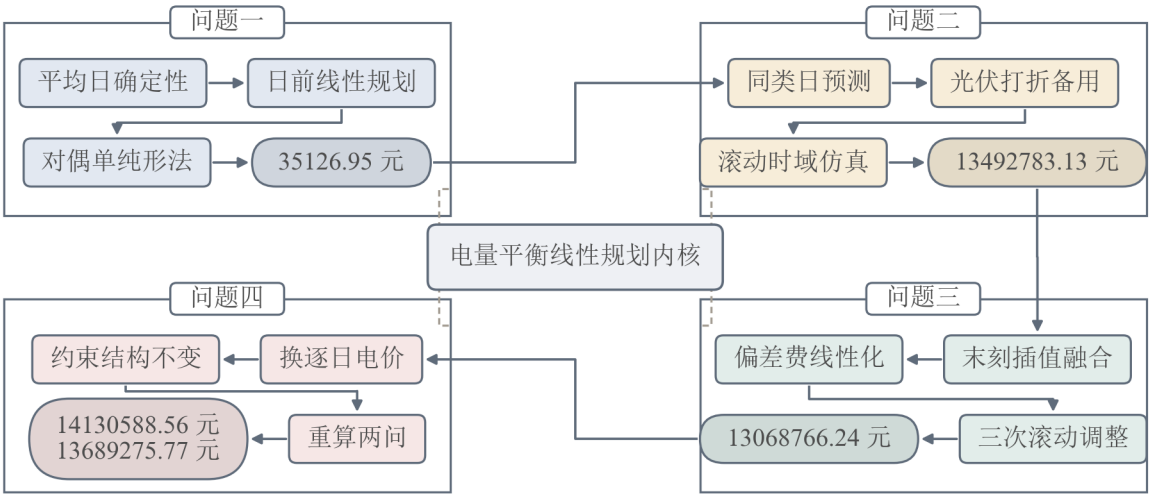


图 2　全文求解思路

图 2 的中间是四问共用的内核：144 段的电量平衡与储电量的线性规划。问题一在确定性条件下解这个内核；问题二在它外面套一层预测与误差抵御；问题三再加一层日内调整与偏差计价；问题四只换电价系数，内核与约束一点不动。四个栏之间的箭头表示后一问建在前一问的模型之上。

3、　模型假设

1. 假设小区负载是刚性的，不可中断也不可转移，微网只能通过购电与储能满足它。
2. 假设光伏发电不计边际成本，储能已满时多余的电量可以弃掉，不必强行消纳。
3. 假设微网不向外网反送电量，购电量非负。
4. 假设题目给出的充放电效率 90% 是充电与放电各自的效率，即 $\eta = 0.9$，且不随荷电状态与温度变化。
5. 假设每天 0:00 可以获得当天完整的电价曲线，波动电价在日前已经公布。
6. 假设储能的充放电指令在时段内实时执行，不存在响应延迟。

4、　符号说明

其它符号在文中具体说明。

5、　模型的建立与求解

5.1　问题一模型的建立与求解

5.1.1　问题一的思路框架

问题一的求解通路如图 3 所示。

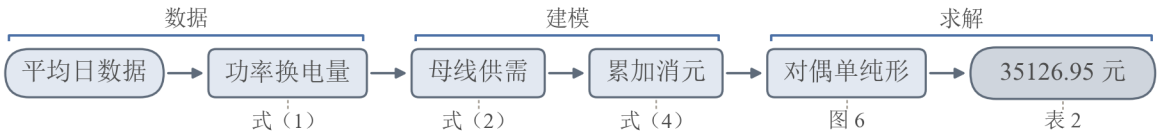


图 3　问题一的思路框架

图 3 上面的括号把这一问分成数据、建模与求解三段，下面挂的是正文里对应的式号与图号。下面按这条通路逐段展开。

符号	说明	单位
T	一天划分的时段数	—
Δt	单个时段的长度	h
p_t	第 t 段的电价	元/kWh
l_t	第 t 段的小区负载电量	kWh
v_t	第 t 段的光伏发电量	kWh
g_t	第 t 段的计划购电量	kWh
a_t	第 t 段的调整购电量	kWh
e_t	第 t 段的紧急购电量	kWh
c_t	第 t 段储能从母线取走的充电量	kWh
d_t	第 t 段储能向母线送出的放电量	kWh
u_t	第 t 段调整量高于计划量的部分	kWh
z_t	第 t 段调整量低于计划量的部分	kWh
S_t	第 t 段末的储电量	kWh
η	储能的充放电效率	—
$E_{\max}$	单个时段的最大充放电量	kWh
R	计划阶段留出的储能备用容量	kWh
β	光伏预测的折减比例	—
α	净负载的安全裕度	—
w	官方预报在融合中的权重	—

5.1.2 时段划分与电量换算

把一天划分为 $T = 144$ 个等长时段，每段长 $\Delta t = 1/6$ 小时，按附件时间列的顺序编号 $t = 1, 2, \ldots, T$。附件给出的负载与光伏都是功率，乘时段长换算成电量

$$l_t = L_t \, \Delta t, \qquad v_t = P_t^{\mathrm{pv}} \, \Delta t, \tag{1}$$

其中 L_t 与 P_t^{pv} 分别是第 t 段的负载功率与光伏功率。附件的第 t 个数给的是该段末刻的功率，写结果文件时按同一位置填模板的第 t 行或第 t 列，论文里的表也从同一位置取数，三者对齐，这一点在问题三处理整点预报时还要再用一次。

5.1.3 机理分析

微网母线上的电量有三个来源和两个去向，来源是购电量、光伏发电量与储能放电量，去向是小区负载与储能充电量。题目要求微网提供的电能不低于小区负载，故

$$g_t + v_t + d_t - c_t \geqslant l_t, \qquad t = 1, 2, \ldots, T. \tag{2}$$

取不等号而不取等号，对应假设 2 中允许弃光的设定。

储能从母线取走 c_t 只有 ηc_t 存进设备，设备放出 d_t/η 才能向母线送出 d_t，因此储电量逐段递推

$$S_t = S_{t-1} + \eta \, c_t - \frac{d_t}{\eta}, \qquad t = 1, 2, \ldots, T, \tag{3}$$

逐段累加即把状态变量消去

$$S_t = S_0 + \sum_{i=1}^{t} \left(\eta \, c_i - \frac{d_i}{\eta} \right). \tag{4}$$

式（4） 的含义如图 4 所示。

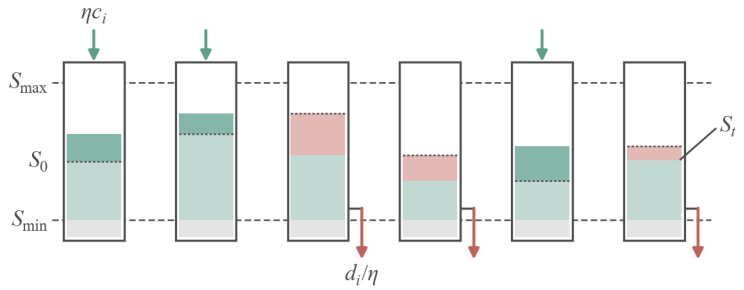


图 4　储电量是逐段增减的累加：进罐的电量折一次效率，出罐的电量再折一次

图 4 中每一个罐是一个时段，虚线是上一段的水位，深色一段是本段的增减；无论中间怎样充放，第 t 段的水位都等于期初水位加上前 t 段增减之和，两条横虚线是储电量的上下限。把这一条写进约束，含状态递推的调度问题就化成了只含 c_i 与 d_i 的线性不等式，这是本文四个模型都能保持线性规划的关键。

储能之所以值得用，是因为电价在一天之内有落差。搬运一度电的过程如图 5 所示。

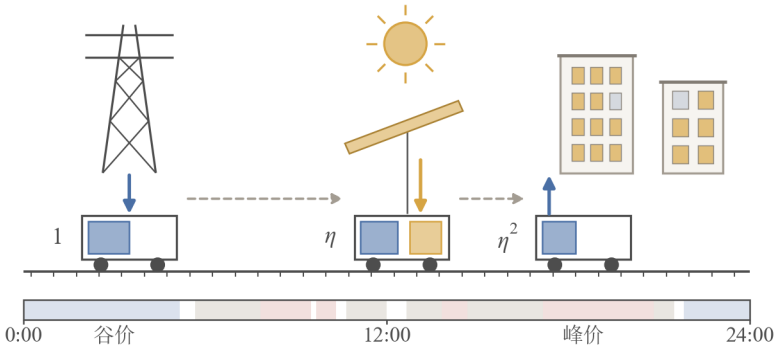


图 5　储能在一天之内的搬运：谷段装进一度，存进罐里折一次，峰段送出再折一次

图 5 中车厢里的方块随着一装一卸缩小两次，底部的色带是一天的电价分档。在谷段用单价 p_i 买进一度电，到峰段只能替代 η^2 度按单价 p_j 的购电，因此搬运划算的条件是 $\eta^2 p_j \geqslant p_i$，往返损耗率为 $1-\eta^2 = 19\%$。附件 1 的电价谷峰落差为 1.0239 元/kWh，远大于损耗代价，所以最优解会把购电量尽量挪到谷段，午间光伏富余时再充一次。

5.1.4　模型的建立

决策变量的建立。每个时段有三个决策量，计划购电量 g_t、储能充电量 c_t 与放电量 d_t，单位都是 kWh，合计 $3T$ 个变量。

目标函数的建立。全天费用只来自购电，故

$$\min\ F = \sum_{t=1}^{T} p_t\, g_t. \tag{5}$$

约束条件的建立。第一类是式（2）的供需约束。第二类是容量约束，储电量须落在 $[S_{\min}, S_{\max}] = [1200, 10800]$ kWh 内，代入式（4）得

$$S_{\min} \leqslant S_0 + \sum_{i=1}^{t} \left(\eta\, c_i - \frac{d_i}{\eta} \right) \leqslant S_{\max}. \tag{6}$$

第三类是功率约束，单段充放电量不超过 $E_{\max} = P_{\max}\Delta t = 833.33$ kWh。第四类是题目要求的首尾储电量相同，即式（4）在 $t = T$ 处的累加和为零。

综上所述，最终得到**基于线性规划的日前计划购电模型**为

$$
\begin{cases}
\min\limits_{g,\,c,\,d} \ \displaystyle\sum_{t=1}^{T} p_t\, g_t & \\
\text{s.t.} \begin{cases}
g_t + v_t + d_t - c_t \geqslant l_t, & t = 1,\ldots,T \\
S_{\min} \leqslant S_0 + \displaystyle\sum_{i=1}^{t}\left(\eta c_i - \frac{d_i}{\eta}\right) \leqslant S_{\max}, & t = 1,\ldots,T \\
\displaystyle\sum_{i=1}^{T}\left(\eta c_i - \frac{d_i}{\eta}\right) = 0, & \\
0 \leqslant c_t \leqslant E_{\max}, \quad 0 \leqslant d_t \leqslant E_{\max}, \quad g_t \geqslant 0, \quad t = 1,\ldots,T &
\end{cases}
\end{cases}
\tag{7}
$$

模型没有引入 0-1 变量禁止同一时段既充又放，理由如下。设某个最优解在第 t 段同时有 $c_t > 0$ 与 $d_t > 0$，取 $\delta = \min\{c_t,\, d_t/\eta^2\}$，把充电量减去 δ、放电量减去 $\eta^2\delta$，则式（3）中本段的储电量增量 $\eta c_t - d_t/\eta$ 一点不变，此后各段的储电量轨迹与首尾条件照旧成立；而母线上的净流入由 $d_t - c_t$ 变成 $d_t - \eta^2\delta - c_t + \delta$，因 $\eta^2 < 1$ 故净流入不减，供需约束仍然满足。新解可行且目标值不变，故总存在满足 $c_t d_t = 0$ 的最优解，模型保持线性，规模为 $3T = 432$ 个变量。

5.1.5 模型的求解与检验

模型（7） 的约束矩阵含下三角全 1 块，整体稀疏，因此按稀疏矩阵组装后调用对偶单纯形法求解，步骤如图 6 所示。

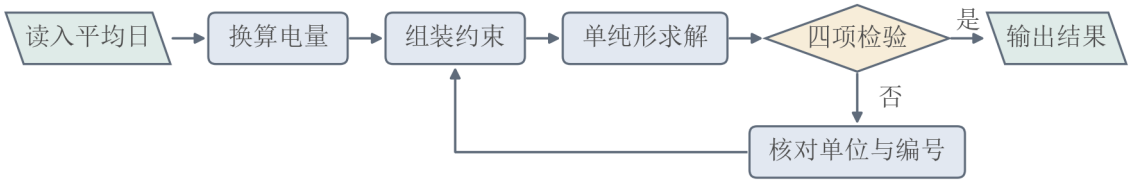


图 6　问题一的求解流程

图 6 中四项检验指供需、储电量边界、首尾储电量与能量守恒，任何一项不过都回到约束组装一步核对单位与编号。为验证求得的确为最优解，选取三类共七种算法对同一模型求解，结果如表 1 所示。

表 1　问题一的多算法对比

算法	类别	目标值/元	相对最优	命中最优比例	单次用时/s
对偶单纯形法	**精确**	**35126.9486**	**0.000%**	**100%**	**0.026**
内点法	精确	35126.9486	0.000%	100%	0.038
动态规划	精确	35244.7034	0.335%	0%	0.222
模拟退火算法	启发式	36062.2828	2.663%	0%	7.760
粒子群算法	启发式	37324.4891	6.256%	0%	0.742
遗传算法	启发式	37853.4708	7.762%	0%	0.723
谷充峰放规则法	基准	42950.0933	22.271%	0%	0.004

表 1 中启发式的目标值取 10 次运行的最好值，命中最优比例按与最优值相差不超过 0.1% 计。三种启发式 10 次运行的均值分别为 37319.2460 元、38498.4674 元与 39989.9085

元，没有一次命中最优。动态规划的 0.335% 差距全部来自储电量的离散化，把档位加密即可收敛，但用时按档位数的平方增长。由此得到选择依据：本问是规模仅 432 个变量的线性规划，精确解法能在 0.03 s 内给出可证明的最优解，启发式既慢又差，因此最终采用对偶单纯形法。

模型的检验分四项。第一项是供需，逐段计算 $g_t + v_t + d_t - c_t - l_t$，最小值为 0，约束全部成立。第二项是储电量边界，全天储电量落在 $[1200, 10800]$ kWh 内，在 5:30 至 5:50 与 15:00 至 18:00 两段触到上界，在 20:30 至 21:50 触到下界，说明容量在午间富余与傍晚放电两头都是紧约束。第三项是首尾储电量，两者均为 6000 kWh。第四项是能量守恒，由式（2） 在最优解处取等号并对 t 求和，代入首尾条件得

$$\sum_{t=1}^{T} g_t - \sum_{t=1}^{T} (l_t - v_t) = \left(1 - \eta^2\right) \sum_{t=1}^{T} c_t, \tag{8}$$

实际计算中式（8） 两端分别为 3940.7266 kWh 与 3940.7266 kWh，吻合度为 100%，即模型的准确性与真实性强。

问题一的结果如表 2 与表 3 所示，一天之内的运行过程如图 7 所示。

表 2　微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	486.4029	14:00-14:10	0.0000
16:00-16:10	394.9315	18:00-18:10	636.9826	20:00-20:10	0.0000
全天购电量		59482.6990	全天购电费		35126.9486

表 3　储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4500.0000	0.0000	4:00-8:00	833.3333	6365.8412
8:00-12:00	4787.9643	1702.9970	12:00-16:00	5286.0352	91.1014
16:00-20:00	0.0000	5780.1319	20:00-24:00	5333.3333	2859.8681
0:00 储电量		6000.0000	24:00 储电量		6000.0000

表 2 中 10:00、14:00 与 20:00 三个指定时段的购电量都是 0 kWh，因为这三段或有光伏富余、或由储能放电顶上；12:00 与 18:00 两段仍要买电，分别为 486.4029 kWh 与 636.9826 kWh。表 3 中 0:00 至 4:00 只充不放，充入 4500.0000 kWh；16:00 至 20:00 只放不充，放出 5780.1319 kWh；0:00 与 24:00 的储电量都是 6000.0000 kWh，与题目要求的首尾相同一致。

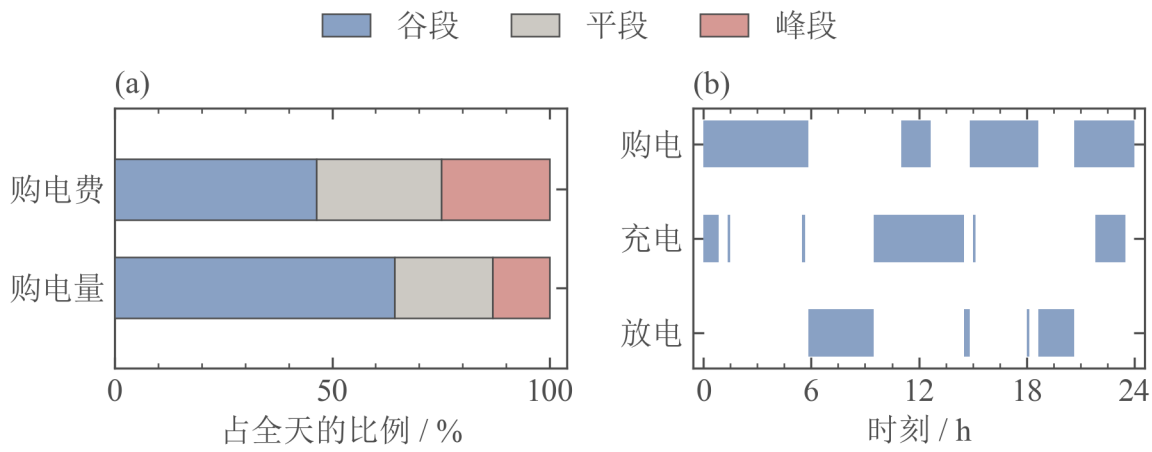


图 7　问题一的计划购电策略：(a) 三档电价下的购电量与购电费构成；(b) 购电、充电与放电的活跃时段

图 7(a) 表明谷段买走了全天 64.4% 的电量，却只花掉 46.4% 的钱，峰段买的电量只有 13.1%，说明模型确实把购电挪到了便宜的时段；图 7(b) 中充电落在凌晨与正午光伏富余的两段，放电落在清晨与傍晚两段，一天之内完成两个完整的充放循环，与图 5 的搬运机理一致。全天负载为 111024.8081 kWh，光伏发电为 55482.8357 kWh，净需求为 55541.9724 kWh，而全天购电量为 59482.6990 kWh，两者之差正是储能的往返损耗。

> 问题一的最优策略下，全天购电量为 59482.6990 kWh，全天购电费为 35126.9486 元，比不使用储能逐段照买的 48052.0466 元节省 26.90%，逐段结果见支撑材料中的 result1.xlsx。

5.2　问题二模型的建立与求解

5.2.1　问题二的思路框架

问题二在问题一的基础上引入了预测误差与紧急购电，求解通路如图 8 所示。

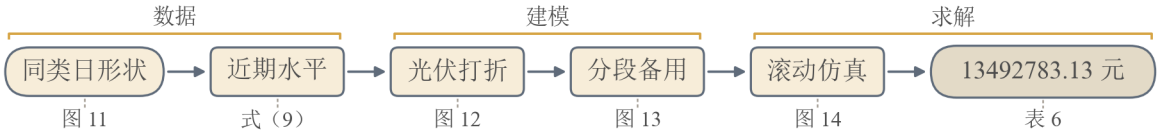


图 8　问题二的思路框架

图 8 与问题一的框架相比，数据一段多了预测，建模一段多了抵御误差的两件事，求解一段由单日求解换成全年滚动仿真；框下挂的式号与图号指向正文里对应的位置。这一问最终的总购电费是 13492783.13 元。

5.2.2　机理分析

与问题一相比，本问的决策时刻与信息到达时刻分开了。0:00 定下的购电曲线当天不能改，真实净负载却要逐段才揭晓。真实净负载低于计划购电量时，富余电量被储能吸收，不产生额外费用；高于计划购电量时，先由储能放电填补，储能补不上的部分才触发紧急购电，单价是正常电价的 5 倍。因此储能在本问中身兼两职，既是套利工具，也是预测误差的缓冲器[4,5]。

紧急购电并不只在储能没电时才发生，共有三种成因，如图 9 所示。

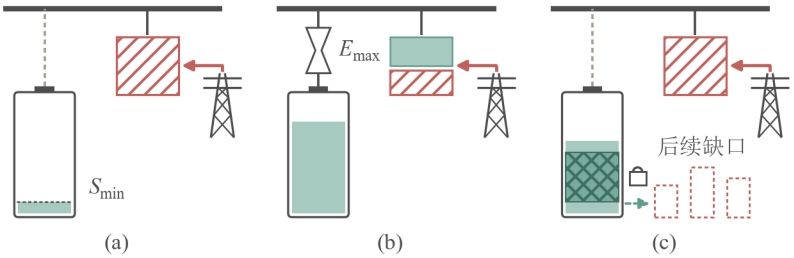


图 9　紧急购电的三种成因：(a) 储电量已到下限；(b) 放电功率顶到上限；(c) 为后面的缺口留着不放

图 9 中斜纹块是当前时段填不上的缺口，右侧的电塔表示按 5 倍电价紧急购电。(a) 是电量不够，储电量已落到下限；(b) 是功率不够，单段放电量顶到 E_{max}；(c) 是储能有电也有功率，但后面还有更贵的缺口要补，这一段故意少放。全年仿真下来，三种成因分别占紧急购电量的 30.6%、9.3% 与 60.0%，最大的一块是有意保留，说明实时调度的放电意图与日前计划同样要紧。这三条分别指向后面的三件事：留出备用容量、认清单段放电功率的上限、按后续缺口决定本段放多少。

5.2.3　负载与光伏的预测

负载的逐日变化由两件事主导，一是周五与周六明显偏低的周内效应，二是年内缓慢的季节漂移。按星期几统计日总负载，周五与周六约为 78400 kWh，其余五天约为 124000 kWh，只有六成左右，如图 10 所示。

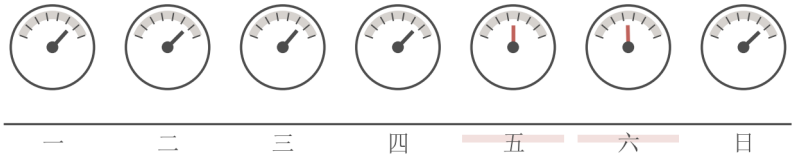


图 10　一周七天的日总负载：周五与周六的表针只走到其余各天的六成

图 10 中周五与周六两块表的指针明显偏低。不看星期几的两种方法误差最大，前一天为 14.08%，全年同段均值为 17.41%，因此预测必须先把这两类日子分开。本文把一天的曲线拆成形状与水平两部分[10]：形状取最近 8 个同类日归一化后的平均，水平取最近三天折算到同一日型后的加权日总量，做法如图 11 所示。

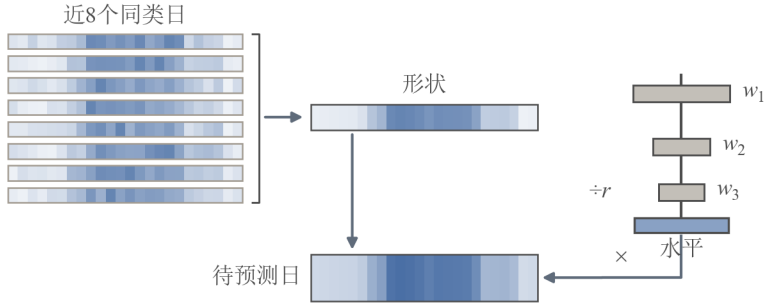


图 11　同类日形状乘近期水平：左边八条同类日的明暗带平均成形状，右边的尺子给出水平

图 11 中每一条明暗带是一天的负载曲线，颜色越深表示该时段负载越高；八条同类日的带子归一化后平均成一条形状带，右边尺子上三个刻度是最近三天的日总量，周末的那一天先除以类型比 r 折算到工作日，加权后的指针位置就是水平，形状带按水平

拉高即得待预测日。写成式子，设第 d 天之前同类日的集合为 $\mathcal{N}_d$，取其中最近的 8 天，则

$$\hat{L}_{d,t} = \frac{1}{|\mathcal{N}_d|} \sum_{j \in \mathcal{N}_d} \frac{L_{j,t}}{\sum_s L_{j,s}} \cdot \frac{\sum_{k=1}^{3} w_k \, \tilde{\Lambda}_{d-k}}{\sum_{k=1}^{3} w_k} \cdot r^{\,\mathbb{I}_{\mathrm{FS}}(d)}, \tag{9}$$

式（9）右端第一个因子是形状，第二个因子是水平，第三个因子把周末折算回来。其中 $\tilde{\Lambda}_j = \sum_s L_{j,s} / r^{\mathbb{I}_{\mathrm{FS}}(j)}$ 是第 j 天折算到工作日的日总量，$\mathbb{I}_{\mathrm{FS}}(d)$ 在第 d 天为周五或周六时取 1，r 是最近 28 天两类日子的日总量之比，权重取 $w = (0.6,\, 0.25,\, 0.15)$。光伏没有周内效应，逐日变化主要受天气影响，取最近 k 天的同段均值

$$\hat{P}^{\mathrm{pv}}_{d,t} = \frac{1}{k} \sum_{j=d-k}^{d-1} P^{\mathrm{pv}}_{j,t}, \qquad k = 5. \tag{10}$$

把 2 月 1 日至 12 月 31 日逐日向前评估，各种方法的精度如表 4 所示。

表 4　负载与光伏日前预测的精度对比

负载预测方法	绝对误差/kW	相对误差	光伏预测方法	绝对误差/kW	相对误差
同类日形状乘近期水平	**129.98**	**2.82%**	指数加权近 8 天	261.97	6.35%
谐波回归近 90 天	153.14	3.32%	**近 5 天均值**	**262.86**	**6.37%**
双向固定效应谐波回归	154.65	3.35%	近 3 天均值	267.97	6.49%
同星期几近 2 次均值	171.91	3.72%	形状乘水平	272.20	6.60%
同星期几指数加权	174.25	3.77%	近 10 天均值	277.15	6.72%
同星期几近 4 次均值	188.88	4.09%	谐波回归	304.40	7.38%
前一天	650.02	14.08%	前一天	321.56	7.79%
全年同段均值	803.65	17.41%	—	—	—

表 4 中负载最准的是同类日形状乘近期水平，比谐波回归低 0.53 个百分点，比同星期几近 2 次均值低 0.90 个百分点；不识别周内效应的全年同段均值误差高达 17.41%。光伏一栏各方法差别很小，最好的指数加权近 8 天与近 5 天均值只差 0.02 个百分点，本文取形式更简单的近 5 天均值。谐波回归在负载上排第三，它把年内趋势写进了模型，但对最近几天的整体抬升反应慢，而本文的水平项直接读最近三天的日总量，跟得更紧。

5.2.4　模型的建立

决策变量的建立。本问是两阶段决策。第一阶段在 0:00 决定全天的计划购电量 g_t；第二阶段在每个时段根据已揭晓的实际数据决定储能的充放电量 c_t、d_t 与紧急购电量 e_t。

目标函数的建立。总费用由计划购电费与紧急购电费构成，设紧急购电的价格倍数为 $\lambda = 5$，则

$$\min \; F = \sum_{t=1}^{T} p_t \, g_t + \lambda \sum_{t=1}^{T} p_t \, e_t. \tag{11}$$

约束条件的建立。供需约束在缺口处由紧急购电补齐，写成

$$g_t + e_t + v_t + d_t - c_t \geqslant l_t. \tag{12}$$

制定计划时用的是预测值，必须留出抵御误差的余地。常见的写法是把净负载按其绝对值加码，即用 $(1+\alpha)\,|\hat{l}_t - \hat{v}_t|$ 代替净负载，但这种写法在正午会失效：正午负载与光伏接近抵消，净负载接近零，加码也跟着接近零，而这正是光伏误差最大的时候。两种写法的差别如图 12 所示。

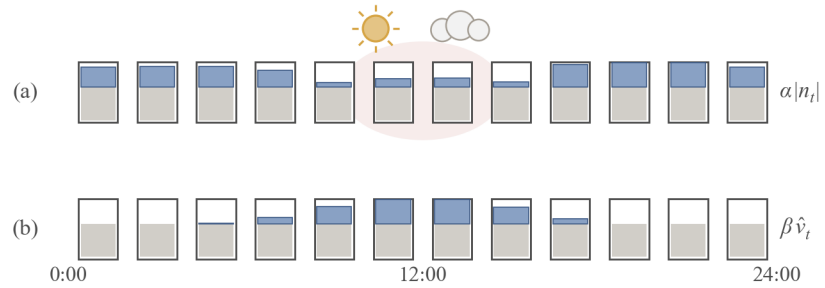


图 12　两种加码写法：(a) 按净负载的绝对值加码，正午几乎不加；(b) 按光伏预测折减，正午加得最多

图 12 中每个桶是两小时，桶上方的深色一层就是加码量。(a) 的加码量随净负载走，虚线圈出的正午几段几乎为零；(b) 把光伏按预测的 $1-\beta$ 计入，加码量随光伏走，正午最厚。因此本文按后一种写法给出计划用的净负载

$$\widehat{n}_t = \left[\hat{l}_t - (1-\beta)\,\hat{v}_t\right]\Delta t, \qquad \beta = 0.05. \tag{13}$$

第二条抵御途径是在储能里留一层不动用的备用容量。傍晚是全天电价最高、光伏已经落下的时段，缺口一旦发生代价最大，所以备用不必全天一样厚，如图 13 所示。

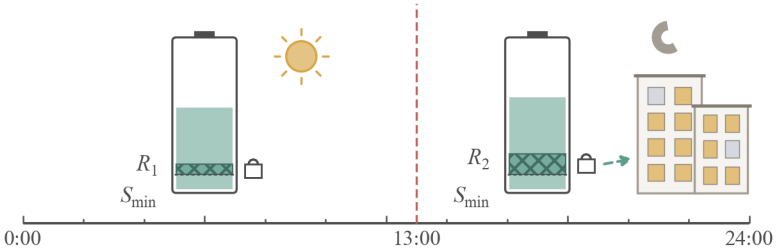


图 13　分段备用：13:00 之前锁住薄薄一层，13:00 之后加厚，留给傍晚的高价段

图 13 中锁住的一层是计划阶段不许动用的备用，13:00 之前取 $R_1 = 500$ kWh，之后取 $R_2 = 1000$ kWh。上午即使放空也还有整个白天的光伏可以补回来，傍晚则没有这样的机会，因此把厚的一层放在后半天。容量约束的下限相应抬高为 $S_{\min} + R_t$，运行阶段仍按真实下限 $S_{\min}$ 执行，备用只用来吸收预测误差。首尾条件放宽为期末储电量不低于参考水平 $S_{\text{ref}} = 6000$ kWh，避免每天强制回到期初把储能锁死。

综上所述，最终得到**基于光伏折减与分段备用的滚动计划购电模型**为

$$\begin{cases} \min\limits_{g,c,d,e} \displaystyle\sum_{t=1}^{T} p_t\, g_t + \lambda \sum_{t=1}^{T} p_t\, e_t \\ \text{s.t.} \begin{cases} g_t + e_t + d_t - c_t \geqslant \widehat{n}_t, & t = 1,\dots,T \\ S_{\min} + R_t \leqslant S_0 + \displaystyle\sum_{i=1}^{t}\left(\eta c_i - \frac{d_i}{\eta}\right) \leqslant S_{\max}, & t = 1,\dots,T \\ S_0 + \displaystyle\sum_{i=1}^{T}\left(\eta c_i - \frac{d_i}{\eta}\right) \geqslant S_{\text{ref}}, \\ R_t = R_1\ (t < 78), \qquad R_t = R_2\ (t \geqslant 78), \\ 0 \leqslant c_t \leqslant E_{\max},\ 0 \leqslant d_t \leqslant E_{\max},\ g_t \geqslant 0,\ e_t \geqslant 0, & t = 1,\dots,T \end{cases} \end{cases} \tag{14}$$

5.2.5　模型的求解与检验

模型（14） 每天在 0:00 解一次，日内用**滚动时域控制算法**逐段仿真，流程如图 14 所示。

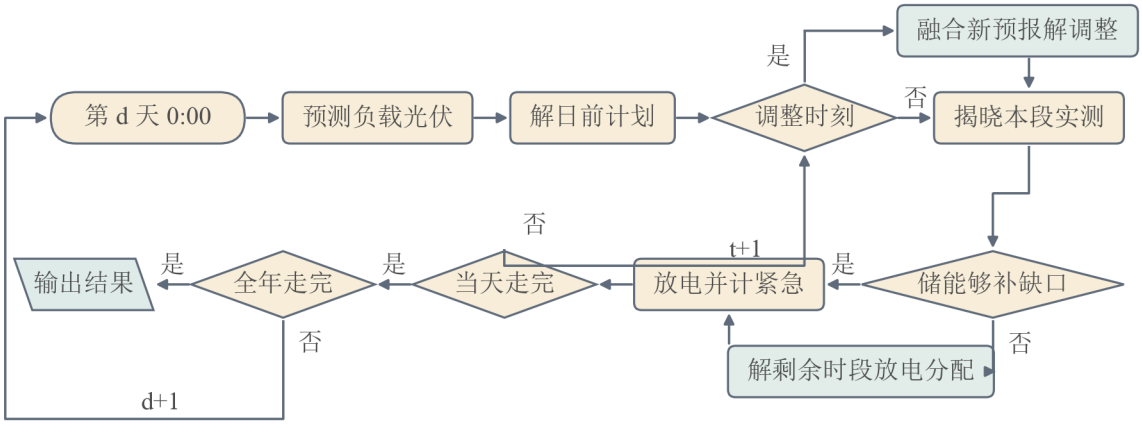


图 14　滚动时域控制的求解流程，上方的调整分支只在问题三启用

图 14 的第一行是日前计划，第二行是日内逐段运行，两条回边分别是段与段之间和天与天之间的循环。其中判断框储能够补缺口对应的实时放电规则如图 15 所示。

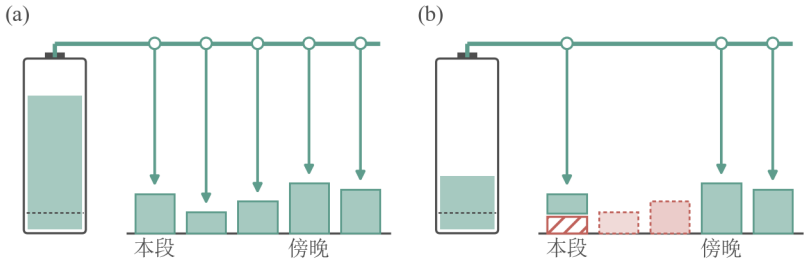


图 15　实时放电规则：(a) 能放出的电量够补后续全部缺口，本段缺多少放多少；(b) 不够时先保傍晚的高价段

图 15 中管子从储能接出，管上的阀门表示这一段放电。(a) 里可放电量 $(S_{t-1}-S_{\min})\,\eta$ 超过本段与后续各段预测缺口之和，直接把本段补满；(b) 里不够，于是在剩余时段上重解一次只优化放电的子问题，把电量让给傍晚颜色更深也就是电价更高的两段，本段只补一部分，剩下的斜纹部分计入紧急购电。求解步骤如下。

Step1　由式（9） 与式（10） 给出第 d 天的点预测。
Step2　按式（13） 折减光伏，得到计划用的净负载。
Step3　按分段备用抬高储电量下限，解模型（14） 得到当天的计划购电量并锁定。
Step4　逐段揭晓实际负载与光伏，按图 15 的规则决定本段放电量。
Step5　富余时按可用容量充电，缺口由放电填补，仍有缺口计入紧急购电。
Step6　更新储电量进入下一段，走完当天把期末储电量作为下一天的期初值。
Step7　从 1 月 1 日滚动到年末，统计 2 月 1 日以后的费用，1 月作为预测模型的历史。

日前计划子问题同样做多算法对比。在 2 月至 12 月每月 15 日共 11 天上各解一次，随机性算法各独立运行 10 次，结果如表 5 所示。

表 5 的平均目标值是 11 天的平均，启发式一列取每天 10 次运行中的最好值。三种启发式 10 次运行的均值分别为模拟退火 41092.97 元、粒子群 38935.45 元、遗传算法 40059.64 元。模拟退火单次用时 8.486 s，是对偶单纯形法的 229 倍，全年 334 天累计约 47 分钟，而对偶单纯形法累计约 12 s。精确解法既快又能给出可证明的最优解，因此问题二仍采用对偶单纯形法。

抵御误差的两条途径与其参数通过全年对照确定，结果如表 6 所示。

表 5　问题二日前计划子问题的多算法对比

算法	类别	平均目标值/元	相对最优	命中最优比例	单次用时/s
对偶单纯形法	**精确**	**36290.4293**	**0.000%**	**100%**	**0.037**
内点法	精确	36290.4293	0.000%	100%	0.064
动态规划	精确	36360.2177	0.223%	27%	0.432
模拟退火算法	启发式	37307.6305	4.148%	3%	8.486
粒子群算法	启发式	37534.7443	4.399%	0%	1.765
遗传算法	启发式	38417.4801	8.098%	0%	1.362
谷充峰放规则法	基准	43543.6229	24.409%	0%	0.001

表 6　问题二的策略对照，统计区间为 2025 年 2 月 1 日至 12 月 31 日

策略	总费用/元	计划费/元	紧急费/元	紧急电量/kWh	紧急天数
事后最优下界	12227243.64	12227243.64	0.00	0	0
逐日完美预测	12245046.92	12245046.92	0.00	0	0
折减 5% 配分段备用	**13492783.13**	**12962970.08**	**529813.05**	**100467**	**173**
折减 5% 配备用 1000	13494794.08	12996526.80	498267.29	91113	156
场景平均近似规划	13529333.88	12886929.43	642404.45	113899	89
换用谐波回归预测	13685893.84	12767122.29	918771.55	165019	170
只按净负载加 1.5% 裕度	13941534.00	12793867.33	1147666.67	206250	163
不做任何抵御	14563713.31	12204070.93	2359642.38	469747	330

表 6 给出四条结论。第一，不做任何抵御时紧急购电费占总费用的 16.20%，比本文策略多花 1070930.18 元，说明忽略预测误差的代价很高。第二，把加码方式从净负载裕度换成光伏折减，总费用由 13941534.00 元降到 13494794.08 元，降幅 3.20%，这正是图 12 指出的正午失效被补上。第三，备用由全天 1000 kWh 改成 13:00 前后 500 与 1000 kWh，费用再降 2010.95 元；此时紧急购电的天数由 156 天升到 173 天、电量由 91113 kWh 升到 100467 kWh，但缺口落在电价较低的时段，总费用反而更低。第四，把日前计划换成对 20 个历史误差场景的样本平均近似规划，费用反而高出 36550.75 元，原因如图 16 所示。

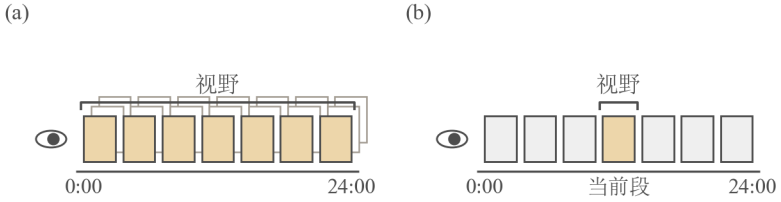


图 16　两种信息条件：(a) 场景规划里每个场景都看得到当天后面的全部数据；(b) 实际运行只看得到当前这一段

图 16(a) 中每条灰线是一个误差场景，场景内的充放电决策对这条线后面的部分有完全的预见，于是模型认为缺口总能提前避开，留的余量偏少；(b) 是真实运行的信息条件，只有当前这一段是已知的。本文的评价口径一律取因果仿真的真实费用，所以场景规划在评价时吃了亏。这也说明报告的费用必须来自因果仿真，不能取规划模型自身的目标值。

模型的检验分两步。第一步是两侧夹逼：全年事后最优下界为 12227243.64 元，逐日完美预测为 12245046.92 元，两者只差 0.15%，说明滚动框架本身几乎没有损失；本文策

略比完美预测高 10.19%，这一段差距正是预测误差的代价，靠调度手段消不掉。第二步是核对结果文件：result2.xlsx 共 334 天，逐日的 0:00 与 24:00 储电量都落在 $[1200, 10800]$ kWh 内，实际取值范围为 3424.4 至 9230.1 kWh，相邻两天的期末与期初储电量完全衔接，最大偏差为 0，紧急购电量与逐段供需缺口逐日一致。两步检验都通过，即模型的可靠性高。

全年滚动的结果如图 17 所示，指定日期的逐段结果见附录 B 的表 10、表 11与表 12。

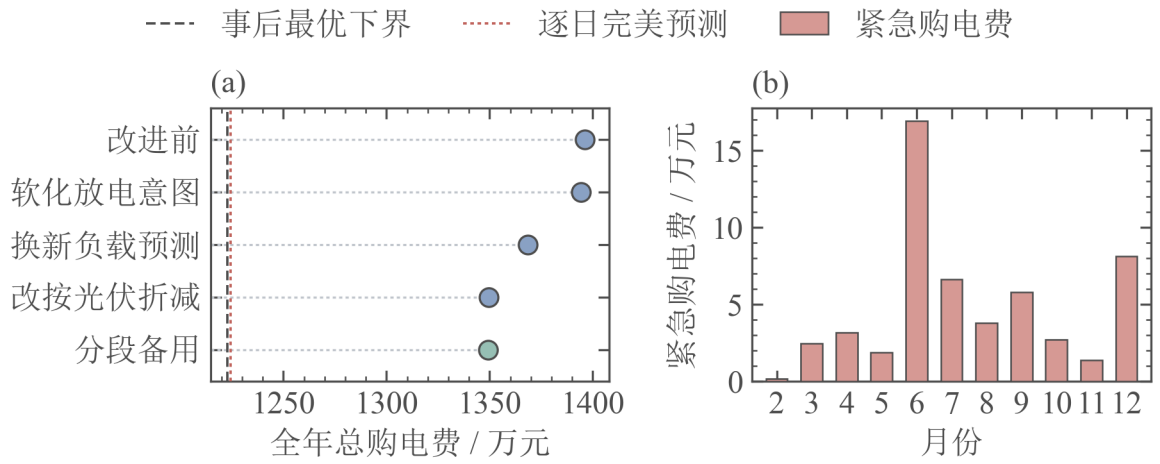


图 17　问题二的全年运行：(a) 抵御手段逐项叠加后的总费用；(b) 逐月紧急购电费

图 17(a) 自上而下是抵御手段逐项叠加后的全年总费用，从改进前的 13960009.68 元一路降到 13492783.13 元，两条竖虚线分别是事后最优下界与逐日完美预测；图 17(b) 中紧急购电费集中在 6 月与 12 月，分别为 16.9 万元与 8.1 万元，6 月是光伏波动最大的月份，12 月是负载全年最高的月份，两者合计占全年紧急购电费的 47%。全年 334 天的总购电费为 13492783.13 元，其中计划购电费 12962970.08 元占 96.07%，紧急购电费 529813.05 元占 3.93%，紧急购电发生在 173 天。与逐日完美预测的 12245046.92 元相比只高 10.19%，而完美预测与全年事后最优仅差 0.15%，说明差距几乎全部来自不确定性本身，而不是滚动框架。

问题二在 2025 年 2 月 1 日至 12 月 31 日的总购电费为 13492783.13 元，其中计划购电费 12962970.08 元、紧急购电费 529813.05 元，比不做任何抵御的 14563713.31 元节省 7.35%，完整策略见支撑材料中的 result2.xlsx。

5.3　问题三模型的建立与求解

5.3.1　问题三的思路框架

问题三在问题二的基础上增加了三次滚动调整，求解通路如图 18 所示。

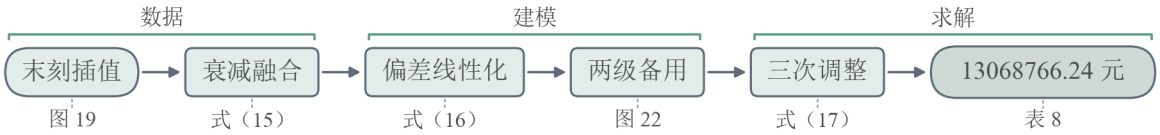


图 18　问题三的思路框架

图 18 的数据一段处理的是附件 3 的整点预报，建模一段多出偏差计价与两级备用，求解一段是在问题二的滚动框架里插入三次调整并逐档对照。这一问最终的总购电费是 13068766.24 元。

5.3.2　机理分析

本问多出来的信息是附件 3 的整点预报，它一天发布四次，每次给出未来 24 小时整点的光伏功率。要把它用进按 10 分钟分段的模型，先要解决取值时刻的问题，如图 19 所示。

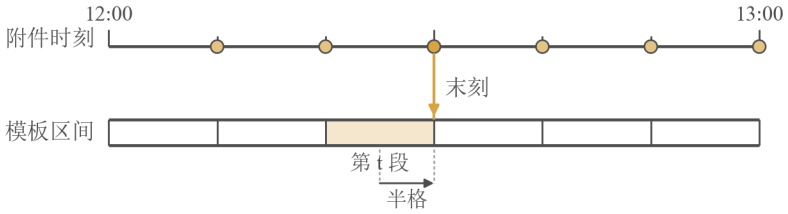


图 19　附件时刻与模板区间差半格：附件的第几个数给的是第几段末刻的功率

图 19 上面一把尺子标的是附件给出数据的时刻，下面一把标的是结果模板的时间段，两把尺子差半格：第 t 个数对应的是第 t 段的末刻而不是中点。插值按末刻取值，12:00 发布的预报误差由按中点取值的 4.70% 降到 3.68%；再把发布时刻已经实测到的功率作为插值的左端点，误差进一步降到 3.24%。

官方预报并非越用越好。逐个发布时刻统计只用官方预报的相对误差，0:00 为 8.30%，反而不如近 5 天历史均值的 6.37%；而 12:00 的预报误差只有 3.24%，明显好过历史。按提前小时数分档看得更清楚：0:00 发布的预报在提前 6 到 9 小时时误差 6.83%，与历史的 6.79% 相当，提前 15 到 18 小时则升到 10.82%，而历史均值不随提前量变化。因此融合权重应随提前量衰减，如图 20 所示。

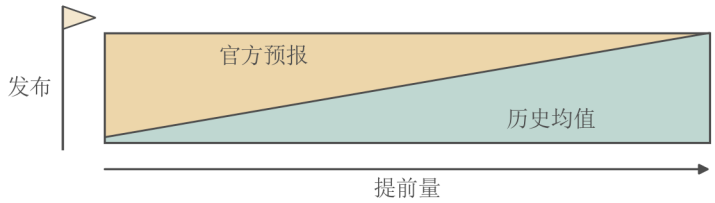


图 20　融合权重随提前小时数衰减：上半是官方预报的份额，下半是历史均值的份额

图 20 中带子的左端是发布时刻，越往右提前量 L 越大，官方预报的份额 w 越薄，到 19 小时以后完全交给历史均值。据此建立**按提前量衰减的预报融合模型**

$$\hat{P}^{\mathrm{pv}}_{d,t}(h) = w(L)\,\widetilde{P}^{\mathrm{pv}}_{d,t}(h) + \big[1-w(L)\big]\frac{1}{k}\sum_{j=d-k}^{d-1} P^{\mathrm{pv}}_{j,t}, \qquad w(L) = \max\{0,\, 0.95-0.05L\}, \tag{15}$$

其中 $\widetilde{P}^{\mathrm{pv}}_{d,t}(h)$ 是第 h 次发布的整点预报按末刻插值的结果，$L = t\Delta t - h_0$ 是该时段相对发布时刻 h_0 的提前小时数。三个发布时刻的融合误差分别为 5.46%、4.15% 与 3.10%，都优于两个单独来源，也优于按发布时刻取常数权重的最好结果 5.49%、4.33% 与 3.18%。

5.3.3　模型的建立

决策变量的建立。在问题二的基础上增加调整购电量 a_t，并把偏差分解为超额量 u_t 与违约量 z_t 两组非负变量。

目标函数的建立。按题目规定，计划量高于调整量的部分按 50% 计违约费用，调整量高于计划量的部分按 1.5 倍计费，因此总费用为

$$F = \sum_t p_t\, g_t + \frac{3}{2}\sum_t p_t\, u_t - \frac{1}{2}\sum_t p_t\, z_t + \lambda \sum_t p_t\, e_t, \qquad a_t - g_t = u_t - z_t. \tag{16}$$

式（16）中 $\sum p_t g_t$ 在调整时刻已经是常数，因此调整模型的目标只剩后三项，仍是线性函数。这一段计价规则的形状如图 21 所示。

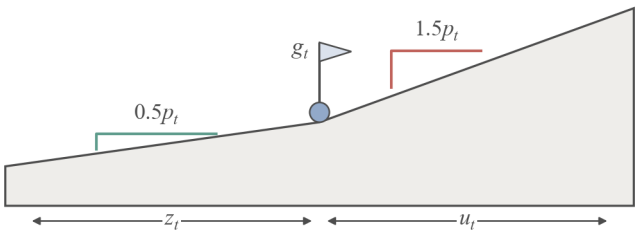


图 21　偏差计价是一段折坡：以计划量为折点，往左调低退回半价，往右调高加收一倍半

图 21 中坡面的高度是当天的购电支出，折点落在计划量 g_t 处。往左调低是缓坡，每少买一度只退回 $0.5p_t$；往右调高是陡坡，每多买一度要付 $1.5p_t$。两侧单价不同正是要把偏差拆成 u_t 与 z_t 两组非负变量的原因：拆开之后目标对两者都线性，且由于陡坡单价高于缓坡，最优解不会让两者同时为正。

调整时刻的预报比 0:00 更准，不确定性变小，备用也可以相应减薄，如图 22 所示。

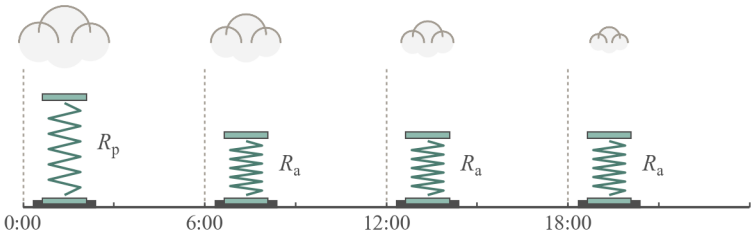


图 22　两级备用：0:00 的计划垫得厚，随后三次调整时云变小，垫子减薄

图 22 中每根弹簧的高度是那一次决策留出的备用，上方的云表示剩余时段的不确定性。0:00 的计划备用取 $R_\mathrm{p} = 500\ \mathrm{kWh}$，三次调整的备用取 $R_\mathrm{a} = 250\ \mathrm{kWh}$。净负载另加 $\alpha = 1.5\%$ 的裕度，与问题二按光伏折减不同，这里已经有了新的预报，剩下的误差更像是随机扰动，按净负载加码更合适。

综上所述，设 τ 是调整时刻对应的时段序号，最终得到**基于偏差费用的日内滚动调整模型**为

$$
\begin{cases}
\min\limits_{a,c,d,e,u,z} \ \dfrac{3}{2}\sum\limits_{t=\tau}^{T} p_t u_t - \dfrac{1}{2}\sum\limits_{t=\tau}^{T} p_t z_t + \lambda \sum\limits_{t=\tau}^{T} p_t e_t \\
\text{s.t.} \begin{cases}
a_t + e_t + d_t - c_t \geqslant \widehat{n}_t(h), & t = \tau, \dots, T \\
a_t - g_t = u_t - z_t, \quad u_t \geqslant 0,\ z_t \geqslant 0, & t = \tau, \dots, T \\
S_{\min} + R_\mathrm{a} \leqslant S_{\tau-1} + \sum\limits_{i=\tau}^{t}\left(\eta c_i - \frac{d_i}{\eta}\right) \leqslant S_{\max}, & t = \tau, \dots, T \\
S_{\tau-1} + \sum\limits_{i=\tau}^{T}\left(\eta c_i - \frac{d_i}{\eta}\right) \geqslant S_{\mathrm{ref}}, & \\
0 \leqslant c_t \leqslant E_{\max},\ 0 \leqslant d_t \leqslant E_{\max},\ a_t \geqslant 0,\ e_t \geqslant 0, & t = \tau, \dots, T
\end{cases}
\end{cases}
\tag{17}
$$

其中 $\widehat{n}_t(h) = \left[\hat{l}_t - \hat{P}^{\mathrm{pv}}_{d,t}(h)\Delta t\right](1+\alpha)$ 是用第 h 次发布的融合预测算出并加码后的净负载。

5.3.4　模型的求解与检验

求解在问题二的滚动框架中插入三次调整[8]，即在 $\tau = 37$、73 与 109 三个时段各解一次模型（17），把剩余时段的购电量替换为调整量，偏差费用相对 0:00 的计划量计算。

调整子问题的多算法对比结果如表 7 所示。

表 7　问题三日内调整子问题的多算法对比

算法	类别	平均目标值/元	相对最优	命中最优比例	单次用时/s
对偶单纯形法	**精确**	**35477.6600**	**0.000%**	**100%**	**0.045**
内点法	精确	35477.6600	0.000%	100%	0.077
动态规划	精确	35659.7820	0.581%	0%	0.665
模拟退火算法	启发式	39003.6682	12.072%	0%	7.457
粒子群算法	启发式	40655.5143	17.117%	0%	0.945
遗传算法	启发式	41219.1999	21.148%	0%	1.036
谷充峰放规则法	基准	54555.3709	60.255%	0%	0.001

表 7 的口径与表 5 相同，平均目标值是 11 天的平均，启发式一列取每天 10 次运行中的最好值。与表 5 相比，启发式的差距由 4.148% 至 8.098% 扩大到 12.072% 至 21.148%，规则法扩大到 60.255%。原因在于调整子问题多了偏差费用，目标函数在计划量处出现折点，随机搜索在折点两侧反复试探代价很高；而精确解法把绝对值拆成两组非负变量后仍是线性规划，用时 0.045 s，因此问题三同样采用对偶单纯形法。

为回答是否需要引入其他时刻的预报，按调整档位逐档求解，结果如表 8 所示。

表 8　问题三的调整档位对照，统计区间为 2025 年 2 月 1 日至 12 月 31 日

调整档位	总费用/元	计划费/元	调整费/元	紧急费/元	紧急电量/kWh
只用 0:00 预报	13496984.66	12621649.06	0.00	875335.59	162909
加 6:00	13353137.48	12600868.83	144349.03	607919.62	115015
加 6:00、12:00	13114289.88	12595759.01	145108.66	373422.21	73162
加 6:00、12:00、18:00	**13068766.24**	**12588139.82**	**202545.47**	**278080.95**	**58380**

对题目所问是否需要引入其他时刻的预报，回答是需要，且三个时刻都要用上。依据有三条。第一，每增加一个调整时刻总费用都下降，从只用 0:00 预报的 13496984.66 元依次降到 13353137.48 元、13114289.88 元与 13068766.24 元，三次调整全开比不调整节省 428218.42 元，降幅 3.17%，中间没有出现增加时刻反而变贵的拐点。第二，调整要付代价，但代价远小于收益：调整相关费用由 0 元升到 202545.47 元，同期紧急购电费由 875335.59 元降到 278080.95 元，紧急购电费的减少量是调整费用增加量的 2.95 倍。第三，收益来自预报精度的提升，18:00 那一次覆盖的正是傍晚的高价段，加上它之后紧急购电量由 73162 kWh 再降到 58380 kWh，是四个档位里最低的。模型的检验采用条件弱化法。把三次调整全部关掉，模型（17） 应退化为一个只用 0:00 预报的日前模型，此时偏差费用恒为零，实际计算得到的调整相关费用为 0.00 元，与理论一致；调整档位的费用序列单调下降，也与偏差费用非负的性质相符。

问题三的结果如图 23 所示，指定日期的逐段结果见附录 B 的表 13、表 14与表 15。

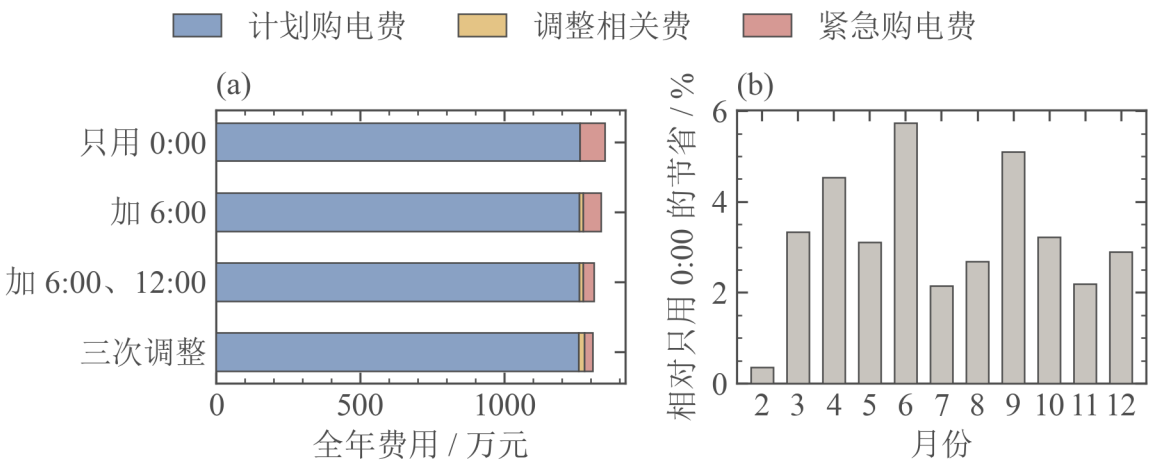


图 23　问题三的滚动调整：(a) 各档位的费用构成；(b) 三次调整相对只用 0:00 预报的逐月节省

图 23(a) 中随着档位增加，紧急购电费这一段迅速收窄，调整费这一段缓慢加宽，总长度持续变短；图 23(b) 中逐月节省全部为正，6 月与 10 月最大，正是光伏波动最大的两个月，说明调整的价值主要来自把光伏预报的改进及时用进购电计划。

> 问题三在三次调整全开时的总购电费为 13068766.24 元，比只用 0:00 预报的 13496984.66 元节省 3.17%，比问题二节省 3.14%；需要引入其他时刻的预报，且 6:00、12:00 与 18:00 三个时刻都要用上，完整策略见支撑材料中的 result3.xlsx。

5.4　问题四模型的建立与求解

5.4.1　问题四的思路框架

问题四的求解通路如图 24 所示。



图 24　问题四的思路框架

图 24 的数据一段换成附件 4 的逐日电价，建模一段只需说明可行域不变，求解一段是把问题二与问题三各重算一遍并做退化检验。这一问最终的总购电费是 14130588.56 元与 13689275.77 元。

5.4.2　机理分析

统计附件 4 得到，全年电价的取值范围为 0.0076 至 1.7936 元/kWh，日内极差的均值为 1.1073 元/kWh，而附件 1 的固定电价日内极差为 1.0239 元/kWh，前者是后者的 1.08 倍。波动电价同时放大两件事：低谷更低使储能套利更划算，尖峰更高使同样一度紧急购电的罚金更重。对模型而言真正的变化只有一处，如图 25 所示。

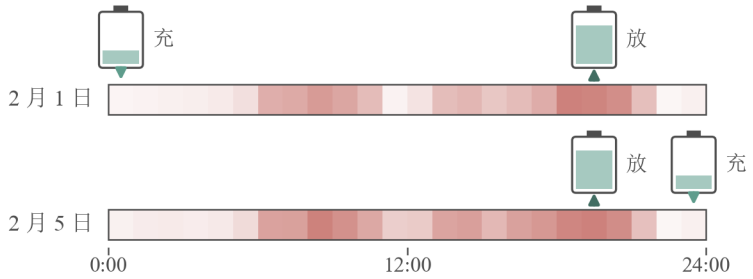


图 25　两天的电价形状不同，规则没变：储能仍在各自的低价段充电、高价段放电

图 25 中两条色带是两天的电价，颜色越深电价越高，两天的低价段位置差了好几个小时，而充放电的做法完全一样：在各自的低价段充、高价段放。原因是电价只以系数向量的形式出现在目标函数与偏差费用中，不改变任何一条约束的结构与右端项，模型的可行域与电价无关，只是目标函数的方向变了。

5.4.3　模型的建立

决策变量的建立。与问题三相同，第一阶段是 0:00 的计划购电量 g_t，第二阶段是三次调整后的购电量 a_t、储能的充放电量 c_t 与 d_t、紧急购电量 e_t，以及偏差分解出的超额量 u_t 与违约量 z_t。

目标函数的建立。把问题三目标中的固定电价 p_t 换成第 d 天的实时电价 $p_{d,t}$，四项计价的倍数不变。

约束条件的建立。供需约束、储电量的上下限与备用、偏差分解的等式、充放电功率上限与各变量的非负性全部沿用模型（17），一条不改，因为电价不出现在任何一条约束里。

综上所述，最终得到**波动电价下的滚动调控模型**为

$$\begin{cases} \min \ \displaystyle\sum_t p_{d,t}\, g_t + \frac{3}{2}\sum_t p_{d,t} u_t - \frac{1}{2}\sum_t p_{d,t} z_t + \lambda \sum_t p_{d,t} e_t \\ \text{s.t.}\ \ \text{同模型（17）的全部约束} \\ \qquad p_{d,t} \text{ 取自附件 4 第} d \text{ 天第} t \text{ 段的实时电价} \end{cases} \tag{18}$$

其中不做调整时令 $u_t = z_t = 0$，模型退化为波动电价下的问题二。

5.4.4　模型的求解与检验

求解沿用问题二与问题三的滚动框架，只替换电价数据，两种情形各跑一遍全年，结果如表 9 所示。

表 9　固定电价与波动电价的结果对照，统计区间为 2025 年 2 月 1 日至 12 月 31 日

情形	电价	总费用/元	紧急费/元	紧急电量/kWh	调整的节省
问题二，不调整	固定	13492783.13	529813.05	100467	—
问题三，不调整	固定	13496984.66	875335.59	162909	基准
问题三，三次调整	固定	13068766.24	278080.95	58380	3.17%
问题四对应问题二	**波动**	**14130588.56**	**588758.55**	**104764**	**—**
问题四对应问题三，不调整	波动	14144510.18	952276.14	169319	基准
问题四对应问题三，三次调整	**波动**	**13689275.77**	**297660.10**	**58262**	**3.22%**
波动电价下不做任何抵御	波动	15273618.40	2511998.65	491814	—

表 9 给出三条结论。第一，波动电价下两种情形的总费用都高于固定电价，分别高出 4.73% 与 4.75%。按结果文件分解，问题二的费用上升 637805.43 元中有 578859.93 元来自计划购电费，只有 58945.50 元来自紧急购电费，可见上升的主因是电价本身的分布变了，而不是罚金变多。第二，波动电价下不做任何抵御的总费用为 15273618.40 元，紧

急购电费高达 2511998.65 元，是本文策略的 8.44 倍，说明电价越不稳，忽略预测误差的代价越高。第三，在同一电价下比较，三次滚动调整相对只用 0:00 预报的节省由固定电价下的 3.17% 上升到波动电价下的 3.22%，即电价波动越剧烈，滚动调整越值钱。表中问题三只用 0:00 预报的费用略高于问题二不调整，两者的信息条件相同，差别在于问题三的计划口径是净负载加 1.5% 裕度、计划备用统一取 500 kWh，而问题二是光伏折减 5% 加分段备用；前者是为配合日内调整定的，单看不调整的情形并不占优。模型的检验采用条件弱化法。把附件 4 的电价替换成它的全年逐段均值，此均值与附件 1 的固定电价最大绝对差为 5.2×10^{-5} 元/kWh，模型（18） 应退化为问题三的模型。实际计算得到的总费用为 13068764.79 元，问题三为 13068766.24 元，两者相差 1.45 元，相对差 1.1×10^{-7}，吻合度接近 100%，即模型的可靠性高。

问题四的结果如图 26 所示，指定日期的逐段结果见附录 B 的表 16、表 17、表 18与表 19、表 20、表 21。

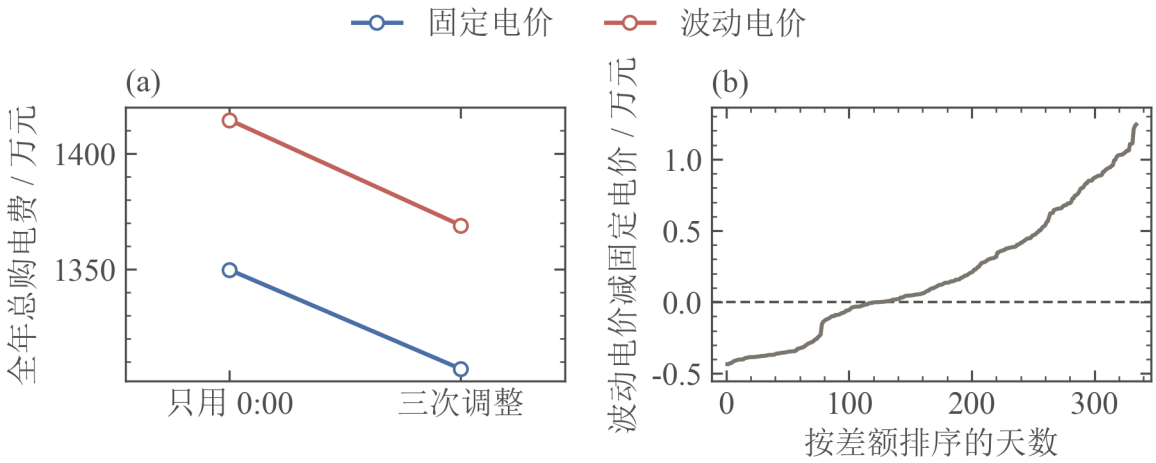


图 26　问题四的结果：(a) 两种电价下调整前后的总费用；(b) 逐日费用差按大小排序

图 26(a) 中两条线的落差是调整带来的绝对节省，波动电价那条落得更多，相对节省则见表 9 的末列；图 26(b) 把 334 天的费用差从小到大排开，绝大多数日子的差额集中在中间一段，两端各有十来天差得较多，对应电价尖峰与光伏大幅波动同时出现的日子。

> 波动电价下，问题二对应的总购电费为 14130588.56 元，问题三对应的总购电费为 13689275.77 元；三次调整相对只用 0:00 预报的节省由固定电价下的 3.17% 升到 3.22%，完整策略见支撑材料中的 result4-2.xlsx 与 result4-3.xlsx。

6、　模型的灵敏度分析

抵御预测误差的两个参数是光伏折减比例 β 与储能备用容量 R，问题三另有计划备用与调整备用的配比和净负载裕度 α。把每个参数在最优值附近逐档变动，其余参数不动，重跑全年仿真，结果如图 27 所示。

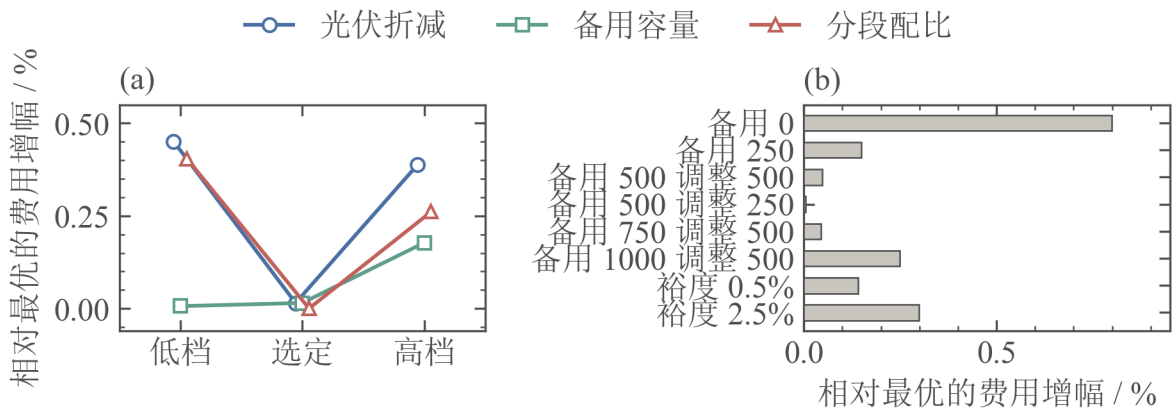


图 27　灵敏度分析：(a) 问题二的折减比例、备用容量与分段配比；(b) 问题三的两级备用与净负载裕度

图 27(a) 中三组参数的费用曲线都在最优点附近平坦：折减比例由 5% 改为 3% 或 7%，总费用分别上升 0.44% 与 0.37%；备用在分段配比之外单取一个常数，由 1000 kWh 改为 750 或 1250 kWh，变化在 0.17% 以内；分段配比由 500 与 1000 kWh 改成全天 1250 kWh 或改成上午厚下午薄，分别上升 0.40% 与 0.26%，可见把厚的一层放在后半天是对的。图 27(b) 中问题三的计划备用取 500 kWh、调整备用取 250 kWh 时最低，备用取零时上升 0.80%，取 1000 kWh 时上升 0.67%，两侧都变差，说明备用过少挡不住误差、过多则挤掉套利空间；净负载裕度由 1.5% 改为 0.5% 或 2.5%，费用分别上升 0.09% 与 0.25%。

综合来看，本文模型对这几个参数都不敏感：在选定值两侧各一档的范围内总费用变化不超过 0.5%，把备用取到零或加倍这样的极端档位也只上升 0.80% 与 0.67%，参数取值不必精调；但把抵御手段全部取消时问题二的总费用上升 7.35%，说明抵御误差这件事本身是必须做的。

7、　模型的评价、改进与推广

7.1　模型的优点

1. 四问共用一套记号与一个线性规划内核，问题三与问题四只在目标函数上增加项或替换系数，约束结构完全不变，因此模型之间可以互相退化验证，问题四换成年均电价后与问题三的相对差只有 1.1×10^{-7}。
2. 把储电量按下三角矩阵累加消元，使含状态递推的调度问题保持线性规划，并证明了最优解自动满足同段不同时充放电，避免引入 0-1 变量，单日求解用时 0.026 s。
3. 把抵御预测误差拆成光伏折减与分段备用两条途径并联合寻优，比不做任何抵御节省 7.35%；用分段备用替代全天等厚的备用，又多省 2010.95 元。
4. 计划用预测数据制定，运行用因果的滚动仿真评价，报告的费用全部来自仿真，不存在对未来的预见；并给出事后最优下界与逐日完美预测两个参照，把改进空间说清楚。

7.2　模型的缺点

1. 折减比例与备用容量是用历史数据整体寻优得到的常数，没有随季节或天气变化。紧急购电费仍占问题二全年总购电费的 3.93%，其中六成来自实时调度有意保留电量，这部分与后续缺口的预测精度直接相关。
2. 假设波动电价在日前已经公布。如果电价本身也需要预测，模型还要增加一层价格预测与相应的风险约束。

7.3 模型的改进

把折减比例写成当日预报离散度的函数，在预报分歧大的日子自动加厚，可以进一步压低紧急购电费；把储能的循环寿命折算成度电成本计入目标函数，可以让套利决策同时考虑设备损耗。

7.4 模型的推广

本模型可以推广到两类场景。一是含电动汽车充电桩的园区微网，把充电负载的可转移部分写成新的决策变量，模型仍是线性规划[9]；二是含风电的多能互补系统，风电的预测误差比光伏更大，本文的折减与备用联合寻优框架可以直接沿用，只需重新标定两个参数。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于借助 DeepSeek 进行代码调试与论文文字润色，详细使用情况见支撑材料。

参考文献

[1] 王成山, 武震, 李鹏. 微电网关键技术研究 [J]. 电工技术学报, 2014, 29(2): 1-12.

[2] 杨新法, 苏剑, 吕志鹏, 等. 微电网技术综述 [J]. 中国电机工程学报, 2014, 34(1): 57-70.

[3] 丁明, 王伟胜, 王秀丽, 等. 大规模光伏发电对电力系统影响综述 [J]. 中国电机工程学报, 2014, 34(1): 1-14.

[4] 李建林, 马会萌, 惠东. 储能技术融合分布式可再生能源的现状及发展趋势 [J]. 电工技术学报, 2016, 31(14): 1-10.

[5] 刘畅, 卓建坤, 赵东明, 等. 利用储能系统实现可再生能源微电网灵活安全运行的研究综述 [J]. 中国电机工程学报, 2020, 40(1): 1-18.

[6] Parisio A, Rikos E, Glielmo L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813-1827.

[7] Palma-Behnke R, Benavides C, Lanas F, et al. A microgrid energy management system based on the rolling horizon strategy[J]. IEEE Transactions on Smart Grid, 2013, 4(2): 996-1006.

[8] Olivares D E, Mehrizi-Sani A, Etemadi A H, et al. Trends in microgrid control[J]. IEEE Transactions on Smart Grid, 2014, 5(4): 1905-1919.

[9] 国家能源局. 分布式光伏发电项目管理暂行办法 [S]. 北京: 国家能源局, 2013.

[10] Hong T, Fan S. Probabilistic electric load forecasting: a tutorial review[J]. International Journal of Forecasting, 2016, 32(3): 914-938.

附录

附录 A 支撑材料文件列表

```
支撑材料/
  AI工具使用详情.pdf
```

```
结果/
  result1.xlsx  result2.xlsx  result3.xlsx  result4-2.xlsx  result4-3.xlsx
代码/
  Q1/  q1_求解.py        q1_算法对比.py
  Q2/  q2_求解.py        q2_算法对比.py     q2_预测精度对比.py
       q2_策略对照.py     q2_事后下界.py
  Q3/  q3_求解.py        q3_算法对比.py     q3_预报精度对比.py
       q3_档位对照.py
  Q4/  q4_求解.py        q4_对照.py
```

四问的代码各放一个文件夹，每个文件都可以单独运行。运行前在文件顶部的 DATA 与 OUT 两行填入附件目录与输出目录，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，两者都留空时程序从自身位置逐级向上查找附件目录。全部程序依赖 numpy、scipy、pandas 与 openpyxl。建模与求解用到的程序已全部列在上面；画图与出表脚本只用来生成插图与表格，不影响结果的复现，未放入支撑材料。

附录 B　后三问指定日期的结果表

下面三组表按题目表 1、表 2 与表 3 的格式给出问题二至问题四在 2025 年 3 月 20 日、6 月 21 日、9 月 23 日与 12 月 21 日四天的结果，取自五个 result 文件的同一位置。问题三与问题四对应问题三的购电量一列给的是三次调整后的最终取用量，全天购电费一列是当天的总费用，含紧急购电费与调整相关费。

表 10　问题二指定日期的购电量及全天的购电量和购电费

日期	时间段	购电量	时间段	购电量	时间段	购电量
2025.03.20	10:00-10:10	0.0000	12:00-12:10	626.0275	14:00-14:10	0.0000
	16:00-16:10	452.7116	18:00-18:10	709.9796	20:00-20:10	0.0000
	全天购电量 67902.4535			全天购电费 41788.3242		
2025.06.21	10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
	16:00-16:10	110.7647	18:00-18:10	401.7310	20:00-20:10	0.0000
	全天购电量 34762.3417			全天购电费 19861.9054		
2025.09.23	10:00-10:10	0.0000	12:00-12:10	515.8098	14:00-14:10	0.0000
	16:00-16:10	487.2083	18:00-18:10	771.7427	20:00-20:10	0.0000
	全天购电量 67484.5762			全天购电费 43158.4910		
2025.12.21	10:00-10:10	0.0000	12:00-12:10	967.2855	14:00-14:10	0.0000
	16:00-16:10	795.9118	18:00-18:10	713.6978	20:00-20:10	0.0000
	全天购电量 94946.1805			全天购电费 61678.2790		

表 11 问题二指定日期储能设备的充放电量及 0:00 和 24:00 的储电量

日期	时间段	充电量	放电量	时间段	充电量	放电量
2025.03.20	0:00-4:00	4511.1185	215.8252	4:00-8:00	1232.1368	4920.0911
	8:00-12:00	6224.9944	2777.3772	12:00-16:00	3414.9042	404.2905
	16:00-20:00	102.5001	5290.6312	20:00-24:00	4447.9643	2905.7494
	0:00 储电量 6030.7351			24:00 储电量 5622.1420		
2025.06.21	0:00-4:00	76.3974	519.4083	4:00-8:00	408.0049	2208.8562
	8:00-12:00	9800.5756	0.0000	12:00-16:00	0.0000	0.0000
	16:00-20:00	59.2086	5422.8971	20:00-24:00	4203.9853	2693.4222
	0:00 储电量 4574.9249			24:00 储电量 5618.7419		
2025.09.23	0:00-4:00	5322.2040	108.0666	4:00-8:00	927.5102	5946.0450
	8:00-12:00	5091.0988	1896.6157	12:00-16:00	4787.2245	557.6975
	16:00-20:00	26.1628	5803.9150	20:00-24:00	4390.4020	2508.0215
	0:00 储电量 5325.3042			24:00 储电量 5126.1560		
2025.12.21	0:00-4:00	4543.1998	228.7218	4:00-8:00	1805.7886	1554.7940
	8:00-12:00	5581.4634	6933.2623	12:00-16:00	6762.2002	2249.9093
	16:00-20:00	20.8596	5256.5424	20:00-24:00	4360.4153	2700.2442
	0:00 储电量 6161.8007			24:00 储电量 5902.2528		

表 12 问题二指定日期的紧急购电量

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.0000	无	0.0000	20:30-20:40	34.0200	无	0.0000
—	—	—	—	21:00-21:40	82.1224	—	—
—	—	—	—	21:50-22:00	11.0338	—	—

表 13 问题三指定日期的购电量及全天的购电量和购电费

日期	时间段	购电量	时间段	购电量	时间段	购电量
2025.03.20	10:00-10:10	0.0000	12:00-12:10	482.3339	14:00-14:10	0.0000
	16:00-16:10	462.8194	18:00-18:10	720.6293	20:00-20:10	0.0000
	全天购电量 66065.8182			全天购电费 43434.8278		
2025.06.21	10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
	16:00-16:10	43.1742	18:00-18:10	399.3384	20:00-20:10	0.0000
	全天购电量 33847.7245			全天购电费 19087.2677		
2025.09.23	10:00-10:10	0.0000	12:00-12:10	389.1827	14:00-14:10	0.0000
	16:00-16:10	474.8106	18:00-18:10	783.3189	20:00-20:10	0.0000
	全天购电量 66721.8295			全天购电费 42626.2110		
2025.12.21	10:00-10:10	0.0000	12:00-12:10	864.0564	14:00-14:10	0.0000
	16:00-16:10	821.3834	18:00-18:10	724.4033	20:00-20:10	0.0000
	全天购电量 95130.1167			全天购电费 62108.8836		

表 14　问题三指定日期储能设备的充放电量及 0:00 和 24:00 的储电量

日期	时间段	充电量	放电量	时间段	充电量	放电量
2025.03.20	0:00-4:00	4285.0014	128.7648	4:00-8:00	934.9025	6071.1928
	8:00-12:00	3980.7809	2638.1828	12:00-16:00	6041.9006	254.7228
	16:00-20:00	168.5125	5127.1718	20:00-24:00	5332.3572	2708.5031
	0:00 储电量 6322.2425			24:00 储电量 6181.8652		
2025.06.21	0:00-4:00	128.8401	400.6420	4:00-8:00	433.2655	3139.8946
	8:00-12:00	9773.4128	0.0000	12:00-16:00	0.0000	0.0000
	16:00-20:00	43.1156	5774.6132	20:00-24:00	4843.8231	2624.3907
	0:00 储电量 5431.9630			24:00 储电量 5866.0183		
2025.09.23	0:00-4:00	4556.0539	47.8394	4:00-8:00	712.1862	6364.1470
	8:00-12:00	4596.8046	1896.6157	12:00-16:00	5774.0713	457.2101
	16:00-20:00	58.7812	5736.3256	20:00-24:00	5281.4734	2555.0415
	0:00 储电量 6148.3857			24:00 储电量 6077.3978		
2025.12.21	0:00-4:00	4463.9739	150.5114	4:00-8:00	1464.0658	1496.8017
	8:00-12:00	5183.8592	7381.5477	12:00-16:00	7747.2979	2271.1787
	16:00-20:00	1.5850	5304.6057	20:00-24:00	5036.7340	2893.5270
	0:00 储电量 6382.3223			24:00 储电量 6225.4509		

表 15　问题三指定日期的紧急购电量

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
9:30-10:10	139.1944	无	0.0000	20:30-20:40	6.3750	无	0.0000
15:30-17:00	223.1415	—	—	—	—	—	—
17:10-17:20	3.6233	—	—	—	—	—	—

表 16　问题四对应问题二指定日期的购电量及全天的购电量和购电费

日期	时间段	购电量	时间段	购电量	时间段	购电量
2025.03.20	10:00-10:10	0.0000	12:00-12:10	626.0275	14:00-14:10	0.0000
	16:00-16:10	452.7116	18:00-18:10	709.9796	20:00-20:10	722.7731
	全天购电量 67918.7865			全天购电费 42391.9885		
2025.06.21	10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
	16:00-16:10	110.7647	18:00-18:10	401.7310	20:00-20:10	0.0000
	全天购电量 35367.8280			全天购电费 17204.1382		
2025.09.23	10:00-10:10	0.0000	12:00-12:10	515.8098	14:00-14:10	0.0000
	16:00-16:10	487.2083	18:00-18:10	771.7427	20:00-20:10	806.6984
	全天购电量 67487.3264			全天购电费 44615.7518		
2025.12.21	10:00-10:10	0.0000	12:00-12:10	967.2855	14:00-14:10	0.0000
	16:00-16:10	795.9118	18:00-18:10	713.6978	20:00-20:10	0.0000
	全天购电量 94977.3251			全天购电费 72083.6228		

表 17　问题四对应问题二指定日期储能设备的充放电量及 0:00 和 24:00 的储电量

日期	时间段	充电量	放电量	时间段	充电量	放电量
2025.03.20	0:00-4:00	2037.0136	282.2592	4:00-8:00	4148.7693	5288.9765
	8:00-12:00	6224.9944	2777.3772	12:00-16:00	3444.7917	404.2905
	16:00-20:00	116.6988	5867.7431	20:00-24:00	4359.8637	2353.7910
	0:00 储电量 6089.2497			24:00 储电量 5527.6820		
2025.06.21	0:00-4:00	868.0131	3754.3539	4:00-8:00	2988.0555	1735.4500
	8:00-12:00	9786.8627	0.0000	12:00-16:00	0.0000	0.0000
	16:00-20:00	36.3896	5423.3460	20:00-24:00	4279.4995	2767.3871
	0:00 储电量 4621.1439			24:00 储电量 5583.4856		
2025.09.23	0:00-4:00	3699.8150	116.5770	4:00-8:00	2554.6093	5946.0450
	8:00-12:00	5070.5408	1896.6157	12:00-16:00	4873.3859	614.7539
	16:00-20:00	38.4205	6616.8491	20:00-24:00	4375.1324	1744.5068
	0:00 储电量 5322.8289			24:00 储电量 5056.4896		
2025.12.21	0:00-4:00	5429.8416	218.8432	4:00-8:00	946.6180	1561.6968
	8:00-12:00	5637.3886	6989.1875	12:00-16:00	6790.5193	2262.2220
	16:00-20:00	20.8596	5256.5424	20:00-24:00	4363.8008	2658.7959
	0:00 储电量 6133.7703			24:00 储电量 5951.3534		

表 18　问题四对应问题二指定日期的紧急购电量

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.0000	3:30-3:40	27.3663	18:30-18:50	3.9097	无	0.0000
—	—	—	—	20:20-20:40	43.2800	—	—
—	—	—	—	21:00-21:40	82.1224	—	—
—	—	—	—	21:50-22:10	50.6504	—	—

表 19　问题四对应问题三指定日期的购电量及全天的购电量和购电费

日期	时间段	购电量	时间段	购电量	时间段	购电量
2025.03.20	10:00-10:10	0.0000	12:00-12:10	482.3339	14:00-14:10	0.0000
	16:00-16:10	459.2910	18:00-18:10	720.6293	20:00-20:10	733.6147
	全天购电量 65946.0778			全天购电费 44012.9870		
2025.06.21	10:00-10:10	0.0000	12:00-12:10	0.0000	14:00-14:10	0.0000
	16:00-16:10	43.1742	18:00-18:10	399.3384	20:00-20:10	0.0000
	全天购电量 34424.8296			全天购电费 16516.6859		
2025.09.23	10:00-10:10	0.0000	12:00-12:10	389.1827	14:00-14:10	0.0000
	16:00-16:10	474.8106	18:00-18:10	783.3189	20:00-20:10	818.7989
	全天购电量 66786.5962			全天购电费 43983.2633		
2025.12.21	10:00-10:10	0.0000	12:00-12:10	864.0564	14:00-14:10	0.0000
	16:00-16:10	821.3834	18:00-18:10	724.4033	20:00-20:10	0.0000
	全天购电量 95171.2436			全天购电费 72532.1752		

表 20　问题四对应问题三指定日期储能设备的充放电量及 0:00 和 24:00 的储电量

日期	时间段	充电量	放电量	时间段	充电量	放电量
2025.03.20	0:00-4:00	1882.5098	161.2891	4:00-8:00	3253.5700	6069.0313
	8:00-12:00	3980.7809	2635.0478	12:00-16:00	6046.2567	255.1211
	16:00-20:00	199.0784	5624.6221	20:00-24:00	5272.6417	2301.6735
	0:00 储电量 6427.9372			24:00 储电量 6058.4189		
2025.06.21	0:00-4:00	919.7333	4415.8779	4:00-8:00	2791.6994	1725.4172
	8:00-12:00	9805.3809	0.0000	12:00-16:00	0.0000	0.0000
	16:00-20:00	17.3690	5748.7200	20:00-24:00	4926.6981	2698.6181
	0:00 储电量 5458.5289			24:00 储电量 5863.7292		
2025.09.23	0:00-4:00	2927.5265	49.9742	4:00-8:00	2369.3904	6364.1470
	8:00-12:00	4559.3719	1896.6157	12:00-16:00	5818.3094	464.0155
	16:00-20:00	90.2929	6579.2895	20:00-24:00	5221.3485	1810.3871
	0:00 储电量 6124.9488			24:00 储电量 5940.9765		
2025.12.21	0:00-4:00	5130.6514	150.5114	4:00-8:00	849.2741	1509.4228
	8:00-12:00	5183.8592	7370.4479	12:00-16:00	7733.5945	2271.1787
	16:00-20:00	1.5850	5199.7807	20:00-24:00	5042.9644	2942.6731
	0:00 储电量 6349.6485			24:00 储电量 6292.9237		

表 21　问题四对应问题三指定日期的紧急购电量

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
9:30-10:10	142.3294	无	0.0000	18:30-18:50	3.9097	无	0.0000
15:30-17:00	226.2716	—	—	20:30-20:40	6.3750	—	—
17:10-17:20	3.6233	—	—	—	—	—	—

附录 C　源程序

Q1/q1_ 求解.py

```
import os

import numpy as np
import openpyxl
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


T = 144
DT = 1.0 / 6.0
CAP = 12000.0
P_MAX = 5000.0
E_MAX = P_MAX * DT
S_LO, S_HI = 1200.0, 10800.0
S_INIT = 6000.0
ETA = 0.9

BLOCKS = ['0:00-4:00', '4:00-8:00', '8:00-12:00',
          '12:00-16:00', '16:00-20:00', '20:00-24:00']
TAB1_SLOTS = [('10:00-10:10', 59), ('12:00-12:10', 71), ('14:00-14:10', 83),
              ('16:00-16:10', 95), ('18:00-18:10', 107), ('20:00-20:10', 119)]


def load_data():
    d = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))
    price = d['电价'].to_numpy(float)
    load = d['小区负载'].to_numpy(float) * DT
    pv = d['光伏发电预测功率'].to_numpy(float) * DT
    assert len(price) == T
    return price, load, pv


def solve(price, load, pv, s0=S_INIT):
    need = load - pv
    I = sparse.identity(T, format='csr')
    Z = sparse.csr_matrix((T, T))
    Tri = sparse.csr_matrix(np.tril(np.ones((T, T))))
    one = sparse.csr_matrix(np.ones((1, T)))
    zero1 = sparse.csr_matrix((1, T))

    A_ub = sparse.vstack([
        sparse.hstack([-I, I, -I]),
        sparse.hstack([Z, ETA * Tri, -Tri / ETA]),
        sparse.hstack([Z, -ETA * Tri, Tri / ETA]),
```

```
    ], format='csr')
    b_ub = np.concatenate([-need,
                           np.full(T, S_HI - s0),
                           np.full(T, s0 - S_LO)])

    A_eq = sparse.hstack([zero1, ETA * one, -one / ETA], format='csr')
    b_eq = np.array([0.0])

    cost = np.concatenate([price, np.zeros(T), np.zeros(T)])
    cost[T:] += 1e-9
    lo = np.zeros(3 * T)
    hi = np.concatenate([np.full(T, np.inf), np.full(T, E_MAX), np.full(T, E_MAX)])

    res = linprog(cost, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=np.stack([lo, hi], axis=1), method='highs')
    assert res.success, res.message

    g, c, d = res.x[:T], res.x[T:2 * T], res.x[2 * T:]
    S = s0 + np.cumsum(ETA * c - d / ETA)
    return g, c, d, S


def check(price, load, pv, g, c, d, S, s0=S_INIT):
    need = load - pv
    lines = []
    gap = (g + pv + d - c) - load
    lines.append('供需缺口最小值 %.6f kWh，应不小于 0' % gap.min())
    lines.append('同段既充又放的总量 %.6f kWh，应为 0' % float(np.minimum(c, d).sum()))
    lines.append('储电量范围 %.4f ~ %.4f，应落在 %.0f ~ %.0f 内' %
                 (S.min(), S.max(), S_LO, S_HI))
    lines.append('期初 %.4f，期末 %.4f，两者应相同' % (s0, S[-1]))
    lines.append('充放电量最大值 %.4f kWh，上限 %.4f' % (max(c.max(), d.max()), E_MAX))
    loss = g.sum() - need.sum()
    cyc = float(c.sum()) * (1 - ETA * ETA)
    lines.append('购电量比净需求多 %.4f kWh，储能两次转换的损耗 %.4f kWh，两者应相等'
                 % (loss, cyc))
    return '\n'.join(lines)


def _pad(s, w, right=False):
    s = str(s)
    n = w - sum(2 if ord(ch) > 0x2E80 else 1 for ch in s)
    return (' ' * max(n, 0) + s) if right else (s + ' ' * max(n, 0))


def _row(cells, widths, rights):
    return '  '.join(_pad(x, w, r) for x, w, r in zip(cells, widths, rights))


def tables(price, g, c, d, S, s0=S_INIT):
    W1, R1 = [14, 14] * 3, [False, True] * 3
    W2, R2 = [14, 13, 13] * 2, [False, True, True] * 2
    L = []
    L.append('表 1  微网在指定时间段的购电量及全天的购电量和购电费')
    L.append(_row(['时间段', '购电量'] * 3, W1, R1))
    for i in [0, 3]:
        cells = []
        for nm, k in TAB1_SLOTS[i:i + 3]:
            cells += [nm, '%.4f' % g[k]]
        L.append(_row(cells, W1, R1))
    L.append(_row(['全天购电量', '%.4f' % g.sum(), '全天购电费', '%.4f' % (price @ g)],
                  W1[:4], R1[:4]))
    L.append('')
    L.append('表 2  储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量')
    L.append(_row(['时间段', '充电量', '放电量'] * 2, W2, R2))
    for i in [0, 2, 4]:
        a, b = slice(24 * i, 24 * (i + 1)), slice(24 * (i + 1), 24 * (i + 2))
        L.append(_row([BLOCKS[i], '%.4f' % c[a].sum(), '%.4f' % d[a].sum(),
                       BLOCKS[i + 1], '%.4f' % c[b].sum(), '%.4f' % d[b].sum()],
                      W2, R2))
    L.append(_row(['0:00 储电量', '%.4f' % s0, '24:00 储电量', '%.4f' % S[-1]],
```

```
                [14, 27, 14, 13], [False, True, False, True]))
    return '\n'.join(L)


def write_result(g, c, d, S, s0=S_INIT):
    wb = openpyxl.load_workbook(os.path.join(DATA, '附件 5', 'result1.xlsx'))
    ws = wb['计划购电量']
    for k in range(T):
        ws.cell(2 + k, 2, round(float(g[k]), 4))
    ws = wb['充放电量']
    for j in range(6):
        sl = slice(24 * j, 24 * (j + 1))
        ws.cell(2 + j, 2, round(float(c[sl].sum()), 4))
        ws.cell(2 + j, 3, round(float(d[sl].sum()), 4))
    ws.cell(2, 5, round(float(s0), 4))
    ws.cell(3, 5, round(float(S[-1]), 4))
    os.makedirs(OUT, exist_ok=True)
    path = os.path.join(OUT, 'result1.xlsx')
    wb.save(path)
    return path


def main():
    price, load, pv = load_data()
    g, c, d, S = solve(price, load, pv)
    print(tables(price, g, c, d, S))
    print()
    print('自检')
    print(check(price, load, pv, g, c, d, S))
    print()
    print('全天负载 %.4f kWh, 光伏 %.4f kWh, 净需求 %.4f kWh' %
          (load.sum(), pv.sum(), (load - pv).sum()))
    print('不用储能逐段照买的费用 %.4f 元, 本策略 %.4f 元, 节省 %.2f%%' %
          (price @ np.maximum(load - pv, 0.0), price @ g,
           100 * (1 - (price @ g) / (price @ np.maximum(load - pv, 0.0)))))
    print('已写出', write_result(g, c, d, S))


if __name__ == '__main__':
    main()
```

Q1/q1_ 算法对比.py

```
# -*- coding: utf-8 -*-
import os
import time

import numpy as np
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
```

```
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


T = 144
DT = 1.0 / 6.0
P_MAX = 5000.0
E_MAX = P_MAX * DT
S_LO, S_HI = 1200.0, 10800.0
S_INIT = 6000.0
ETA = 0.9
N_RUN = 10


def load_data():
    d = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))
    price = d['电价'].to_numpy(float)
    need = (d['小区负载'].to_numpy(float) - d['光伏发电预测功率'].to_numpy(float)) * DT
    return price, need


def lp_matrices(price, need):
    I = sparse.identity(T, format='csr')
    Z = sparse.csr_matrix((T, T))
    Tri = sparse.csr_matrix(np.tril(np.ones((T, T))))
    one = sparse.csr_matrix(np.ones((1, T)))
    z1 = sparse.csr_matrix((1, T))
    A_ub = sparse.vstack([sparse.hstack([-I, I, -I]),
                          sparse.hstack([Z, ETA * Tri, -Tri / ETA]),
                          sparse.hstack([Z, -ETA * Tri, Tri / ETA])], format='csr')
    b_ub = np.concatenate([-need, np.full(T, S_HI - S_INIT), np.full(T, S_INIT - S_LO)])
    A_eq = sparse.hstack([z1, ETA * one, -one / ETA], format='csr')
    b_eq = np.array([0.0])
    cost = np.concatenate([price, np.full(T, 1e-9), np.full(T, 1e-9)])
    lo = np.zeros(3 * T)
    hi = np.concatenate([np.full(T, np.inf), np.full(T, E_MAX), np.full(T, E_MAX)])
    return A_ub, b_ub, A_eq, b_eq, cost, np.stack([lo, hi], axis=1)


def run_lp(price, need, method):
    A_ub, b_ub, A_eq, b_eq, cost, bnd = lp_matrices(price, need)
    t0 = time.time()
    res = linprog(cost, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bnd, method=method)
    dtm = time.time() - t0
    g = res.x[:T]
    return float(price @ g), dtm, res.success


def run_dp(price, need, step=25.0):
    grid = np.arange(S_LO, S_HI + 1e-9, step)
    n = len(grid)
    i0 = int(round((S_INIT - S_LO) / step))
    t0 = time.time()
    delta = grid[None, :] - grid[:, None]
    chg = np.where(delta >= 0, delta / ETA, 0.0)
    dis = np.where(delta < 0, -delta * ETA, 0.0)
    feas = (chg <= E_MAX + 1e-9) & (dis <= E_MAX + 1e-9)
    base = chg - dis
    J = np.full(n, np.inf)
    J[i0] = 0.0
    for t in range(T - 1, -1, -1):
        step_cost = price[t] * np.maximum(need[t] + base, 0.0)
        step_cost = np.where(feas, step_cost, np.inf)
        J = np.min(step_cost + J[None, :], axis=1)
    dtm = time.time() - t0
    return float(J[i0]), dtm, np.isfinite(J[i0])


def decode(u, price, need):
```

```
    u = np.clip(u, -1.0, 1.0)
    s = S_INIT
    c = np.zeros(T)
    d = np.zeros(T)
    for t in range(T):
        if u[t] >= 0:
            ct = min(u[t] * E_MAX, (S_HI - s) / ETA)
            dt_ = 0.0
        else:
            ct = 0.0
            dt_ = min(-u[t] * E_MAX, (s - S_LO) * ETA)
        c[t], d[t] = ct, dt_
        s += ETA * ct - dt_ / ETA
    g = np.maximum(need + c - d, 0.0)
    pen = abs(s - S_INIT) * price.max() * 5.0
    return float(price @ g) + pen, float(price @ g), s


def decode_batch(U, price, need):
    U = np.clip(U, -1.0, 1.0)
    m = U.shape[0]
    s = np.full(m, S_INIT)
    C = np.zeros((m, T))
    D = np.zeros((m, T))
    for t in range(T):
        up = U[:, t]
        ct = np.where(up >= 0, np.minimum(up * E_MAX, (S_HI - s) / ETA), 0.0)
        dt_ = np.where(up < 0, np.minimum(-up * E_MAX, (s - S_LO) * ETA), 0.0)
        C[:, t], D[:, t] = ct, dt_
        s = s + ETA * ct - dt_ / ETA
    G = np.maximum(need[None, :] + C - D, 0.0)
    raw = G @ price
    return raw + np.abs(s - S_INIT) * price.max() * 5.0, raw


def run_ga(price, need, seed, pop=80, gen=300):
    rng = np.random.default_rng(seed)
    t0 = time.time()
    U = rng.uniform(-1, 1, (pop, T))
    fit, raw = decode_batch(U, price, need)
    for _ in range(gen):
        idx = rng.integers(0, pop, (pop, 2))
        win = np.where(fit[idx[:, 0]] < fit[idx[:, 1]], idx[:, 0], idx[:, 1])
        P = U[win]
        mask = rng.random((pop, T)) < 0.5
        Q = np.where(mask, P, P[rng.permutation(pop)])
        mut = rng.random((pop, T)) < 0.08
        Q = Q + mut * rng.normal(0, 0.25, (pop, T))
        f2, r2 = decode_batch(Q, price, need)
        best = np.argmin(fit)
        worst = np.argmax(f2)
        Q[worst], f2[worst], r2[worst] = U[best], fit[best], raw[best]
        U, fit, raw = Q, f2, r2
    return float(raw[np.argmin(fit)]), time.time() - t0, True


def run_pso(price, need, seed, pop=80, it=300):
    rng = np.random.default_rng(seed)
    t0 = time.time()
    X = rng.uniform(-1, 1, (pop, T))
    V = rng.uniform(-0.2, 0.2, (pop, T))
    fit, raw = decode_batch(X, price, need)
    pb, pbf, pbr = X.copy(), fit.copy(), raw.copy()
    gi = np.argmin(pbf)
    for k in range(it):
        w = 0.9 - 0.5 * k / it
        V = (w * V + 1.5 * rng.random((pop, T)) * (pb - X)
             + 1.5 * rng.random((pop, T)) * (pb[gi] - X))
        V = np.clip(V, -0.4, 0.4)
        X = np.clip(X + V, -1, 1)
        fit, raw = decode_batch(X, price, need)
```

```
        imp = fit < pbf
        pb[imp], pbf[imp], pbr[imp] = X[imp], fit[imp], raw[imp]
        gi = np.argmin(pbf)
    return float(pbr[gi]), time.time() - t0, True


def run_sa(price, need, seed, iters=40000):
    rng = np.random.default_rng(seed)
    t0 = time.time()
    u = rng.uniform(-1, 1, T)
    f, raw, _ = decode(u, price, need)
    bu, bf, braw = u.copy(), f, raw
    Tp = 5000.0
    for k in range(iters):
        Tp = 5000.0 * (1 - k / iters) ** 2 + 1.0
        v = u.copy()
        j = rng.integers(0, T, 3)
        v[j] = np.clip(v[j] + rng.normal(0, 0.3, 3), -1, 1)
        f2, raw2, _ = decode(v, price, need)
        if f2 < f or rng.random() < np.exp(-(f2 - f) / Tp):
            u, f, raw = v, f2, raw2
            if f < bf:
                bu, bf, braw = u.copy(), f, raw
    return float(braw), time.time() - t0, True


def run_rule(price, need):
    t0 = time.time()
    lo_q, hi_q = np.quantile(price, 0.35), np.quantile(price, 0.65)
    s = S_INIT
    c = np.zeros(T); d = np.zeros(T)
    for t in range(T):
        if price[t] <= lo_q:
            ct = min(E_MAX, (S_HI - s) / ETA)
            c[t] = ct
            s += ETA * ct
        elif price[t] >= hi_q:
            dt_ = min(E_MAX, (s - S_LO) * ETA, max(need[t], 0.0))
            d[t] = dt_
            s -= dt_ / ETA
    g = np.maximum(need + c - d, 0.0)
    return float(price @ g), time.time() - t0, True


def main():
    price, need = load_data()
    rows = []

    v, tm, ok = run_lp(price, need, 'highs-ds')
    rows.append(('对偶单纯形法', v, tm, 1.0, '精确'))
    best = v
    v2, tm2, _ = run_lp(price, need, 'highs-ipm')
    rows.append(('内点法', v2, tm2, 1.0, '精确'))
    v3, tm3, _ = run_dp(price, need)
    rows.append(('动态规划', v3, tm3, 1.0, '精确'))
    v4, tm4, _ = run_rule(price, need)
    rows.append(('谷充峰放规则法', v4, tm4, 0.0, '基准'))

    for nm, fn in [('遗传算法', run_ga), ('粒子群算法', run_pso), ('模拟退火算法', run_sa)]:
        vals, tms = [], []
        for r in range(N_RUN):
            a, b, _ = fn(price, need, 20260910 + r)
            vals.append(a); tms.append(b)
        vals = np.array(vals)
        hit = float(np.mean(vals <= best * 1.001))
        rows.append((nm, float(vals.min()), float(np.mean(tms)), hit, '启发式',
                     float(vals.mean())))

    L = []
    L.append('%-16s %14s %10s %12s %10s' % ('算法', '目标值 元', '相对最优', '命中最优比例', '用时 s'))
    for r in rows:
```

```
        gap = 100 * (r[1] - best) / best
        L.append('%-16s %14.4f %9.3f%% %11.0f%% %10.3f'
                 % (r[0], r[1], gap, 100 * r[3], r[2]))
    L.append('')
    L.append('精确解法最优值 %.4f 元，启发式十次运行的最好值与均值见下' % best)
    for r in rows:
        if len(r) > 5:
            L.append('  %-14s 最好 %12.4f  十次均值 %12.4f  相对最优 %.3f%%'
                     % (r[0], r[1], r[5], 100 * (r[5] - best) / best))
    txt = '\n'.join(L)
    print(txt)
    os.makedirs(OUT, exist_ok=True)
    open(os.path.join(OUT, 'q1_ 算法对比.txt'), 'w', encoding='utf-8').write(txt)


if __name__ == '__main__':
    main()
```

Q2/q2_ 求解.py

```
# -*- coding: utf-8 -*-
import datetime as dt
import os

import numpy as np
import openpyxl
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


T = 144
DT = 1.0 / 6.0
P_MAX = 5000.0
E_MAX = P_MAX * DT
S_LO, S_HI = 1200.0, 10800.0
S_INIT = 6000.0
ETA = 0.9
EMERG = 5.0
MARGIN_L = 0.0        # 负载预测的安全裕度
MARGIN_P = 0.05       # 光伏预测的安全裕度
RESERVE_M = 500.0     # 13:00 之前的储能备用
RESERVE_E = 1000.0    # 13:00 之后的储能备用
SPLIT = 78
K_PV = 5
N_SHAPE = 8
LEVEL_W = (0.6, 0.25, 0.15)
N_RATIO = 28
```

```
REPORT_FROM = 31
EPS = 1e-7

DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]
DOW = np.array([d.weekday() for d in DATES])
WK = ((DOW >= 4) & (DOW <= 5)).astype(int)
BLOCKS = [('0:00-4:00', slice(0, 24)), ('4:00-8:00', slice(24, 48)),
          ('8:00-12:00', slice(48, 72)), ('12:00-16:00', slice(72, 96)),
          ('16:00-20:00', slice(96, 120)), ('20:00-24:00', slice(120, 144))]


def load_a1():
    d = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))
    return d['电价'].to_numpy(float)


def load_a2():
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    return load, pv


# ---------------- 预测 ----------------
_CACHE = {}


def _harm(load, d):
    js = np.arange(d)
    X = lambda j: np.column_stack([np.ones(len(j)), WK[j],
                                   np.sin(2 * np.pi * j / 365), np.cos(2 * np.pi * j / 365),
                                   np.sin(4 * np.pi * j / 365), np.cos(4 * np.pi * j / 365)])
    if d < 12:
        return load[:d].mean(0) if d > 0 else load.mean(0)
    b, *_ = np.linalg.lstsq(X(js), load[js], rcond=None)
    return X(np.array([d]))[0] @ b


def load_hat(load, d):
    """ 日型形状乘日总量水平：形状取最近 N_SHAPE 个同类日的归一化曲线，
    水平取最近三天折算到同一日型后的加权日总量。"""
    if d in _CACHE:
        return _CACHE[d]
    if d < 14:
        v = _harm(load, d)
    else:
        js = [j for j in range(d - 1, -1, -1) if WK[j] == WK[d]][:N_SHAPE]
        shape = (load[js] / load[js].sum(1, keepdims=True)).mean(0)
        jr = np.arange(max(0, d - N_RATIO), d)
        ratio = load[jr][WK[jr] == 1].sum(1).mean() / load[jr][WK[jr] == 0].sum(1).mean()
        tot = load[:d].sum(1) / np.where(WK[:d] == 1, ratio, 1.0)
        level = sum(w * tot[-1 - i] for i, w in enumerate(LEVEL_W)) / sum(LEVEL_W)
        v = shape * level * (ratio if WK[d] == 1 else 1.0)
    _CACHE[d] = v
    return v


def pv_hat(pv, d):
    return pv.mean(0) if d == 0 else pv[max(0, d - K_PV):d].mean(0)


# ---------------- 线性规划 ----------------
_MAT = {}


def _mats(n):
    if n not in _MAT:
        I = sparse.identity(n, format='csr'); Z = sparse.csr_matrix((n, n))
        Dm = sparse.diags([np.ones(n), -np.ones(n - 1)], [0, -1], format='csr')
        A_ub = sparse.hstack([-I, I, -I, -I, Z], format='csr')
        A_eq = sparse.hstack([Z, -ETA * I, I / ETA, Z, Dm], format='csr')
```

```
        _MAT[n] = (A_ub, A_eq)
    return _MAT[n]


def day_lp(price, need, s0, s_end=S_INIT, lo=S_LO, g_fix=None):
    """ 变量依次为购电 g、充电 c、放电 d、紧急购电 e、储电量 S，各 n 个。"""
    n = len(price)
    A_ub, A_eq = _mats(n)
    b_eq = np.zeros(n); b_eq[0] = s0
    cost = np.zeros(5 * n)
    cost[:n] = price
    cost[n:3 * n] = EPS
    cost[3 * n:4 * n] = EMERG * price
    lb = np.zeros(5 * n); ub = np.full(5 * n, np.inf)
    ub[n:3 * n] = E_MAX
    lo_v = np.minimum(np.broadcast_to(np.asarray(lo, float), (n,)).copy(), S_HI)
    lo_v[-1] = min(max(lo_v[-1], s_end), S_HI)
    lb[4 * n:] = lo_v; ub[4 * n:] = S_HI
    if g_fix is not None:
        lb[:n] = g_fix; ub[:n] = g_fix
    res = linprog(cost, A_ub=A_ub, b_ub=-np.asarray(need, float), A_eq=A_eq, b_eq=b_eq,
                  bounds=np.stack([lb, ub], 1), method='highs')
    if not res.success:
        return None
    x = res.x
    return dict(g=x[:n], c=x[n:2 * n], d=x[2 * n:3 * n], e=x[3 * n:4 * n], S=x[4 * n:])


def plan_day(price, load_fc, pv_fc, s0):
    need_plan = (load_fc * (1 + MARGIN_L) - pv_fc * (1 - MARGIN_P)) * DT
    tt = np.arange(T)
    lo = np.minimum(S_LO + np.where(tt < SPLIT, RESERVE_M, RESERVE_E), s0)
    r = day_lp(price, need_plan, s0, s_end=max(S_INIT, lo[-1]), lo=lo)
    if r is None:
        r = day_lp(price, need_plan, s0, s_end=S_LO)
    return r['g']


def run_day(price, need_act, need_fc, plan_g, s0):
    """ 逐段揭晓实际净负载。有富余就充电；有缺口时先判断储能是否足够覆盖
    预测的后续全部缺口，够就直接放电补齐，不够再解剩余时段的放电分配问题。"""
    a = np.asarray(plan_g, float)
    c = np.zeros(T); dis = np.zeros(T); e = np.zeros(T); S = np.zeros(T)
    s = float(s0)
    for t in range(T):
        net = a[t] - need_act[t]
        avail_dis = min(E_MAX, max(0.0, (s - S_LO) * ETA))
        avail_chg = min(E_MAX, max(0.0, (S_HI - s) / ETA))
        if net >= 0:
            ct, dt_ = min(net, avail_chg), 0.0
        else:
            want = -net
            intent = want
            if avail_dis > 1e-9:
                fc = np.concatenate([[need_act[t]], need_fc[t + 1:]])
                future_def = np.maximum(fc[1:] - a[t + 1:], 0.0).sum()
                if (s - S_LO) * ETA - min(want, avail_dis) < future_def:
                    r = day_lp(price[t:], fc, s, s_end=S_LO, g_fix=a[t:])
                    if r is not None:
                        intent = max(r['d'][0], 0.0)
            ct, dt_ = 0.0, min(intent, want, avail_dis)
        e[t] = max(0.0, -net - dt_)
        c[t], dis[t] = ct, dt_
        s = float(np.clip(s + ETA * ct - dt_ / ETA, S_LO, S_HI))
        S[t] = s
    cost_plan = float(price @ a)
    cost_emerg = float(EMERG * (price @ e))
    return dict(a=a, c=c, d=dis, e=e, S=S, s_end=s, plan=a.copy(),
                cost_plan=cost_plan, cost_dev=0.0, cost_emerg=cost_emerg,
                cost=cost_plan + cost_emerg)
```

```
def run_year(price, load, pv, n_days=365):
    s = float(S_INIT)
    recs = []
    for d in range(n_days):
        need_act = (load[d] - pv[d]) * DT
        lf, pf = load_hat(load, d), pv_hat(pv, d)
        g = plan_day(price, lf, pf, s)
        rec = run_day(price, need_act, (lf - pf) * DT, g, s)
        rec['day'] = d
        rec['s0'] = s
        s = rec['s_end']
        recs.append(rec)
        if d % 30 == 0:
            print('  第 %3d 天  费用 %9.2f  紧急 %8.2f  期末储电 %8.1f'
                  % (d, rec['cost'], rec['cost_emerg'], s), flush=True)
    return recs


def slot_labels():
    ws = openpyxl.load_workbook(os.path.join(DATA, '附件 5', 'result2.xlsx'))['计划购电量']
    return [str(ws.cell(1, 2 + k).value) for k in range(T)]


LABELS = slot_labels()


def emergency_intervals(e, tol=1e-6):
    out = []
    k = 0
    while k < T:
        if e[k] > tol:
            j = k
            while j + 1 < T and e[j + 1] > tol:
                j += 1
            a = LABELS[k].split('-')[0]
            b = LABELS[j].split('-')[-1]
            out.append(('%s-%s' % (a, b), float(e[k:j + 1].sum())))
            k = j + 1
        else:
            k += 1
    return out


def r4(x):
    return float(np.round(float(x), 4))


def clear_below(ws, first_row=2):
    for row in ws.iter_rows(min_row=first_row, max_row=ws.max_row):
        for cell in row:
            cell.value = None


def write_result(recs, name='result2.xlsx', template='result2.xlsx'):
    wb = openpyxl.load_workbook(os.path.join(DATA, '附件 5', template))

    ws = wb['计划购电量']
    clear_below(ws)
    for i, r in enumerate(recs):
        row = 2 + i
        ws.cell(row, 1, DATES[r['day']].strftime('%Y/%m/%d'))
        for k in range(T):
            ws.cell(row, 2 + k, r4(r['plan'][k]))
        ws.cell(row, 2 + T, r4(float(r['plan'].sum())))
        ws.cell(row, 3 + T, r4(r['cost']))

    ws = wb['充放电量']
    clear_below(ws)
    row = 2
    for r in recs:
```

```
        for j, (nm, sl) in enumerate(BLOCKS):
            ws.cell(row + j, 1, DATES[r['day']].strftime('%Y/%m/%d') if j == 0 else None)
            ws.cell(row + j, 2, nm)
            ws.cell(row + j, 3, r4(float(r['c'][sl].sum())))
            ws.cell(row + j, 4, r4(float(r['d'][sl].sum())))
        ws.cell(row, 5, '0:00')
        ws.cell(row, 6, r4(r['s0']))
        ws.cell(row + 1, 5, '24:00')
        ws.cell(row + 1, 6, r4(r['s_end']))
        row += 6

    ws = wb['紧急购电量']
    clear_below(ws)
    row = 2
    for r in recs:
        iv = emergency_intervals(r['e'])
        if not iv:
            ws.cell(row, 1, DATES[r['day']].strftime('%Y/%m/%d'))
            ws.cell(row, 2, '无')
            ws.cell(row, 3, 0.0)
            row += 1
            continue
        for j, (lab, v) in enumerate(iv):
            ws.cell(row + j, 1, DATES[r['day']].strftime('%Y/%m/%d') if j == 0 else None)
            ws.cell(row + j, 2, lab)
            ws.cell(row + j, 3, r4(v))
        row += len(iv)

    os.makedirs(OUT, exist_ok=True)
    path = os.path.join(OUT, name)
    wb.save(path)
    return path


def summary(recs):
    sub = recs[REPORT_FROM:]
    tot = sum(r['cost'] for r in sub)
    pl = sum(r['cost_plan'] for r in sub)
    em = sum(r['cost_emerg'] for r in sub)
    kwh = sum(float(r['e'].sum()) for r in sub)
    days = sum(1 for r in sub if r['e'].sum() > 1e-6)
    L = []
    L.append('统计区间 %s 至 %s，共 %d 天'
             % (DATES[sub[0]['day']], DATES[sub[-1]['day']], len(sub)))
    L.append('总购电费 %.2f 元' % tot)
    L.append('  计划购电费 %.2f 元，占 %.2f%%' % (pl, 100 * pl / tot))
    L.append('  紧急购电费 %.2f 元，占 %.2f%%' % (em, 100 * em / tot))
    L.append('紧急购电量 %.2f kWh，发生在 %d 天' % (kwh, days))
    return '\n'.join(L)


def main():
    price = load_a1()
    load, pv = load_a2()
    print('开始滚动求解，共 365 天')
    recs = run_year(price, load, pv)
    print()
    print(summary(recs))
    print('已写出', write_result(recs[REPORT_FROM:]))


if __name__ == '__main__':
    main()
```

Q2/q2_ 算法对比.py

```
# -*- coding: utf-8 -*-
import datetime as dt
import os
import time
```

```
import numpy as np
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


T, DT = 144, 1.0 / 6.0
ETA, S_LO, S_HI, S_INIT = 0.9, 1200.0, 10800.0, 6000.0
E_MAX = 5000.0 * DT
EMERG, DEV = 5.0, 0.5
# 与求解脚本一致的新口径
Q2_MARGIN_P, Q2_RESERVE_M, Q2_RESERVE_E, SPLIT = 0.05, 500.0, 1000.0, 78
Q3_MARGIN, Q3_RESERVE_PLAN, Q3_RESERVE_ADJ = 0.015, 500.0, 250.0
K_PV, N_SHAPE, LEVEL_W, N_RATIO = 5, 8, (0.6, 0.25, 0.15), 28
W0, W_SLOPE, W_MAX = 0.95, 0.05, 0.95
N_RUN = 10
DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]
DOW = np.array([d.weekday() for d in DATES])
WK = ((DOW >= 4) & (DOW <= 5)).astype(int)
SAMPLE = [DATES.index(dt.date(2025, m, 15)) for m in range(2, 13)] + \
         [DATES.index(dt.date(2025, 1, 15))]


def load_all():
    a1 = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    return a1['电价'].to_numpy(float), load, pv


def load_a3():
    d = pd.read_excel(os.path.join(DATA, '附件 3.xlsx'))
    return d.iloc[:, 2:].to_numpy(float).reshape(365, 4, 24)


def load_hat(load, d):
    """ 同类日形状乘近期水平，取样日都在 2 月之后，历史足够。"""
    js = [j for j in range(d - 1, -1, -1) if WK[j] == WK[d]][:N_SHAPE]
    shape = (load[js] / load[js].sum(1, keepdims=True)).mean(0)
    jr = np.arange(max(0, d - N_RATIO), d)
    ratio = load[jr][WK[jr] == 1].sum(1).mean() / load[jr][WK[jr] == 0].sum(1).mean()
    tot = load[:d].sum(1) / np.where(WK[:d] == 1, ratio, 1.0)
    level = sum(w * tot[-1 - i] for i, w in enumerate(LEVEL_W)) / sum(LEVEL_W)
    return shape * level * (ratio if WK[d] == 1 else 1.0)


def pv_hist(pv, d):
```

```
    return pv[max(0, d - K_PV):d].mean(0)


def pv_blend(pv, fc, d, issue):
    h0 = (0, 6, 12, 18)[issue]
    hrs = np.arange(h0 + 1, h0 + 25, dtype=float); f = fc[d, issue]
    if h0 > 0:
        hrs = np.concatenate([[h0], hrs]); f = np.concatenate([[pv[d, h0 * 6 - 1]], f])
    tm = (np.arange(T) + 1) * DT
    w = np.clip(W0 - W_SLOPE * (tm - h0), 0.0, W_MAX)
    return w * np.maximum(np.interp(tm, hrs, f), 0.0) + (1 - w) * pv_hist(pv, d)


def reserve_q2():
    return np.where(np.arange(T) < SPLIT, Q2_RESERVE_M, Q2_RESERVE_E)


def need_plan(load, pv, d):
    """ 问题二日前计划子问题的净负载：光伏按预测的 95% 计入。"""
    return (load_hat(load, d) - (1 - Q2_MARGIN_P) * pv_hist(pv, d)) * DT


def need_q3(load, pv, fc, d, issue):
    """ 问题三的净负载：issue=0 为 0:00 计划，issue=1 为 6:00 调整，此前的时段已揭晓取实际值。"""
    n = (load_hat(load, d) - pv_blend(pv, fc, d, issue)) * DT
    n = n + Q3_MARGIN * np.abs(n)
    if issue > 0:
        t0 = (0, 36, 72, 108)[issue]
        n[:t0] = (load[d, :t0] - pv[d, :t0]) * DT
    return n


# ---------------------------------------------------------------- 精确解法

def _lo(reserve, s0):
    return np.minimum(np.minimum(S_LO + np.broadcast_to(np.asarray(reserve, float), (T,)), s0), S_HI)


def build(price, need, s0, s_end, reserve, ref=None):
    n = T
    s_lo = _lo(reserve, s0)
    I = sparse.identity(n, format='csr')
    Z = sparse.csr_matrix((n, n))
    Tri = sparse.csr_matrix(np.tril(np.ones((n, n))))
    one = sparse.csr_matrix(np.ones((1, n)))
    z1 = sparse.csr_matrix((1, n))
    nb = 6 if ref is not None else 4
    blocks = lambda *b: sparse.hstack(list(b) + [Z] * (nb - len(b)), format='csr')
    b1 = lambda *b: sparse.hstack(list(b) + [z1] * (nb - len(b)), format='csr')
    A_ub = sparse.vstack([
        blocks(-I, I, -I, -I),
        blocks(Z, ETA * Tri, -Tri / ETA, Z),
        blocks(Z, -ETA * Tri, Tri / ETA, Z),
        b1(z1, -ETA * one, one / ETA, z1)], format='csr')
    b_ub = np.concatenate([-need, np.full(n, S_HI - s0),
                           s0 - s_lo,
                           np.array([s0 - max(s_end, s_lo[-1])])])
    A_eq = b_eq = None
    cost = np.zeros(nb * n)
    cost[n:3 * n] = 1e-9
    cost[3 * n:4 * n] = EMERG * price
    if ref is None:
        cost[:n] = price
    else:
        A_eq = blocks(I, Z, Z, Z, -I, I)
        b_eq = np.asarray(ref, float)
        cost[4 * n:5 * n] = (1 + DEV) * price
        cost[5 * n:6 * n] = -DEV * price
    lo = np.zeros(nb * n)
    hi = np.full(nb * n, np.inf)
    hi[n:2 * n] = E_MAX
```

```
    hi[2 * n:3 * n] = E_MAX
    return A_ub, b_ub, A_eq, b_eq, cost, np.stack([lo, hi], axis=1), nb


def solve_lp(price, need, s0, s_end, reserve, ref, method):
    A_ub, b_ub, A_eq, b_eq, cost, bnd, nb = build(price, need, s0, s_end, reserve, ref)
    t0 = time.time()
    r = linprog(cost, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                bounds=bnd, method=method)
    dtm = time.time() - t0
    base = float(price @ np.asarray(ref)) if ref is not None else 0.0
    return float(r.fun) + base, dtm, r.success


def solve_dp(price, need, s0, s_end, reserve, ref, step=25.0):
    """ 储电量从下限起每 25 kWh 一档，逐段下限随备用变化，低于当段下限的档位不可达。"""
    s_lo = _lo(reserve, s0)
    base_lo = min(S_LO, s0)
    grid = np.arange(base_lo, S_HI + 1e-9, step)
    n = len(grid)
    i0 = int(round((s0 - base_lo) / step))
    i0 = min(max(i0, 0), n - 1)
    t0 = time.time()
    delta = grid[None, :] - grid[:, None]
    chg = np.where(delta >= 0, delta / ETA, 0.0)
    dis = np.where(delta < 0, -delta * ETA, 0.0)
    feas = (chg <= E_MAX + 1e-9) & (dis <= E_MAX + 1e-9)
    base = chg - dis
    J = np.where(grid >= max(s_end, s_lo[-1]) - 1e-6, 0.0, np.inf)
    for t in range(T - 1, -1, -1):
        J = np.where(grid >= s_lo[t] - 1e-6, J, np.inf)
        take = np.maximum(need[t] + base, 0.0)
        if ref is None:
            step_cost = price[t] * take
        else:
            step_cost = price[t] * take + DEV * price[t] * np.abs(take - ref[t])
        step_cost = np.where(feas, step_cost, np.inf)
        J = np.min(step_cost + J[None, :], axis=1)
    return float(J[i0]), time.time() - t0, np.isfinite(J[i0])


# ---------------------------------------------------------------------- 启发式

def decode(U, price, need, s0, ref):
    U = np.clip(U, -1.0, 1.0)
    m = U.shape[0]
    s = np.full(m, s0)
    C = np.zeros((m, T)); D = np.zeros((m, T))
    for t in range(T):
        up = U[:, t]
        ct = np.where(up >= 0, np.minimum(up * E_MAX, (S_HI - s) / ETA), 0.0)
        dt_ = np.where(up < 0, np.minimum(-up * E_MAX, (s - S_LO) * ETA), 0.0)
        C[:, t], D[:, t] = ct, dt_
        s = s + ETA * ct - dt_ / ETA
    A = np.maximum(need[None, :] + C - D, 0.0)
    raw = A @ price
    if ref is not None:
        raw = raw + DEV * (np.abs(A - ref[None, :]) @ price)
    pen = np.abs(s - S_INIT) * price.max() * 5.0
    return raw + pen, raw


def ga(price, need, s0, ref, seed, pop=80, gen=250):
    rng = np.random.default_rng(seed)
    t0 = time.time()
    U = rng.uniform(-1, 1, (pop, T))
    fit, raw = decode(U, price, need, s0, ref)
    for _ in range(gen):
        idx = rng.integers(0, pop, (pop, 2))
        win = np.where(fit[idx[:, 0]] < fit[idx[:, 1]], idx[:, 0], idx[:, 1])
        P = U[win]
```

```
        mask = rng.random((pop, T)) < 0.5
        Q = np.where(mask, P, P[rng.permutation(pop)])
        Q = Q + (rng.random((pop, T)) < 0.08) * rng.normal(0, 0.25, (pop, T))
        f2, r2 = decode(Q, price, need, s0, ref)
        b = np.argmin(fit); w = np.argmax(f2)
        Q[w], f2[w], r2[w] = U[b], fit[b], raw[b]
        U, fit, raw = Q, f2, r2
    return float(raw[np.argmin(fit)]), time.time() - t0, True


def pso(price, need, s0, ref, seed, pop=80, it=250):
    rng = np.random.default_rng(seed)
    t0 = time.time()
    X = rng.uniform(-1, 1, (pop, T))
    V = rng.uniform(-0.2, 0.2, (pop, T))
    fit, raw = decode(X, price, need, s0, ref)
    pb, pbf, pbr = X.copy(), fit.copy(), raw.copy()
    gi = int(np.argmin(pbf))
    for k in range(it):
        w = 0.9 - 0.5 * k / it
        V = np.clip(w * V + 1.5 * rng.random((pop, T)) * (pb - X)
                    + 1.5 * rng.random((pop, T)) * (pb[gi] - X), -0.4, 0.4)
        X = np.clip(X + V, -1, 1)
        fit, raw = decode(X, price, need, s0, ref)
        imp = fit < pbf
        pb[imp], pbf[imp], pbr[imp] = X[imp], fit[imp], raw[imp]
        gi = int(np.argmin(pbf))
    return float(pbr[gi]), time.time() - t0, True


def sa_batch(price, need, s0, ref, seed, chains=N_RUN, iters=20000):
    """ 多条独立退火链一起跑，一次解码算完所有链，返回每条链的最好值。"""
    rng = np.random.default_rng(seed)
    t0 = time.time()
    u = rng.uniform(-1, 1, (chains, T))
    f, raw = decode(u, price, need, s0, ref)
    bf, braw = f.copy(), raw.copy()
    rows = np.arange(chains)
    for k in range(iters):
        Tp = 5000.0 * (1 - k / iters) ** 2 + 1.0
        v = u.copy()
        j = rng.integers(0, T, (chains, 3))
        v[rows[:, None], j] = np.clip(
            v[rows[:, None], j] + rng.normal(0, 0.3, (chains, 3)), -1, 1)
        f2, r2 = decode(v, price, need, s0, ref)
        acc = (f2 < f) | (rng.random(chains) < np.exp(-(f2 - f) / Tp))
        u[acc], f[acc], raw[acc] = v[acc], f2[acc], r2[acc]
        imp = f < bf
        bf[imp], braw[imp] = f[imp], raw[imp]
    return braw, (time.time() - t0) / chains


def rule(price, need, s0, ref):
    t0 = time.time()
    lo_q, hi_q = np.quantile(price, 0.35), np.quantile(price, 0.65)
    s = s0
    c = np.zeros(T); d = np.zeros(T)
    for t in range(T):
        if price[t] <= lo_q:
            ct = min(E_MAX, (S_HI - s) / ETA); c[t] = ct; s += ETA * ct
        elif price[t] >= hi_q:
            dt_ = min(E_MAX, (s - S_LO) * ETA, max(need[t], 0.0))
            d[t] = dt_; s -= dt_ / ETA
    a = np.maximum(need + c - d, 0.0)
    v = float(price @ a)
    if ref is not None:
        v += DEV * float(price @ np.abs(a - ref))
    return v, time.time() - t0, True


# ---------------------------------------------------------------- 主流程
```

```
def compare(title, price, needs, s0, s_end, reserve, refs):
    print('\n' + title)
    exact = []
    rows = {}
    for nm, fn in [('对偶单纯形法', lambda p, n, r: solve_lp(p, n, s0, s_end, reserve, r, 'highs-ds')),
                   ('内点法', lambda p, n, r: solve_lp(p, n, s0, s_end, reserve, r, 'highs-ipm')),
                   ('动态规划', lambda p, n, r: solve_dp(p, n, s0, s_end, reserve, r))]:
        vals, tms = [], []
        for need, ref in zip(needs, refs):
            v, tm, ok = fn(price, need, ref)
            vals.append(v); tms.append(tm)
        rows[nm] = (np.array(vals), float(np.mean(tms)), 1.0)
    best = rows['对偶单纯形法'][0]

    for nm, fn in [('遗传算法', ga), ('粒子群算法', pso)]:
        allv, tms, hit = [], [], []
        for i, (need, ref) in enumerate(zip(needs, refs)):
            vs = []
            for r in range(N_RUN):
                v, tm, _ = fn(price, need, s0, ref, 20260910 + r)
                vs.append(v); tms.append(tm)
            vs = np.array(vs)
            allv.append(vs.min())
            hit.append(float(np.mean(vs <= best[i] * 1.001)))
        rows[nm] = (np.array(allv), float(np.mean(tms)), float(np.mean(hit)))

    allv, tms, hit = [], [], []
    for i, (need, ref) in enumerate(zip(needs, refs)):
        vs, tm = sa_batch(price, need, s0, ref, 20260910)
        tms.append(tm)
        allv.append(float(vs.min()))
        hit.append(float(np.mean(vs <= best[i] * 1.001)))
    rows['模拟退火算法'] = (np.array(allv), float(np.mean(tms)), float(np.mean(hit)))

    vals, tms = [], []
    for need, ref in zip(needs, refs):
        v, tm, _ = rule(price, need, s0, ref)
        vals.append(v); tms.append(tm)
    rows['谷充峰放规则法'] = (np.array(vals), float(np.mean(tms)), 0.0)

    print('  %-14s %14s %10s %12s %10s' % ('算法', '平均目标值 元', '相对最优', '命中最优比例', '单次用时
    s'))
    for nm, (v, tm, h) in rows.items():
        gap = 100 * np.mean((v - best) / best)
        print('  %-14s %14.4f %9.3f%% %11.0f%% %10.3f' % (nm, v.mean(), gap, 100 * h, tm))
    return rows


def main():
    price, load, pv = load_all()
    days = sorted(SAMPLE)[1:]
    needs = [need_plan(load, pv, d) for d in days]
    print('取样日期: 每月 15 日, 共 %d 天' % len(days))

    compare('问题二 日前计划子问题的多算法对比', price, needs, S_INIT, S_INIT,
            reserve_q2(), [None] * len(needs))


if __name__ == '__main__':
    main()
```

Q2/q2_ 预测精度对比.py

```
# -*- coding: utf-8 -*-
import datetime as dt
import os

import numpy as np
import pandas as pd
```

```
# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


T, DT = 144, 1.0 / 6.0
DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]
DOW = np.array([d.weekday() for d in DATES])
DAYS = np.arange(31, 365)


def load_all():
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    return load, pv


def design(js):
    js = np.asarray(js, float)
    return np.column_stack([
        np.ones_like(js),
        (DOW[js.astype(int)] >= 4) & (DOW[js.astype(int)] <= 5),
        np.sin(2 * np.pi * js / 365.0), np.cos(2 * np.pi * js / 365.0),
        np.sin(4 * np.pi * js / 365.0), np.cos(4 * np.pi * js / 365.0),
    ]).astype(float)


def fit_harmonic(Y, d, back=None):
    lo = 0 if back is None else max(0, d - back)
    js = np.arange(lo, d)
    if len(js) < 12:
        return Y[:d].mean(0) if d else Y.mean(0)
    X = design(js)
    beta, *_ = np.linalg.lstsq(X, Y[js], rcond=None)
    return design([d])[0] @ beta


def nn_same_dow(Y, d, m):
    h = np.where(DOW[:d] == DOW[d])[0]
    if len(h) == 0:
        return Y[:d].mean(0) if d else Y.mean(0)
    return Y[h[-m:]].mean(0)


def ewma_same_dow(Y, d, lam=0.55):
    h = np.where(DOW[:d] == DOW[d])[0]
    if len(h) == 0:
        return Y[:d].mean(0) if d else Y.mean(0)
    h = h[-8:]
    w = lam ** np.arange(len(h) - 1, -1, -1)
    w = w / w.sum()
```

```
    return (w[:, None] * Y[h]).sum(0)


def recent_mean(Y, d, k):
    if d == 0:
        return Y.mean(0)
    return Y[max(0, d - k):d].mean(0)


def shape_level(Y, d, k=5):
    if d < 2:
        return recent_mean(Y, d, k)
    win = Y[max(0, d - k):d]
    tot = win.sum(1, keepdims=True)
    shape = (win / np.maximum(tot, 1e-9)).mean(0)
    return shape * Y[d - 1].sum()


WK = ((DOW >= 4) & (DOW <= 5)).astype(int)


def type_shape_level(Y, d, n_shape=8, level_w=(0.6, 0.25, 0.15), n_ratio=28):
    """ 同类日形状乘近期水平：最近 n_shape 个同类日的归一化曲线，乘最近三天折算到同一日型的加权日总量。"""
    js = [j for j in range(d - 1, -1, -1) if WK[j] == WK[d]][:n_shape]
    shape = (Y[js] / Y[js].sum(1, keepdims=True)).mean(0)
    jr = np.arange(max(0, d - n_ratio), d)
    ratio = Y[jr][WK[jr] == 1].sum(1).mean() / Y[jr][WK[jr] == 0].sum(1).mean()
    tot = Y[:d].sum(1) / np.where(WK[:d] == 1, ratio, 1.0)
    level = sum(w * tot[-1 - i] for i, w in enumerate(level_w)) / sum(level_w)
    return shape * level * (ratio if WK[d] == 1 else 1.0)


LOG = []


def evaluate(name_fns, Y, mask, title):
    print('\n' + title)
    LOG.append('\n' + title)
    act = Y[DAYS]
    base = act[:, mask].mean()
    rows = []
    for nm, fn in name_fns:
        P = np.stack([fn(d) for d in DAYS])
        e = np.abs(act - P)[:, mask]
        rows.append((nm, e.mean(), 100 * e.mean() / base))
    rows.sort(key=lambda r: r[1])
    for nm, mae, rel in rows:
        s = '  %-26s MAE %8.2f kW   相对 %5.2f%%' % (nm, mae, rel)
        print(s)
        LOG.append(s)
    return rows[0]


def main():
    load, pv = load_all()
    day = pv.mean(0) > 1.0
    allslot = np.ones(T, bool)

    best_l = evaluate([
        ('全年同段均值', lambda d: load[:d].mean(0)),
        ('前一天', lambda d: load[d - 1]),
        ('同星期几近 2 次均值', lambda d: nn_same_dow(load, d, 2)),
        ('同星期几近 4 次均值', lambda d: nn_same_dow(load, d, 4)),
        ('同星期几指数加权', lambda d: ewma_same_dow(load, d)),
        ('双向固定效应加谐波回归', lambda d: fit_harmonic(load, d)),
        ('谐波回归近 90 天', lambda d: fit_harmonic(load, d, 90)),
        ('形状乘水平', lambda d: shape_level(load, d, 5)),
        ('同类日形状乘近期水平', lambda d: type_shape_level(load, d)),
    ], load, allslot, '负载的日前预测，2 月 1 日至 12 月 31 日走向前评估')

    best_p = evaluate([
```

```
        ('前一天', lambda d: pv[d - 1]),
        ('近 3 天均值', lambda d: recent_mean(pv, d, 3)),
        ('近 5 天均值', lambda d: recent_mean(pv, d, 5)),
        ('近 10 天均值', lambda d: recent_mean(pv, d, 10)),
        ('指数加权近 8 天',
         lambda d: (lambda w, W: (w[:, None] * W).sum(0))(
             (0.6 ** np.arange(min(8, max(d, 1)) - 1, -1, -1))
             / (0.6 ** np.arange(min(8, max(d, 1)) - 1, -1, -1)).sum(),
             pv[max(0, d - 8):d] if d else pv[:1])),
        ('双向固定效应加谐波回归', lambda d: fit_harmonic(pv, d)),
        ('谐波回归近 90 天', lambda d: fit_harmonic(pv, d, 90)),
        ('形状乘水平', lambda d: shape_level(pv, d, 5)),
    ], pv, day, '光伏的日前预测，只统计白天时段')

    print('\n 结论：负载最优 %s，光伏最优 %s' % (best_l[0], best_p[0]))
    LOG.append('\n 结论：负载最优 %s，光伏最优 %s' % (best_l[0], best_p[0]))
    os.makedirs(OUT, exist_ok=True)
    open(os.path.join(OUT, 'q2_ 预测精度对比.txt'), 'w', encoding='utf-8').write('\n'.join(LOG))


if __name__ == '__main__':
    main()
```

Q2/q2_ 策略对照.py

```
# -*- coding: utf-8 -*-
""" 问题二的策略对照：把抵御预测误差的手段逐项换掉，各跑一遍全年因果仿真，比较总购电费。

对照的算例有：改进前的口径（谐波回归预测、净负载加裕度、全天等厚备用、实时放电意图
要求期末回到 6000 kWh）、逐项改进后的口径、样本平均近似规划、逐日完美预测，以及完全
不做抵御的确定性策略。评价一律取因果仿真的真实费用，规划模型自身的目标值不作数。
"""
import datetime as dt
import os
import sys
import time

import numpy as np
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')


def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')

T, DT = 144, 1.0 / 6.0
ETA, EMAX = 0.9, 5000 / 6
S_LO, S_HI, S0 = 1200.0, 10800.0, 6000.0
EM = 5.0
EPS = 1e-7
```

```
REPORT_FROM = 31
DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]
DOW = np.array([x.weekday() for x in DATES])
WK = ((DOW >= 4) & (DOW <= 5)).astype(int)      # 周五与周六单独成一类


def load_data():
    price = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))['电价'].to_numpy(float)
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    return price, load, pv


PRICE, LOAD, PV = load_data()

# ---------------- 两种负载预测 ----------------
_CACHE = {}


def load_harm(d):
    """ 双向固定效应加两阶年内谐波的回归，改进前用的就是它。"""
    key = ('harm', d)
    if key in _CACHE:
        return _CACHE[key]
    js = np.arange(d)

    def design(j):
        j = np.asarray(j, float)
        return np.column_stack([np.ones_like(j), WK[j.astype(int)],
                                np.sin(2 * np.pi * j / 365), np.cos(2 * np.pi * j / 365),
                                np.sin(4 * np.pi * j / 365), np.cos(4 * np.pi * j / 365)])

    if d < 12:
        v = LOAD[:d].mean(0) if d else LOAD.mean(0)
    else:
        beta, *_ = np.linalg.lstsq(design(js), LOAD[js], rcond=None)
        v = design([d])[0] @ beta
    _CACHE[key] = v
    return v


def load_arf(d, n_shape=8, w=(0.6, 0.25, 0.15), n_ratio=28):
    """ 同类日形状乘近期水平，本文采用的预测。"""
    key = ('arf', d)
    if key in _CACHE:
        return _CACHE[key]
    if d < 14:
        v = load_harm(d)
    else:
        js = [j for j in range(d - 1, -1, -1) if WK[j] == WK[d]][:n_shape]
        shape = (LOAD[js] / LOAD[js].sum(1, keepdims=True)).mean(0)
        jr = np.arange(max(0, d - n_ratio), d)
        ratio = LOAD[jr][WK[jr] == 1].sum(1).mean() / LOAD[jr][WK[jr] == 0].sum(1).mean()
        tot = LOAD[:d].sum(1) / np.where(WK[:d] == 1, ratio, 1.0)
        level = sum(wi * tot[-1 - i] for i, wi in enumerate(w)) / sum(w)
        v = shape * level * (ratio if WK[d] == 1 else 1.0)
    _CACHE[key] = v
    return v


def pv_hist(d, k=5):
    return PV.mean(0) if d == 0 else PV[max(0, d - k):d].mean(0)


# ---------------- 线性规划与逐段执行 ----------------
_MAT = {}


def _mats(n):
    if n not in _MAT:
```

```
        I = sparse.identity(n, format='csr')
        Z = sparse.csr_matrix((n, n))
        Dm = sparse.diags([np.ones(n), -np.ones(n - 1)], [0, -1], format='csr')
        _MAT[n] = (sparse.hstack([-I, I, -I, -I, Z], format='csr'),
                   sparse.hstack([Z, -ETA * I, I / ETA, Z, Dm], format='csr'))
    return _MAT[n]


def day_lp(price, need, s0, s_end=S0, lo=S_LO, g_fix=None):
    """ 变量依次为购电、充电、放电、紧急购电与储电量，各 n 个。"""
    n = len(price)
    A_ub, A_eq = _mats(n)
    b_eq = np.zeros(n)
    b_eq[0] = s0
    cost = np.zeros(5 * n)
    cost[:n] = price
    cost[n:3 * n] = EPS
    cost[3 * n:4 * n] = EM * price
    lb = np.zeros(5 * n)
    ub = np.full(5 * n, np.inf)
    ub[n:3 * n] = EMAX
    lo_v = np.minimum(np.broadcast_to(np.asarray(lo, float), (n,)).copy(), S_HI)
    lo_v[-1] = min(max(lo_v[-1], s_end), S_HI)
    lb[4 * n:] = lo_v
    ub[4 * n:] = S_HI
    if g_fix is not None:
        lb[:n] = g_fix
        ub[:n] = g_fix
    r = linprog(cost, A_ub=A_ub, b_ub=-np.asarray(need, float), A_eq=A_eq, b_eq=b_eq,
                bounds=np.stack([lb, ub], 1), method='highs')
    if not r.success:
        return None
    x = r.x
    return dict(g=x[:n], c=x[n:2 * n], d=x[2 * n:3 * n], e=x[3 * n:4 * n], S=x[4 * n:])


def execute(price, need_act, plan, need_fc, s0, soft_intent):
    """ 逐段揭晓实际净负载。有富余就充电；有缺口时先看储能是否够补后续全部缺口，
    够就直接补齐，不够再解一次剩余时段的放电分配。soft_intent 为真时不要求期末回到 6000。"""
    c = np.zeros(T)
    dis = np.zeros(T)
    e = np.zeros(T)
    s = float(s0)
    for t in range(T):
        net = plan[t] - need_act[t]
        avail_dis = min(EMAX, max(0.0, (s - S_LO) * ETA))
        avail_chg = min(EMAX, max(0.0, (S_HI - s) / ETA))
        if net >= 0:
            ct, dt_ = min(net, avail_chg), 0.0
        else:
            want = -net
            intent = want
            if avail_dis > 1e-9:
                fc = np.concatenate([[need_act[t]], need_fc[t + 1:]])
                future = np.maximum(fc[1:] - plan[t + 1:], 0.0).sum()
                plenty = soft_intent and (s - S_LO) * ETA - min(want, avail_dis) >= future
                if not plenty:
                    r = day_lp(price[t:], fc, s, s_end=S_LO if soft_intent else S0, g_fix=plan[t:])
                    if r is not None:
                        intent = max(r['d'][0], 0.0)
            ct, dt_ = 0.0, min(intent, want, avail_dis)
        e[t] = max(0.0, -net - dt_)
        c[t], dis[t] = ct, dt_
        s = float(np.clip(s + ETA * ct - dt_ / ETA, S_LO, S_HI))
    return dict(c=c, d=dis, e=e, s_end=s)


def saa_plan(price, nf, s0, errs, n_scene=20, tail=0.6, reserve=0.0):
    """ 样本平均近似：各场景共享同一条购电曲线，各自配一套充放电与紧急购电。"""
    n = T
    E = errs[-n_scene:]
```

```
    K = len(E)
    if K < 3:
        return None
    I = sparse.identity(n, format='csr')
    Z = sparse.csr_matrix((n, n))
    Dm = sparse.diags([np.ones(n), -np.ones(n - 1)], [0, -1], format='csr')
    blk_ub = sparse.hstack([I, -I, -I, Z, sparse.csr_matrix((n, 1))], format='csr')
    blk_eq = sparse.hstack([-ETA * I, I / ETA, Z, Dm, sparse.csr_matrix((n, 1))], format='csr')
    end = sparse.csr_matrix(([-1.0, -1.0], ([0, 0], [4 * n - 1, 4 * n])), shape=(1, 4 * n + 1))
    A_ub = sparse.vstack([sparse.hstack([sparse.kron(np.ones((K, 1)), -I), sparse.block_diag([blk_ub] *
    K)]),
                          sparse.hstack([sparse.csr_matrix((K, n)), sparse.block_diag([end] * K)])],
                          format='csr')
    A_eq = sparse.hstack([sparse.csr_matrix((K * n, n)), sparse.block_diag([blk_eq] * K)], format='csr')
    m = 4 * n + 1
    b_ub = np.concatenate([-(nf[None, :] + E).ravel(), np.full(K, -S0)])
    b_eq = np.zeros(K * n)
    b_eq[::n] = s0
    c = np.zeros(n + K * m)
    c[:n] = price
    blk = np.zeros(m)
    blk[:2 * n] = EPS
    blk[2 * n:3 * n] = EM * price / K
    blk[-1] = tail / K
    c[n:] = np.tile(blk, K)
    lo = min(S_LO + reserve, s0)
    lb_b = np.zeros(m)
    ub_b = np.full(m, np.inf)
    ub_b[:2 * n] = EMAX
    lb_b[3 * n:4 * n] = lo
    ub_b[3 * n:4 * n] = S_HI
    lb = np.concatenate([np.zeros(n), np.tile(lb_b, K)])
    ub = np.concatenate([np.full(n, np.inf), np.tile(ub_b, K)])
    r = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq, bounds=np.stack([lb, ub], 1),
    method='highs')
    return r.x[:n] if r.success else None


def run_year(cfg):
    s = S0
    tot = dict(plan=0.0, em=0.0, kwh=0.0, days=0)
    errs = []
    for d in range(365):
        need_act = (LOAD[d] - PV[d]) * DT
        if cfg.get('oracle'):
            lf, pf = LOAD[d], PV[d]
        else:
            lf = load_arf(d) if cfg.get('load', 'arf') == 'arf' else load_harm(d)
            pf = pv_hist(d)
        nf = (lf - pf) * DT
        errs.append(need_act - nf)
        if 'pv_cut' in cfg:
            need_plan = (lf - (1 - cfg['pv_cut']) * pf) * DT
        else:
            need_plan = nf + cfg.get('margin', 0.0) * np.abs(nf)
        if 'reserve_am' in cfg:
            tt = np.arange(T)
            lo = np.minimum(S_LO + np.where(tt < 78, cfg['reserve_am'], cfg['reserve_pm']), s)
        else:
            lo = min(S_LO + cfg.get('reserve', 0.0), s, S_HI)
        r = day_lp(PRICE, need_plan, s, s_end=max(S0, float(np.atleast_1d(lo)[-1])), lo=lo)
        if r is None:
            r = day_lp(PRICE, need_plan, s, s_end=S_LO)
        g = r['g']
        if cfg.get('saa') and d >= 20:
            gs = saa_plan(PRICE, nf, s, np.array(errs[:-1]), reserve=cfg.get('reserve', 0.0))
            if gs is not None:
                g = gs
        st = execute(PRICE, need_act, g, nf, s, cfg.get('soft_intent', True))
        if d >= REPORT_FROM:
            tot['plan'] += float(PRICE @ g)
```

```
            tot['em'] += float(EM * (PRICE @ st['e']))
            tot['kwh'] += float(st['e'].sum())
            tot['days'] += int(st['e'].sum() > 1e-6)
        s = st['s_end']
    tot['total'] = tot['plan'] + tot['em']
    return tot


CASES = [
    ('改进前：谐波回归、净负载裕度 1.5%、备用 1000、硬期末',
     dict(load='harm', margin=0.015, reserve=1000.0, soft_intent=False)),
    ('软化实时放电意图', dict(load='harm', margin=0.015, reserve=1000.0)),
    ('换成同类日形状乘近期水平', dict(load='arf', margin=0.015, reserve=1000.0)),
    ('改按光伏折减 5%，备用 1000', dict(load='arf', pv_cut=0.05, reserve=1000.0)),
    ('分段备用 500 与 1000（本文）', dict(load='arf', pv_cut=0.05, reserve_am=500.0, reserve_pm=1000.0)),
    ('光伏折减 3%，备用 1000', dict(load='arf', pv_cut=0.03, reserve=1000.0)),
    ('光伏折减 7%，备用 1000', dict(load='arf', pv_cut=0.07, reserve=1000.0)),
    ('光伏折减 5%，备用 750', dict(load='arf', pv_cut=0.05, reserve=750.0)),
    ('光伏折减 5%，备用 1250', dict(load='arf', pv_cut=0.05, reserve=1250.0)),
    ('分段备用 0 与 1250', dict(load='arf', pv_cut=0.05, reserve_am=0.0, reserve_pm=1250.0)),
    ('分段备用 1000 与 500', dict(load='arf', pv_cut=0.05, reserve_am=1000.0, reserve_pm=500.0)),
    ('样本平均近似规划', dict(load='arf', reserve=500.0, saa=True)),
    ('不做任何抵御', dict(load='arf', margin=0.0, reserve=0.0)),
    ('逐日完美预测', dict(oracle=True, margin=0.0, reserve=0.0)),
]


def main():
    sel = sys.argv[1:]
    lines = ['问题二的策略对照，统计 2 月 1 日至 12 月 31 日',
             '  %-38s %14s %13s %12s %9s %5s' % ('算例', '总费用', '计划购电费', '紧急购电费', '紧急电量',
             '天数')]
    for name, cfg in CASES:
        if sel and not any(s in name for s in sel):
            continue
        t0 = time.time()
        r = run_year(cfg)
        lines.append('  %-38s %14.2f %13.2f %12.2f %9.0f %5d' %
                     (name, r['total'], r['plan'], r['em'], r['kwh'], r['days']))
        print(lines[-1], '  用时 %.0f s' % (time.time() - t0), flush=True)
    os.makedirs(OUT, exist_ok=True)
    open(os.path.join(OUT, 'q2_ 策略对照.txt'), 'w', encoding='utf-8').write('\n'.join(lines))


if __name__ == '__main__':
    main()
```

Q2/q2_ 事后下界.py

```
# -*- coding: utf-8 -*-
""" 事后最优下界：假设全年的负载与光伏都已知，把 365 天当成一个整体线性规划求解。

这个值不可能在实际中达到，它的用处是给出滚动策略还差多少。固定电价与波动电价各算一次。
"""
import datetime as dt
import os
import time

import numpy as np
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')


def _find_root():
```

```
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')

T, DT = 144, 1.0 / 6.0
ETA, EMAX = 0.9, 5000 / 6
S_LO, S_HI, S0 = 1200.0, 10800.0, 6000.0
DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]


def load_data():
    price1 = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))['电价'].to_numpy(float)
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    price4 = pd.read_excel(os.path.join(DATA, '附件 4.xlsx')).iloc[:, 1:].to_numpy(float)
    return price1, load, pv, price4


def year_lp(price, need, s0=S0):
    """ 变量依次为购电、充电、放电与储电量，长度都等于时段总数。"""
    n = price.size
    I = sparse.identity(n, format='csr')
    Dm = sparse.diags([np.ones(n), -np.ones(n - 1)], [0, -1], format='csr')
    A_ub = sparse.hstack([-I, I, -I, sparse.csr_matrix((n, n))], format='csr')
    A_eq = sparse.hstack([sparse.csr_matrix((n, n)), -ETA * I, I / ETA, Dm], format='csr')
    b_eq = np.zeros(n)
    b_eq[0] = s0
    cost = np.concatenate([price, np.full(n, 1e-7), np.full(n, 1e-7), np.zeros(n)])
    lo = np.concatenate([np.zeros(3 * n), np.full(n, S_LO)])
    hi = np.concatenate([np.full(n, np.inf), np.full(2 * n, EMAX), np.full(n, S_HI)])
    r = linprog(cost, A_ub=A_ub, b_ub=-need, A_eq=A_eq, b_eq=b_eq,
                bounds=np.stack([lo, hi], 1), method='highs')
    assert r.success, r.message
    x = r.x
    return x[:n], x[3 * n:]


def main():
    price1, load, pv, price4 = load_data()
    need = ((load - pv) * DT).ravel()
    lines = ['全年事后最优下界，统计 2 月 1 日至 12 月 31 日']
    for name, P in (('固定电价', np.tile(price1, (365, 1))), ('波动电价', price4)):
        t0 = time.time()
        g, S = year_lp(P.ravel(), need)
        cost_day = (P * g.reshape(365, T)).sum(1)
        s_jan = S.reshape(365, T)[30, -1]
        g2, _ = year_lp(P[31:].ravel(), need.reshape(365, T)[31:].ravel(), s0=s_jan)
        lines.append('  %s: 全年 %.2f 元，2 月 1 日起 %.2f 元，1 月末储电量 %.1f kWh，用时 %.1f s' %
                     (name, cost_day.sum(), float((P[31:].ravel() * g2).sum()), s_jan, time.time() - t0))
        print(lines[-1], flush=True)
    os.makedirs(OUT, exist_ok=True)
    open(os.path.join(OUT, 'q2_ 事后下界.txt'), 'w', encoding='utf-8').write('\n'.join(lines))


if __name__ == '__main__':
    main()
```

Q3/q3_ 求解.py

```
# -*- coding: utf-8 -*-
import datetime as dt
import os

import numpy as np
import openpyxl
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


T = 144
DT = 1.0 / 6.0
P_MAX = 5000.0
E_MAX = P_MAX * DT
S_LO, S_HI = 1200.0, 10800.0
S_INIT = 6000.0
ETA = 0.9
EMERG = 5.0
DEV_LO, DEV_HI = 0.5, 1.5
# 计划与调整所用的安全裕度与备用：净负载按 n + MARGIN_N*|n| + (MARGIN_L* 负载 + MARGIN_P* 光伏)*DT 加码
PLAN = dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=500.0)
ADJ = dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=250.0)
SPLIT = 78
ISSUE_HOUR = (0, 6, 12, 18)
ISSUE_SLOT = (0, 36, 72, 108)
ADJUST = (1, 2, 3)
W0, W_SLOPE, W_MAX = 0.95, 0.05, 0.95   # 官方预报权重 = W0 - W_SLOPE * 提前小时数
K_PV = 5
N_SHAPE = 8
LEVEL_W = (0.6, 0.25, 0.15)
N_RATIO = 28
REPORT_FROM = 31
EPS = 1e-7

DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]
DOW = np.array([d.weekday() for d in DATES])
WK = ((DOW >= 4) & (DOW <= 5)).astype(int)
BLOCKS = [('0:00-4:00', slice(0, 24)), ('4:00-8:00', slice(24, 48)),
          ('8:00-12:00', slice(48, 72)), ('12:00-16:00', slice(72, 96)),
          ('16:00-20:00', slice(96, 120)), ('20:00-24:00', slice(120, 144))]


def load_a1():
    d = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))
    return d['电价'].to_numpy(float)


def load_a2():
```

```
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    return load, pv


def load_a3():
    d = pd.read_excel(os.path.join(DATA, '附件 3.xlsx'))
    return d.iloc[:, 2:].to_numpy(float).reshape(365, 4, 24)


# ---------------- 预测 ----------------
_CACHE = {}


def _harm(load, d):
    js = np.arange(d)
    X = lambda j: np.column_stack([np.ones(len(j)), WK[j],
                                   np.sin(2 * np.pi * j / 365), np.cos(2 * np.pi * j / 365),
                                   np.sin(4 * np.pi * j / 365), np.cos(4 * np.pi * j / 365)])
    if d < 12:
        return load[:d].mean(0) if d > 0 else load.mean(0)
    b, *_ = np.linalg.lstsq(X(js), load[js], rcond=None)
    return X(np.array([d]))[0] @ b


def load_hat(load, d):
    """ 日型形状乘日总量水平：形状取最近 N_SHAPE 个同类日的归一化曲线，
    水平取最近三天折算到同一日型后的加权日总量。"""
    if d in _CACHE:
        return _CACHE[d]
    if d < 14:
        v = _harm(load, d)
    else:
        js = [j for j in range(d - 1, -1, -1) if WK[j] == WK[d]][:N_SHAPE]
        shape = (load[js] / load[js].sum(1, keepdims=True)).mean(0)
        jr = np.arange(max(0, d - N_RATIO), d)
        ratio = load[jr][WK[jr] == 1].sum(1).mean() / load[jr][WK[jr] == 0].sum(1).mean()
        tot = load[:d].sum(1) / np.where(WK[:d] == 1, ratio, 1.0)
        level = sum(w * tot[-1 - i] for i, w in enumerate(LEVEL_W)) / sum(LEVEL_W)
        v = shape * level * (ratio if WK[d] == 1 else 1.0)
    _CACHE[d] = v
    return v


def forecast_to_slots(pv, fc, d, issue):
    """ 整点预报插值到各时段末刻。附件的第 k 个数是 (k+1)*10 分钟时刻的功率，
    发布时刻之后第一个整点之前的时段，用发布时刻已实测的功率做左端点。"""
    h0 = ISSUE_HOUR[issue]
    hrs = np.arange(h0 + 1, h0 + 25, dtype=float)
    f = fc[d, issue]
    if h0 > 0:
        hrs = np.concatenate([[h0], hrs])
        f = np.concatenate([[pv[d, h0 * 6 - 1]], f])
    tm = (np.arange(T) + 1) * DT
    return np.maximum(np.interp(tm, hrs, f), 0.0)


def pv_hat(pv, fc, d, issue):
    hist = pv.mean(0) if d == 0 else pv[max(0, d - K_PV):d].mean(0)
    lead = (np.arange(T) + 1) * DT - ISSUE_HOUR[issue]
    w = np.clip(W0 - W_SLOPE * lead, 0.0, W_MAX)
    return w * forecast_to_slots(pv, fc, d, issue) + (1 - w) * hist


def hedged_need(load_fc, pv_fc, prm):
    n = (load_fc - pv_fc) * DT
    return n + prm['MARGIN_N'] * np.abs(n) + (prm['MARGIN_L'] * load_fc + prm['MARGIN_P'] * pv_fc) * DT


# ---------------- 线性规划 ----------------
```

```
_MAT = {}


def _mats(n, adj):
    key = (n, adj)
    if key not in _MAT:
        I = sparse.identity(n, format='csr'); Z = sparse.csr_matrix((n, n))
        Dm = sparse.diags([np.ones(n), -np.ones(n - 1)], [0, -1], format='csr')
        if adj:
            A_ub = sparse.hstack([-I, I, -I, -I, Z, Z, Z], format='csr')
            A_eq = sparse.vstack([sparse.hstack([Z, -ETA * I, I / ETA, Z, Dm, Z, Z]),
                                  sparse.hstack([I, Z, Z, Z, Z, -I, I])], format='csr')
        else:
            A_ub = sparse.hstack([-I, I, -I, -I, Z], format='csr')
            A_eq = sparse.hstack([Z, -ETA * I, I / ETA, Z, Dm], format='csr')
        _MAT[key] = (A_ub, A_eq)
    return _MAT[key]


def day_lp(price, need, s0, s_end=S_INIT, lo=S_LO, g_fix=None, ref=None):
    """ 变量依次为购电 g、充电 c、放电 d、紧急购电 e、储电量 S;
    给出 ref 时为调整模型，另加超额量 u 与违约量 z，g = ref + u - z。"""
    n = len(price); adj = ref is not None
    A_ub, A_eq = _mats(n, adj)
    nb = 7 if adj else 5
    b_eq = np.zeros(A_eq.shape[0]); b_eq[0] = s0
    if adj:
        b_eq[n:] = ref
    cost = np.zeros(nb * n)
    cost[:n] = 0.0 if adj else price
    cost[n:3 * n] = EPS
    cost[3 * n:4 * n] = EMERG * price
    if adj:
        cost[5 * n:6 * n] = DEV_HI * price
        cost[6 * n:7 * n] = -DEV_LO * price
    lb = np.zeros(nb * n); ub = np.full(nb * n, np.inf)
    ub[n:3 * n] = E_MAX
    lo_v = np.minimum(np.broadcast_to(np.asarray(lo, float), (n,)).copy(), S_HI)
    lo_v[-1] = min(max(lo_v[-1], s_end), S_HI)
    lb[4 * n:5 * n] = lo_v; ub[4 * n:5 * n] = S_HI
    if g_fix is not None:
        lb[:n] = g_fix; ub[:n] = g_fix
    res = linprog(cost, A_ub=A_ub, b_ub=-np.asarray(need, float), A_eq=A_eq, b_eq=b_eq,
                  bounds=np.stack([lb, ub], 1), method='highs')
    if not res.success:
        return None
    x = res.x
    return dict(g=x[:n], c=x[n:2 * n], d=x[2 * n:3 * n], e=x[3 * n:4 * n], S=x[4 * n:5 * n])


def solve_hedged(price, need, s0, prm, ref=None, t0=0):
    """ 备用在 SPLIT 段之前取 RESERVE，之后取 RESERVE_E，未给 RESERVE_E 时全天相同。"""
    tt = np.arange(t0, T)
    lo = S_LO + np.where(tt < SPLIT, prm['RESERVE'], prm.get('RESERVE_E', prm['RESERVE']))
    lo = np.minimum(np.minimum(lo, s0), S_HI)
    r = day_lp(price, need, s0, s_end=max(S_INIT, lo[-1]), lo=lo, ref=ref)
    if r is None:
        r = day_lp(price, need, s0, s_end=S_LO, ref=ref)
    return r


def step(price, need_act, a, need_fc, s, t, out):
    """ 执行第 t 段。有富余就充电；有缺口时先判断储能是否足够覆盖预测的后续全部缺口，
    够就直接放电补齐，不够再解剩余时段的放电分配问题。"""
    net = a[t] - need_act[t]
    avail_dis = min(E_MAX, max(0.0, (s - S_LO) * ETA))
    avail_chg = min(E_MAX, max(0.0, (S_HI - s) / ETA))
    if net >= 0:
        ct, dt_ = min(net, avail_chg), 0.0
    else:
        want = -net
```

```
        intent = want
        if avail_dis > 1e-9:
            fc = np.concatenate([[need_act[t]], need_fc[t + 1:]])
            future_def = np.maximum(fc[1:] - a[t + 1:], 0.0).sum()
            if (s - S_LO) * ETA - min(want, avail_dis) < future_def:
                r = day_lp(price[t:], fc, s, s_end=S_LO, g_fix=a[t:])
                if r is not None:
                    intent = max(r['d'][0], 0.0)
        ct, dt_ = 0.0, min(intent, want, avail_dis)
    out['e'][t] = max(0.0, -net - dt_)
    out['c'][t], out['d'][t] = ct, dt_
    s = float(np.clip(s + ETA * ct - dt_ / ETA, S_LO, S_HI))
    out['S'][t] = s
    return s


def run_day(price, need_act, load_fc, pv_fc_by_issue, s0, adjust=ADJUST):
    need_fc = (load_fc - pv_fc_by_issue[0]) * DT
    plan = solve_hedged(price, hedged_need(load_fc, pv_fc_by_issue[0], PLAN), s0, PLAN)['g']
    a = plan.copy()
    out = dict(c=np.zeros(T), d=np.zeros(T), e=np.zeros(T), S=np.zeros(T))
    s = float(s0)
    for t in range(T):
        if t in ISSUE_SLOT[1:] and ISSUE_SLOT.index(t) in adjust:
            i = ISSUE_SLOT.index(t)
            need_fc = (load_fc - pv_fc_by_issue[i]) * DT
            need_fc[:t] = need_act[:t]
            nd = hedged_need(load_fc, pv_fc_by_issue[i], ADJ)
            r = solve_hedged(price[t:], nd[t:], s, ADJ, ref=plan[t:], t0=t)
            if r is not None:
                a[t:] = r['g']
        s = step(price, need_act, a, need_fc, s, t, out)
    up = np.maximum(a - plan, 0.0)
    dn = np.maximum(plan - a, 0.0)
    cost_plan = float(price @ plan)
    cost_dev = float(DEV_HI * (price @ up) - DEV_LO * (price @ dn))
    cost_emerg = float(EMERG * (price @ out['e']))
    return dict(a=a, s_end=s, plan=plan, cost_plan=cost_plan, cost_dev=cost_dev,
                cost_emerg=cost_emerg, cost=cost_plan + cost_dev + cost_emerg, **out)


def run_year(prices, load, pv, fc, n_days=365, adjust=ADJUST):
    s = float(S_INIT)
    recs = []
    for d in range(n_days):
        need_act = (load[d] - pv[d]) * DT
        pvs = [pv_hat(pv, fc, d, i) for i in range(4)]
        rec = run_day(prices[d], need_act, load_hat(load, d), pvs, s, adjust)
        rec['day'] = d
        rec['s0'] = s
        s = rec['s_end']
        recs.append(rec)
        if d % 30 == 0:
            print('  第 %3d 天  费用 %9.2f  调整 %9.2f  紧急 %8.2f  期末储电 %8.1f'
                  % (d, rec['cost'], rec['cost_dev'], rec['cost_emerg'], s), flush=True)
    return recs


def slot_labels():
    ws = openpyxl.load_workbook(os.path.join(DATA, '附件 5', 'result3.xlsx'))['计划购电量']
    return [str(ws.cell(1, 2 + k).value) for k in range(T)]


LABELS = slot_labels()


def emergency_intervals(e, tol=1e-6):
    out = []
    k = 0
    while k < T:
        if e[k] > tol:
```

```
            j = k
            while j + 1 < T and e[j + 1] > tol:
                j += 1
            out.append(('%s-%s' % (LABELS[k].split('-')[0], LABELS[j].split('-')[-1]),
                       float(e[k:j + 1].sum())))
            k = j + 1
        else:
            k += 1
    return out


def r4(x):
    return float(np.round(float(x), 4))


def clear_below(ws, first_row=2):
    for row in ws.iter_rows(min_row=first_row, max_row=ws.max_row):
        for cell in row:
            cell.value = None


def write_grid(ws, recs, key, cost_key='cost'):
    clear_below(ws)
    for i, r in enumerate(recs):
        row = 2 + i
        ws.cell(row, 1, DATES[r['day']].strftime('%Y/%m/%d'))
        v = r[key]
        for k in range(T):
            ws.cell(row, 2 + k, r4(v[k]))
        ws.cell(row, 2 + T, r4(float(np.sum(v))))
        ws.cell(row, 3 + T, r4(float(r[cost_key])))


def write_result(recs, name='result3.xlsx', template='result3.xlsx', with_adj=True):
    wb = openpyxl.load_workbook(os.path.join(DATA, '附件 5', template))
    write_grid(wb['计划购电量'], recs, 'plan')
    if with_adj:
        write_grid(wb['调整购电量'], recs, 'a')

    ws = wb['充放电量']
    clear_below(ws)
    row = 2
    for r in recs:
        for j, (nm, sl) in enumerate(BLOCKS):
            ws.cell(row + j, 1, DATES[r['day']].strftime('%Y/%m/%d') if j == 0 else None)
            ws.cell(row + j, 2, nm)
            ws.cell(row + j, 3, r4(float(r['c'][sl].sum())))
            ws.cell(row + j, 4, r4(float(r['d'][sl].sum())))
        ws.cell(row, 5, '0:00')
        ws.cell(row, 6, r4(r['s0']))
        ws.cell(row + 1, 5, '24:00')
        ws.cell(row + 1, 6, r4(r['s_end']))
        row += 6

    ws = wb['紧急购电量']
    clear_below(ws)
    row = 2
    for r in recs:
        iv = emergency_intervals(r['e'])
        if not iv:
            ws.cell(row, 1, DATES[r['day']].strftime('%Y/%m/%d'))
            ws.cell(row, 2, '无')
            ws.cell(row, 3, 0.0)
            row += 1
            continue
        for j, (lab, v) in enumerate(iv):
            ws.cell(row + j, 1, DATES[r['day']].strftime('%Y/%m/%d') if j == 0 else None)
            ws.cell(row + j, 2, lab)
            ws.cell(row + j, 3, r4(v))
        row += len(iv)
```

```
    os.makedirs(OUT, exist_ok=True)
    path = os.path.join(OUT, name)
    wb.save(path)
    return path


def summary(recs, title='问题 3'):
    sub = recs[REPORT_FROM:]
    tot = sum(r['cost'] for r in sub)
    L = ['%s  统计 %s 至 %s，共 %d 天'
         % (title, DATES[sub[0]['day']], DATES[sub[-1]['day']], len(sub)),
         '  总购电费 %.2f 元' % tot,
         '  计划购电费 %.2f 元' % sum(r['cost_plan'] for r in sub),
         '  调整相关费 %.2f 元' % sum(r['cost_dev'] for r in sub),
         '  紧急购电费 %.2f 元' % sum(r['cost_emerg'] for r in sub),
         '  紧急购电量 %.2f kWh，发生在 %d 天'
         % (sum(float(r['e'].sum()) for r in sub),
            sum(1 for r in sub if r['e'].sum() > 1e-6))]
    return '\n'.join(L)


def main():
    price = load_a1()
    load, pv = load_a2()
    fc = load_a3()
    prices = np.tile(price, (365, 1))
    print('开始滚动求解，共 365 天，每天 0:00 定计划，6:00 12:00 18:00 各调整一次')
    recs = run_year(prices, load, pv, fc)
    print()
    print(summary(recs))
    print('已写出', write_result(recs[REPORT_FROM:]))


if __name__ == '__main__':
    main()
```

Q3/q3_ 算法对比.py

```
# -*- coding: utf-8 -*-
import datetime as dt
import os
import time

import numpy as np
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填，也可以用环境变量 C2026_DATA 与 C2026_OUT 指定，
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')
```

```
T, DT = 144, 1.0 / 6.0
ETA, S_LO, S_HI, S_INIT = 0.9, 1200.0, 10800.0, 6000.0
E_MAX = 5000.0 * DT
EMERG, DEV = 5.0, 0.5
# 与求解脚本一致的新口径
Q2_MARGIN_P, Q2_RESERVE_M, Q2_RESERVE_E, SPLIT = 0.05, 500.0, 1000.0, 78
Q3_MARGIN, Q3_RESERVE_PLAN, Q3_RESERVE_ADJ = 0.015, 500.0, 250.0
K_PV, N_SHAPE, LEVEL_W, N_RATIO = 5, 8, (0.6, 0.25, 0.15), 28
W0, W_SLOPE, W_MAX = 0.95, 0.05, 0.95
N_RUN = 10
DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]
DOW = np.array([d.weekday() for d in DATES])
WK = ((DOW >= 4) & (DOW <= 5)).astype(int)
SAMPLE = [DATES.index(dt.date(2025, m, 15)) for m in range(2, 13)] + \
         [DATES.index(dt.date(2025, 1, 15))]


def load_all():
    a1 = pd.read_excel(os.path.join(DATA, '附件 1.xlsx'))
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    return a1['电价'].to_numpy(float), load, pv


def load_a3():
    d = pd.read_excel(os.path.join(DATA, '附件 3.xlsx'))
    return d.iloc[:, 2:].to_numpy(float).reshape(365, 4, 24)


def load_hat(load, d):
    """ 同类日形状乘近期水平，取样日都在 2 月之后，历史足够。"""
    js = [j for j in range(d - 1, -1, -1) if WK[j] == WK[d]][:N_SHAPE]
    shape = (load[js] / load[js].sum(1, keepdims=True)).mean(0)
    jr = np.arange(max(0, d - N_RATIO), d)
    ratio = load[jr][WK[jr] == 1].sum(1).mean() / load[jr][WK[jr] == 0].sum(1).mean()
    tot = load[:d].sum(1) / np.where(WK[:d] == 1, ratio, 1.0)
    level = sum(w * tot[-1 - i] for i, w in enumerate(LEVEL_W)) / sum(LEVEL_W)
    return shape * level * (ratio if WK[d] == 1 else 1.0)


def pv_hist(pv, d):
    return pv[max(0, d - K_PV):d].mean(0)


def pv_blend(pv, fc, d, issue):
    h0 = (0, 6, 12, 18)[issue]
    hrs = np.arange(h0 + 1, h0 + 25, dtype=float); f = fc[d, issue]
    if h0 > 0:
        hrs = np.concatenate([[h0], hrs]); f = np.concatenate([[pv[d, h0 * 6 - 1]], f])
    tm = (np.arange(T) + 1) * DT
    w = np.clip(W0 - W_SLOPE * (tm - h0), 0.0, W_MAX)
    return w * np.maximum(np.interp(tm, hrs, f), 0.0) + (1 - w) * pv_hist(pv, d)


def reserve_q2():
    return np.where(np.arange(T) < SPLIT, Q2_RESERVE_M, Q2_RESERVE_E)


def need_plan(load, pv, d):
    """ 问题二日前计划子问题的净负载：光伏按预测的 95% 计入。"""
    return (load_hat(load, d) - (1 - Q2_MARGIN_P) * pv_hist(pv, d)) * DT


def need_q3(load, pv, fc, d, issue):
    """ 问题三的净负载：issue=0 为 0:00 计划，issue=1 为 6:00 调整，此前的时段已揭晓取实际值。"""
    n = (load_hat(load, d) - pv_blend(pv, fc, d, issue)) * DT
    n = n + Q3_MARGIN * np.abs(n)
    if issue > 0:
        t0 = (0, 36, 72, 108)[issue]
```

```
        n[:t0] = (load[d, :t0] - pv[d, :t0]) * DT
    return n


# ---------------------------------------------------------------------- 精确解法

def _lo(reserve, s0):
    return np.minimum(np.minimum(S_LO + np.broadcast_to(np.asarray(reserve, float), (T,)), s0), S_HI)


def build(price, need, s0, s_end, reserve, ref=None):
    n = T
    s_lo = _lo(reserve, s0)
    I = sparse.identity(n, format='csr')
    Z = sparse.csr_matrix((n, n))
    Tri = sparse.csr_matrix(np.tril(np.ones((n, n))))
    one = sparse.csr_matrix(np.ones((1, n)))
    z1 = sparse.csr_matrix((1, n))
    nb = 6 if ref is not None else 4
    blocks = lambda *b: sparse.hstack(list(b) + [Z] * (nb - len(b)), format='csr')
    b1 = lambda *b: sparse.hstack(list(b) + [z1] * (nb - len(b)), format='csr')
    A_ub = sparse.vstack([
        blocks(-I, I, -I, -I),
        blocks(Z, ETA * Tri, -Tri / ETA, Z),
        blocks(Z, -ETA * Tri, Tri / ETA, Z),
        b1(z1, -ETA * one, one / ETA, z1)], format='csr')
    b_ub = np.concatenate([-need, np.full(n, S_HI - s0),
                           s0 - s_lo,
                           np.array([s0 - max(s_end, s_lo[-1])])])
    A_eq = b_eq = None
    cost = np.zeros(nb * n)
    cost[n:3 * n] = 1e-9
    cost[3 * n:4 * n] = EMERG * price
    if ref is None:
        cost[:n] = price
    else:
        A_eq = blocks(I, Z, Z, Z, -I, I)
        b_eq = np.asarray(ref, float)
        cost[4 * n:5 * n] = (1 + DEV) * price
        cost[5 * n:6 * n] = -DEV * price
    lo = np.zeros(nb * n)
    hi = np.full(nb * n, np.inf)
    hi[n:2 * n] = E_MAX
    hi[2 * n:3 * n] = E_MAX
    return A_ub, b_ub, A_eq, b_eq, cost, np.stack([lo, hi], axis=1), nb


def solve_lp(price, need, s0, s_end, reserve, ref, method):
    A_ub, b_ub, A_eq, b_eq, cost, bnd, nb = build(price, need, s0, s_end, reserve, ref)
    t0 = time.time()
    r = linprog(cost, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                bounds=bnd, method=method)
    dtm = time.time() - t0
    base = float(price @ np.asarray(ref)) if ref is not None else 0.0
    return float(r.fun) + base, dtm, r.success


def solve_dp(price, need, s0, s_end, reserve, ref, step=25.0):
    """ 储电量从下限起每 25 kWh 一档，逐段下限随备用变化，低于当段下限的档位不可达。"""
    s_lo = _lo(reserve, s0)
    base_lo = min(S_LO, s0)
    grid = np.arange(base_lo, S_HI + 1e-9, step)
    n = len(grid)
    i0 = int(round((s0 - base_lo) / step))
    i0 = min(max(i0, 0), n - 1)
    t0 = time.time()
    delta = grid[None, :] - grid[:, None]
    chg = np.where(delta >= 0, delta / ETA, 0.0)
    dis = np.where(delta < 0, -delta * ETA, 0.0)
    feas = (chg <= E_MAX + 1e-9) & (dis <= E_MAX + 1e-9)
    base = chg - dis
```

```
    J = np.where(grid >= max(s_end, s_lo[-1]) - 1e-6, 0.0, np.inf)
    for t in range(T - 1, -1, -1):
        J = np.where(grid >= s_lo[t] - 1e-6, J, np.inf)
        take = np.maximum(need[t] + base, 0.0)
        if ref is None:
            step_cost = price[t] * take
        else:
            step_cost = price[t] * take + DEV * price[t] * np.abs(take - ref[t])
        step_cost = np.where(feas, step_cost, np.inf)
        J = np.min(step_cost + J[None, :], axis=1)
    return float(J[i0]), time.time() - t0, np.isfinite(J[i0])


# ---------------------------------------------------------------- 启发式

def decode(U, price, need, s0, ref):
    U = np.clip(U, -1.0, 1.0)
    m = U.shape[0]
    s = np.full(m, s0)
    C = np.zeros((m, T)); D = np.zeros((m, T))
    for t in range(T):
        up = U[:, t]
        ct = np.where(up >= 0, np.minimum(up * E_MAX, (S_HI - s) / ETA), 0.0)
        dt_ = np.where(up < 0, np.minimum(-up * E_MAX, (s - S_LO) * ETA), 0.0)
        C[:, t], D[:, t] = ct, dt_
        s = s + ETA * ct - dt_ / ETA
    A = np.maximum(need[None, :] + C - D, 0.0)
    raw = A @ price
    if ref is not None:
        raw = raw + DEV * (np.abs(A - ref[None, :]) @ price)
    pen = np.abs(s - S_INIT) * price.max() * 5.0
    return raw + pen, raw


def ga(price, need, s0, ref, seed, pop=80, gen=250):
    rng = np.random.default_rng(seed)
    t0 = time.time()
    U = rng.uniform(-1, 1, (pop, T))
    fit, raw = decode(U, price, need, s0, ref)
    for _ in range(gen):
        idx = rng.integers(0, pop, (pop, 2))
        win = np.where(fit[idx[:, 0]] < fit[idx[:, 1]], idx[:, 0], idx[:, 1])
        P = U[win]
        mask = rng.random((pop, T)) < 0.5
        Q = np.where(mask, P, P[rng.permutation(pop)])
        Q = Q + (rng.random((pop, T)) < 0.08) * rng.normal(0, 0.25, (pop, T))
        f2, r2 = decode(Q, price, need, s0, ref)
        b = np.argmin(fit); w = np.argmax(f2)
        Q[w], f2[w], r2[w] = U[b], fit[b], raw[b]
        U, fit, raw = Q, f2, r2
    return float(raw[np.argmin(fit)]), time.time() - t0, True


def pso(price, need, s0, ref, seed, pop=80, it=250):
    rng = np.random.default_rng(seed)
    t0 = time.time()
    X = rng.uniform(-1, 1, (pop, T))
    V = rng.uniform(-0.2, 0.2, (pop, T))
    fit, raw = decode(X, price, need, s0, ref)
    pb, pbf, pbr = X.copy(), fit.copy(), raw.copy()
    gi = int(np.argmin(pbf))
    for k in range(it):
        w = 0.9 - 0.5 * k / it
        V = np.clip(w * V + 1.5 * rng.random((pop, T)) * (pb - X)
                    + 1.5 * rng.random((pop, T)) * (pb[gi] - X), -0.4, 0.4)
        X = np.clip(X + V, -1, 1)
        fit, raw = decode(X, price, need, s0, ref)
        imp = fit < pbf
        pb[imp], pbf[imp], pbr[imp] = X[imp], fit[imp], raw[imp]
        gi = int(np.argmin(pbf))
    return float(pbr[gi]), time.time() - t0, True
```

```
def sa_batch(price, need, s0, ref, seed, chains=N_RUN, iters=20000):
    """ 多条独立退火链一起跑，一次解码算完所有链，返回每条链的最好值。"""
    rng = np.random.default_rng(seed)
    t0 = time.time()
    u = rng.uniform(-1, 1, (chains, T))
    f, raw = decode(u, price, need, s0, ref)
    bf, braw = f.copy(), raw.copy()
    rows = np.arange(chains)
    for k in range(iters):
        Tp = 5000.0 * (1 - k / iters) ** 2 + 1.0
        v = u.copy()
        j = rng.integers(0, T, (chains, 3))
        v[rows[:, None], j] = np.clip(
            v[rows[:, None], j] + rng.normal(0, 0.3, (chains, 3)), -1, 1)
        f2, r2 = decode(v, price, need, s0, ref)
        acc = (f2 < f) | (rng.random(chains) < np.exp(-(f2 - f) / Tp))
        u[acc], f[acc], raw[acc] = v[acc], f2[acc], r2[acc]
        imp = f < bf
        bf[imp], braw[imp] = f[imp], raw[imp]
    return braw, (time.time() - t0) / chains


def rule(price, need, s0, ref):
    t0 = time.time()
    lo_q, hi_q = np.quantile(price, 0.35), np.quantile(price, 0.65)
    s = s0
    c = np.zeros(T); d = np.zeros(T)
    for t in range(T):
        if price[t] <= lo_q:
            ct = min(E_MAX, (S_HI - s) / ETA); c[t] = ct; s += ETA * ct
        elif price[t] >= hi_q:
            dt_ = min(E_MAX, (s - S_LO) * ETA, max(need[t], 0.0))
            d[t] = dt_; s -= dt_ / ETA
    a = np.maximum(need + c - d, 0.0)
    v = float(price @ a)
    if ref is not None:
        v += DEV * float(price @ np.abs(a - ref))
    return v, time.time() - t0, True


# ---------------------------------------------------------------- 主流程

def compare(title, price, needs, s0, s_end, reserve, refs):
    print('\n' + title)
    exact = []
    rows = {}
    for nm, fn in [('对偶单纯形法', lambda p, n, r: solve_lp(p, n, s0, s_end, reserve, r, 'highs-ds')),
                   ('内点法', lambda p, n, r: solve_lp(p, n, s0, s_end, reserve, r, 'highs-ipm')),
                   ('动态规划', lambda p, n, r: solve_dp(p, n, s0, s_end, reserve, r))]:
        vals, tms = [], []
        for need, ref in zip(needs, refs):
            v, tm, ok = fn(price, need, ref)
            vals.append(v); tms.append(tm)
        rows[nm] = (np.array(vals), float(np.mean(tms)), 1.0)
    best = rows['对偶单纯形法'][0]

    for nm, fn in [('遗传算法', ga), ('粒子群算法', pso)]:
        allv, tms, hit = [], [], []
        for i, (need, ref) in enumerate(zip(needs, refs)):
            vs = []
            for r in range(N_RUN):
                v, tm, _ = fn(price, need, s0, ref, 20260910 + r)
                vs.append(v); tms.append(tm)
            vs = np.array(vs)
            allv.append(vs.min())
            hit.append(float(np.mean(vs <= best[i] * 1.001)))
        rows[nm] = (np.array(allv), float(np.mean(tms)), float(np.mean(hit)))

    allv, tms, hit = [], [], []
```

```
    for i, (need, ref) in enumerate(zip(needs, refs)):
        vs, tm = sa_batch(price, need, s0, ref, 20260910)
        tms.append(tm)
        allv.append(float(vs.min()))
        hit.append(float(np.mean(vs <= best[i] * 1.001)))
    rows['模拟退火算法'] = (np.array(allv), float(np.mean(tms)), float(np.mean(hit)))

    vals, tms = [], []
    for need, ref in zip(needs, refs):
        v, tm, _ = rule(price, need, s0, ref)
        vals.append(v); tms.append(tm)
    rows['谷充峰放规则法'] = (np.array(vals), float(np.mean(tms)), 0.0)

    print('  %-14s %14s %10s %12s %10s' % ('算法', '平均目标值 元', '相对最优', '命中最优比例', '单次用时
    s'))
    for nm, (v, tm, h) in rows.items():
        gap = 100 * np.mean((v - best) / best)
        print('  %-14s %14.4f %9.3f%% %11.0f%% %10.3f' % (nm, v.mean(), gap, 100 * h, tm))
    return rows


def main():
    price, load, pv = load_all()
    fc = load_a3()
    days = sorted(SAMPLE)[1:]
    print('取样日期: 每月 15 日, 共 %d 天' % len(days))

    refs, needs = [], []
    for d in days:
        A_ub, b_ub, A_eq, b_eq, cost, bnd, nb = build(
            price, need_q3(load, pv, fc, d, 0), S_INIT, S_INIT, Q3_RESERVE_PLAN, None)
        r = linprog(cost, A_ub=A_ub, b_ub=b_ub, bounds=bnd, method='highs-ds')
        refs.append(r.x[:T])
        needs.append(need_q3(load, pv, fc, d, 1))
    compare('问题三 日内调整子问题的多算法对比', price, needs, S_INIT, S_INIT,
            Q3_RESERVE_ADJ, refs)


if __name__ == '__main__':
    main()
```

Q3/q3_ 预报精度对比.py

```
# -*- coding: utf-8 -*-
""" 问题三的光伏预报精度对比, 只算误差, 不求解购电策略。

对照三处口径:
    一、整点预报插值到时段中点还是时段末刻。附件 2 的第 k 个数是 (k+1)*10 分钟时刻的功率,
        按末刻取值才与实测对得上。
    二、6:00、12:00、18:00 发布时, 是否把发布时刻已经实测到的功率作为插值的左端点。
    三、官方预报与近 5 天历史均值的融合权重: 按发布时刻取常数, 还是随提前小时数线性衰减。
评估区间为 2 月 1 日至 12 月 31 日, 每个发布时刻只统计当天剩余的时段,
误差取绝对误差之和除以实测之和。
"""
import os

import numpy as np
import pandas as pd

# 附件目录与输出目录。可以直接在引号里填, 也可以用环境变量 C2026_DATA 与 C2026_OUT 指定,
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')


def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
```

```
                    (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')

T, DT = 144, 1.0 / 6.0
ISSUE_HOUR = (0, 6, 12, 18)
K_PV = 5
DAYS = np.arange(31, 365)
W_OLD = (0.3, 0.5, 0.7, 0.0)          # 按发布时刻取常数的旧权重
W0, W_SLOPE, W_MAX = 0.95, 0.05, 0.95  # 随提前小时数衰减的新权重


def load_data():
    pv = pd.read_excel(os.path.join(DATA, '附件2.xlsx'), sheet_name='光伏发电实际功率').iloc[:,
    1:].to_numpy(float)
    fc = pd.read_excel(os.path.join(DATA, '附件 3.xlsx')).iloc[:, 2:].to_numpy(float).reshape(365, 4, 24)
    return pv, fc


def official(pv, fc, d, issue, shift=1.0, anchor=True):
    """ 第 d 天第 issue 次发布的整点预报插值到 144 个时段。shift=1 取时段末刻，0.5 取中点。"""
    h0 = ISSUE_HOUR[issue]
    hrs = np.arange(h0 + 1, h0 + 25, dtype=float)
    f = fc[d, issue]
    if anchor and h0 > 0:
        hrs = np.concatenate([[h0], hrs])
        f = np.concatenate([[pv[d, h0 * 6 - 1]], f])
    tm = (np.arange(T) + shift) * DT
    return np.maximum(np.interp(tm, hrs, f), 0.0)


def history(pv, d):
    return pv.mean(0) if d == 0 else pv[max(0, d - K_PV):d].mean(0)


def lead_weight(issue):
    lead = (np.arange(T) + 1) * DT - ISSUE_HOUR[issue]
    return np.clip(W0 - W_SLOPE * lead, 0.0, W_MAX)


def rel_err(pv, F, issue):
    s0 = ISSUE_HOUR[issue] * 6
    A = pv[DAYS, s0:]
    return np.abs(F[:, s0:] - A).sum() / A.sum()


def main():
    pv, fc = load_data()
    H = np.stack([history(pv, d) for d in DAYS])
    lines = ['发布时刻  中点插值  末刻插值  末刻加锚点  近 5 天历史  旧融合  新融合  常数最优权重及误差']
    for issue in range(3):
        mid = np.stack([official(pv, fc, d, issue, 0.5, False) for d in DAYS])
        end = np.stack([official(pv, fc, d, issue, 1.0, False) for d in DAYS])
        anc = np.stack([official(pv, fc, d, issue, 1.0, True) for d in DAYS])
        old = W_OLD[issue] * mid + (1 - W_OLD[issue]) * H
        w = lead_weight(issue)
        new = w * anc + (1 - w) * H
        grid = np.linspace(0, 1, 21)
        errs = [rel_err(pv, g * anc + (1 - g) * H, issue) for g in grid]
        k = int(np.argmin(errs))
        lines.append('%5d:00  %7.2f%%  %7.2f%%  %8.2f%%  %8.2f%%  %6.2f%%  %6.2f%%  %.2f / %.2f%%' % (
            ISSUE_HOUR[issue], 100 * rel_err(pv, mid, issue), 100 * rel_err(pv, end, issue),
            100 * rel_err(pv, anc, issue), 100 * rel_err(pv, H, issue), 100 * rel_err(pv, old, issue),
```

```
            100 * rel_err(pv, new, issue), grid[k], 100 * errs[k]))
    # 按提前小时数分档的误差：官方预报随提前量变差、历史均值不随提前量变
    lines.append('')
    lines.append('0:00 发布的预报按提前小时数分档的误差（只统计白天）')
    anc0 = np.stack([official(pv, fc, d, 0, 1.0, True) for d in DAYS])
    for lo, hi in ((6, 9), (9, 12), (12, 15), (15, 18)):
        sl = slice(lo * 6, hi * 6)
        A = pv[DAYS, sl]
        lines.append('  提前 %2d-%2d 小时  官方 %.2f%%  历史 %.2f%%' % (
            lo, hi, 100 * np.abs(anc0[:, sl] - A).sum() / A.sum(), 100 * np.abs(H[:, sl] - A).sum() /
            A.sum()))
    text = '\n'.join(lines)
    print(text)
    os.makedirs(OUT, exist_ok=True)
    open(os.path.join(OUT, 'q3_ 预报精度对比.txt'), 'w', encoding='utf-8').write(text)


if __name__ == '__main__':
    main()
```

Q3/q3_ 档位对照.py

```
# -*- coding: utf-8 -*-
""" 问题三的档位与参数对照：调整时刻开几个、备用与裕度怎么取，各跑一遍全年因果仿真。

直接调用同目录的 q3_ 求解.py，只改它的参数，保证与正式结果同一套代码。
"""
import datetime as dt
import importlib.util
import os
import sys
import time

import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.environ.get('C2026_OUT', '')


def load_solver():
    spec = importlib.util.spec_from_file_location('q3_solver', os.path.join(HERE, 'q3_ 求解.py'))
    m = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(m)
    return m


def stat(m, recs):
    sub = recs[m.REPORT_FROM:]
    return dict(total=sum(r['cost'] for r in sub),
                plan=sum(r['cost_plan'] for r in sub),
                dev=sum(r['cost_dev'] for r in sub),
                emerg=sum(r['cost_emerg'] for r in sub),
                kwh=float(sum(r['e'].sum() for r in sub)),
                days=int(sum(1 for r in sub if r['e'].sum() > 1e-6)))


CASES = [
    ('只用 0:00 预报', dict(adjust=())),
    ('加 6:00', dict(adjust=(1,))),
    ('加 6:00、12:00', dict(adjust=(1, 2))),
    ('加 6:00、12:00、18:00（本文）', dict()),
    ('不用附件 3 的预报', dict(W0=0.0, W_MAX=0.0)),
    ('计划备用 0', dict(PLAN=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=0.0),
                        ADJ=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=0.0))),
    ('计划备用 250', dict(PLAN=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=250.0),
                          ADJ=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=250.0))),
    ('计划备用 500，调整备用 500', dict(PLAN=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0,
    RESERVE=500.0),
                                        ADJ=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0,
                                        RESERVE=500.0))),
```

```
    ('计划备用 750, 调整备用 500', dict(PLAN=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0,
    RESERVE=750.0),
                                        ADJ=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0,
                                        RESERVE=500.0))),
    ('计划备用 1000, 调整备用 500', dict(PLAN=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0,
    RESERVE=1000.0),
                                         ADJ=dict(MARGIN_N=0.015, MARGIN_L=0.0, MARGIN_P=0.0,
                                         RESERVE=500.0))),
    ('净负载裕度 0.5%', dict(PLAN=dict(MARGIN_N=0.005, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=500.0),
                             ADJ=dict(MARGIN_N=0.005, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=500.0))),
    ('净负载裕度 2.5%', dict(PLAN=dict(MARGIN_N=0.025, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=500.0),
                             ADJ=dict(MARGIN_N=0.025, MARGIN_L=0.0, MARGIN_P=0.0, RESERVE=500.0))),
]


def main():
    sel = sys.argv[1:]
    lines = ['问题三的档位与参数对照, 统计 2 月 1 日至 12 月 31 日',
             '  %-30s %14s %13s %12s %12s %9s %5s'
             % ('算例', '总费用', '计划购电费', '调整相关费', '紧急购电费', '紧急电量', '天数')]
    for name, cfg in CASES:
        if sel and not any(s in name for s in sel):
            continue
        t0 = time.time()
        m = load_solver()
        adjust = cfg.pop('adjust', (1, 2, 3))
        for k, v in cfg.items():
            setattr(m, k, v)
        price = m.load_a1()
        load, pv = m.load_a2()
        fc = m.load_a3()
        recs = m.run_year(np.tile(price, (365, 1)), load, pv, fc, adjust=adjust)
        r = stat(m, recs)
        lines.append('  %-30s %14.2f %13.2f %12.2f %12.2f %9.0f %5d'
                     % (name, r['total'], r['plan'], r['dev'], r['emerg'], r['kwh'], r['days']))
        print(lines[-1], '  用时 %.0f s' % (time.time() - t0), flush=True)
    out = OUT or os.path.join(os.path.dirname(HERE), '结果')
    os.makedirs(out, exist_ok=True)
    open(os.path.join(out, 'q3_ 档位对照.txt'), 'w', encoding='utf-8').write('\n'.join(lines))


if __name__ == '__main__':
    main()
```

Q4/q4_ 求解.py

```
# -*- coding: utf-8 -*-
import datetime as dt
import os

import numpy as np
import openpyxl
import pandas as pd
from scipy import sparse
from scipy.optimize import linprog

# 附件目录与输出目录。可以直接在引号里填, 也可以用环境变量 C2026_DATA 与 C2026_OUT 指定,
# 都不给时从本文件所在位置逐级向上查找。
DATA = os.environ.get('C2026_DATA', '')
OUT = os.environ.get('C2026_OUT', '')

def _find_root():
    d = os.path.dirname(os.path.abspath(__file__))
    for _ in range(6):
        for p, out in ((os.path.join(d, '题目', '附件'), os.path.join(d, '结果')),
                       (os.path.join(d, 'data', '附件'), os.path.join(d, 'out')),
                       (os.path.join(d, '附件'), os.path.join(d, '结果'))):
            if os.path.isfile(os.path.join(p, '附件 1.xlsx')):
                return p, out
        d = os.path.dirname(d)
```

```
    return '', os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


if not DATA:
    DATA, _out = _find_root()
    OUT = OUT or _out
OUT = OUT or os.path.join(os.path.dirname(os.path.abspath(__file__)), '结果')


T = 144
DT = 1.0 / 6.0
P_MAX = 5000.0
E_MAX = P_MAX * DT
S_LO, S_HI = 1200.0, 10800.0
S_INIT = 6000.0
ETA = 0.9
EMERG = 5.0
DEV_LO, DEV_HI = 0.5, 1.5
# 问题二口径：负载与光伏分项裕度，上午与下午分段备用
Q2_MARGIN_L, Q2_MARGIN_P = 0.0, 0.05
Q2_RESERVE_M, Q2_RESERVE_E, Q2_SPLIT = 500.0, 1000.0, 78
# 问题三口径：净负载按 n + MARGIN_N*|n| 加码，计划与调整各自的备用
Q3_PLAN = dict(MARGIN_N=0.015, RESERVE=500.0)
Q3_ADJ = dict(MARGIN_N=0.015, RESERVE=250.0)
ISSUE_HOUR = (0, 6, 12, 18)
ISSUE_SLOT = (0, 36, 72, 108)
W0, W_SLOPE, W_MAX = 0.95, 0.05, 0.95
K_PV = 5
N_SHAPE = 8
LEVEL_W = (0.6, 0.25, 0.15)
N_RATIO = 28
REPORT_FROM = 31
EPS = 1e-7

DATES = [dt.date(2025, 1, 1) + dt.timedelta(days=i) for i in range(365)]
DOW = np.array([d.weekday() for d in DATES])
WK = ((DOW >= 4) & (DOW <= 5)).astype(int)
BLOCKS = [('0:00-4:00', slice(0, 24)), ('4:00-8:00', slice(24, 48)),
          ('8:00-12:00', slice(48, 72)), ('12:00-16:00', slice(72, 96)),
          ('16:00-20:00', slice(96, 120)), ('20:00-24:00', slice(120, 144))]


def load_a2():
    xl = pd.ExcelFile(os.path.join(DATA, '附件 2.xlsx'))
    load = pd.read_excel(xl, '小区负载').iloc[:, 1:].to_numpy(float)
    pv = pd.read_excel(xl, '光伏发电实际功率').iloc[:, 1:].to_numpy(float)
    return load, pv


def load_a3():
    d = pd.read_excel(os.path.join(DATA, '附件 3.xlsx'))
    return d.iloc[:, 2:].to_numpy(float).reshape(365, 4, 24)


def load_a4():
    d = pd.read_excel(os.path.join(DATA, '附件 4.xlsx'))
    return d.iloc[:, 1:].to_numpy(float)


# ---------------- 预测 ----------------
_CACHE = {}


def _harm(load, d):
    js = np.arange(d)
    X = lambda j: np.column_stack([np.ones(len(j)), WK[j],
                                   np.sin(2 * np.pi * j / 365), np.cos(2 * np.pi * j / 365),
                                   np.sin(4 * np.pi * j / 365), np.cos(4 * np.pi * j / 365)])
    if d < 12:
        return load[:d].mean(0) if d > 0 else load.mean(0)
    b, *_ = np.linalg.lstsq(X(js), load[js], rcond=None)
```

```
    return X(np.array([d]))[0] @ b


def load_hat(load, d):
    """ 日型形状乘日总量水平：形状取最近 N_SHAPE 个同类日的归一化曲线，
    水平取最近三天折算到同一日型后的加权日总量。"""
    if d in _CACHE:
        return _CACHE[d]
    if d < 14:
        v = _harm(load, d)
    else:
        js = [j for j in range(d - 1, -1, -1) if WK[j] == WK[d]][:N_SHAPE]
        shape = (load[js] / load[js].sum(1, keepdims=True)).mean(0)
        jr = np.arange(max(0, d - N_RATIO), d)
        ratio = load[jr][WK[jr] == 1].sum(1).mean() / load[jr][WK[jr] == 0].sum(1).mean()
        tot = load[:d].sum(1) / np.where(WK[:d] == 1, ratio, 1.0)
        level = sum(w * tot[-1 - i] for i, w in enumerate(LEVEL_W)) / sum(LEVEL_W)
        v = shape * level * (ratio if WK[d] == 1 else 1.0)
    _CACHE[d] = v
    return v


def pv_hist(pv, d):
    return pv.mean(0) if d == 0 else pv[max(0, d - K_PV):d].mean(0)


def forecast_to_slots(pv, fc, d, issue):
    h0 = ISSUE_HOUR[issue]
    hrs = np.arange(h0 + 1, h0 + 25, dtype=float)
    f = fc[d, issue]
    if h0 > 0:
        hrs = np.concatenate([[h0], hrs])
        f = np.concatenate([[pv[d, h0 * 6 - 1]], f])
    tm = (np.arange(T) + 1) * DT
    return np.maximum(np.interp(tm, hrs, f), 0.0)


def pv_blend(pv, fc, d, issue):
    lead = (np.arange(T) + 1) * DT - ISSUE_HOUR[issue]
    w = np.clip(W0 - W_SLOPE * lead, 0.0, W_MAX)
    return w * forecast_to_slots(pv, fc, d, issue) + (1 - w) * pv_hist(pv, d)


# ---------------- 线性规划 ----------------
_MAT = {}


def _mats(n, adj):
    key = (n, adj)
    if key not in _MAT:
        I = sparse.identity(n, format='csr'); Z = sparse.csr_matrix((n, n))
        Dm = sparse.diags([np.ones(n), -np.ones(n - 1)], [0, -1], format='csr')
        if adj:
            A_ub = sparse.hstack([-I, I, -I, -I, Z, Z, Z], format='csr')
            A_eq = sparse.vstack([sparse.hstack([Z, -ETA * I, I / ETA, Z, Dm, Z, Z]),
                                  sparse.hstack([I, Z, Z, Z, Z, -I, I])], format='csr')
        else:
            A_ub = sparse.hstack([-I, I, -I, -I, Z], format='csr')
            A_eq = sparse.hstack([Z, -ETA * I, I / ETA, Z, Dm], format='csr')
        _MAT[key] = (A_ub, A_eq)
    return _MAT[key]


def day_lp(price, need, s0, s_end=S_INIT, lo=S_LO, g_fix=None, ref=None):
    n = len(price); adj = ref is not None
    A_ub, A_eq = _mats(n, adj)
    nb = 7 if adj else 5
    b_eq = np.zeros(A_eq.shape[0]); b_eq[0] = s0
    if adj:
        b_eq[n:] = ref
    cost = np.zeros(nb * n)
```

```
    cost[:n] = 0.0 if adj else price
    cost[n:3 * n] = EPS
    cost[3 * n:4 * n] = EMERG * price
    if adj:
        cost[5 * n:6 * n] = DEV_HI * price
        cost[6 * n:7 * n] = -DEV_LO * price
    lb = np.zeros(nb * n); ub = np.full(nb * n, np.inf)
    ub[n:3 * n] = E_MAX
    lo_v = np.minimum(np.broadcast_to(np.asarray(lo, float), (n,)).copy(), S_HI)
    lo_v[-1] = min(max(lo_v[-1], s_end), S_HI)
    lb[4 * n:5 * n] = lo_v; ub[4 * n:5 * n] = S_HI
    if g_fix is not None:
        lb[:n] = g_fix; ub[:n] = g_fix
    res = linprog(cost, A_ub=A_ub, b_ub=-np.asarray(need, float), A_eq=A_eq, b_eq=b_eq,
                  bounds=np.stack([lb, ub], 1), method='highs')
    if not res.success:
        return None
    x = res.x
    return dict(g=x[:n], c=x[n:2 * n], d=x[2 * n:3 * n], e=x[3 * n:4 * n], S=x[4 * n:5 * n])


def solve_with_fallback(price, need, s0, lo, ref=None):
    lo_last = float(np.atleast_1d(lo)[-1])
    r = day_lp(price, need, s0, s_end=max(S_INIT, lo_last), lo=lo, ref=ref)
    if r is None:
        r = day_lp(price, need, s0, s_end=S_LO, ref=ref)
    return r


def step(price, need_act, a, need_fc, s, t, out):
    """ 执行第 t 段。有富余就充电；有缺口时先判断储能是否足够覆盖预测的后续全部缺口，
    够就直接放电补齐，不够再解剩余时段的放电分配问题。"""
    net = a[t] - need_act[t]
    avail_dis = min(E_MAX, max(0.0, (s - S_LO) * ETA))
    avail_chg = min(E_MAX, max(0.0, (S_HI - s) / ETA))
    if net >= 0:
        ct, dt_ = min(net, avail_chg), 0.0
    else:
        want = -net
        intent = want
        if avail_dis > 1e-9:
            fc = np.concatenate([[need_act[t]], need_fc[t + 1:]])
            future_def = np.maximum(fc[1:] - a[t + 1:], 0.0).sum()
            if (s - S_LO) * ETA - min(want, avail_dis) < future_def:
                r = day_lp(price[t:], fc, s, s_end=S_LO, g_fix=a[t:])
                if r is not None:
                    intent = max(r['d'][0], 0.0)
        ct, dt_ = 0.0, min(intent, want, avail_dis)
    out['e'][t] = max(0.0, -net - dt_)
    out['c'][t], out['d'][t] = ct, dt_
    s = float(np.clip(s + ETA * ct - dt_ / ETA, S_LO, S_HI))
    out['S'][t] = s
    return s


def run_day_q2(price, need_act, lf, pf, s0):
    need_plan = (lf * (1 + Q2_MARGIN_L) - pf * (1 - Q2_MARGIN_P)) * DT
    tt = np.arange(T)
    lo = np.minimum(S_LO + np.where(tt < Q2_SPLIT, Q2_RESERVE_M, Q2_RESERVE_E), s0)
    plan = solve_with_fallback(price, need_plan, s0, lo)['g']
    need_fc = (lf - pf) * DT
    out = dict(c=np.zeros(T), d=np.zeros(T), e=np.zeros(T), S=np.zeros(T))
    s = float(s0)
    for t in range(T):
        s = step(price, need_act, plan, need_fc, s, t, out)
    cost_plan = float(price @ plan)
    cost_emerg = float(EMERG * (price @ out['e']))
    return dict(a=plan.copy(), plan=plan, s_end=s, cost_plan=cost_plan, cost_dev=0.0,
                cost_emerg=cost_emerg, cost=cost_plan + cost_emerg, **out)
```

```
def run_day_q3(price, need_act, lf, pfs, s0, adjust=(1, 2, 3)):
    def hedged(pf, prm):
        n = (lf - pf) * DT
        return n + prm['MARGIN_N'] * np.abs(n)
    plan = solve_with_fallback(price, hedged(pfs[0], Q3_PLAN), s0,
                               min(S_LO + Q3_PLAN['RESERVE'], s0, S_HI))['g']
    a = plan.copy()
    need_fc = (lf - pfs[0]) * DT
    out = dict(c=np.zeros(T), d=np.zeros(T), e=np.zeros(T), S=np.zeros(T))
    s = float(s0)
    for t in range(T):
        if t in ISSUE_SLOT[1:] and ISSUE_SLOT.index(t) in adjust:
            i = ISSUE_SLOT.index(t)
            need_fc = (lf - pfs[i]) * DT
            need_fc[:t] = need_act[:t]
            nd = hedged(pfs[i], Q3_ADJ)
            r = solve_with_fallback(price[t:], nd[t:], s,
                                    min(S_LO + Q3_ADJ['RESERVE'], s, S_HI), ref=plan[t:])
            if r is not None:
                a[t:] = r['g']
        s = step(price, need_act, a, need_fc, s, t, out)
    up = np.maximum(a - plan, 0.0)
    dn = np.maximum(plan - a, 0.0)
    cost_plan = float(price @ plan)
    cost_dev = float(DEV_HI * (price @ up) - DEV_LO * (price @ dn))
    cost_emerg = float(EMERG * (price @ out['e']))
    return dict(a=a, plan=plan, s_end=s, cost_plan=cost_plan, cost_dev=cost_dev,
                cost_emerg=cost_emerg, cost=cost_plan + cost_dev + cost_emerg, **out)


def run_year(prices, load, pv, fc, mode, n_days=365, tag='', adjust=(1, 2, 3)):
    s = float(S_INIT)
    recs = []
    for d in range(n_days):
        need_act = (load[d] - pv[d]) * DT
        lf = load_hat(load, d)
        if mode == 2:
            rec = run_day_q2(prices[d], need_act, lf, pv_hist(pv, d), s)
        else:
            rec = run_day_q3(prices[d], need_act, lf, [pv_blend(pv, fc, d, i) for i in range(4)],
                             s, adjust)
        rec['day'] = d
        rec['s0'] = s
        s = rec['s_end']
        recs.append(rec)
        if d % 30 == 0:
            print('  %s 第 %3d 天  费用 %9.2f  紧急 %8.2f  期末储电 %8.1f'
                  % (tag, d, rec['cost'], rec['cost_emerg'], s), flush=True)
    return recs


def slot_labels():
    ws = openpyxl.load_workbook(os.path.join(DATA, '附件 5', 'result2.xlsx'))['计划购电量']
    return [str(ws.cell(1, 2 + k).value) for k in range(T)]


LABELS = slot_labels()


def emergency_intervals(e, tol=1e-6):
    out = []
    k = 0
    while k < T:
        if e[k] > tol:
            j = k
            while j + 1 < T and e[j + 1] > tol:
                j += 1
            out.append(('%s-%s' % (LABELS[k].split('-')[0], LABELS[j].split('-')[-1]),
                        float(e[k:j + 1].sum())))
            k = j + 1
        else:
```

```
            k += 1
    return out


def r4(x):
    return float(np.round(float(x), 4))


def clear_below(ws, first_row=2):
    for row in ws.iter_rows(min_row=first_row, max_row=ws.max_row):
        for cell in row:
            cell.value = None


def write_grid(ws, recs, key):
    clear_below(ws)
    for i, r in enumerate(recs):
        row = 2 + i
        ws.cell(row, 1, DATES[r['day']].strftime('%Y/%m/%d'))
        v = r[key]
        for k in range(T):
            ws.cell(row, 2 + k, r4(v[k]))
        ws.cell(row, 2 + T, r4(float(np.sum(v))))
        ws.cell(row, 3 + T, r4(float(r['cost'])))


def write_result(recs, name, template, with_adj):
    wb = openpyxl.load_workbook(os.path.join(DATA, '附件 5', template))
    write_grid(wb['计划购电量'], recs, 'plan')
    if with_adj:
        write_grid(wb['调整购电量'], recs, 'a')

    ws = wb['充放电量']
    clear_below(ws)
    row = 2
    for r in recs:
        for j, (nm, sl) in enumerate(BLOCKS):
            ws.cell(row + j, 1, DATES[r['day']].strftime('%Y/%m/%d') if j == 0 else None)
            ws.cell(row + j, 2, nm)
            ws.cell(row + j, 3, r4(float(r['c'][sl].sum())))
            ws.cell(row + j, 4, r4(float(r['d'][sl].sum())))
        ws.cell(row, 5, '0:00')
        ws.cell(row, 6, r4(r['s0']))
        ws.cell(row + 1, 5, '24:00')
        ws.cell(row + 1, 6, r4(r['s_end']))
        row += 6

    ws = wb['紧急购电量']
    clear_below(ws)
    row = 2
    for r in recs:
        iv = emergency_intervals(r['e'])
        if not iv:
            ws.cell(row, 1, DATES[r['day']].strftime('%Y/%m/%d'))
            ws.cell(row, 2, '无')
            ws.cell(row, 3, 0.0)
            row += 1
            continue
        for j, (lab, v) in enumerate(iv):
            ws.cell(row + j, 1, DATES[r['day']].strftime('%Y/%m/%d') if j == 0 else None)
            ws.cell(row + j, 2, lab)
            ws.cell(row + j, 3, r4(v))
        row += len(iv)

    os.makedirs(OUT, exist_ok=True)
    path = os.path.join(OUT, name)
    wb.save(path)
    return path


def summary(recs, title):
```

```
    sub = recs[REPORT_FROM:]
    tot = sum(r['cost'] for r in sub)
    L = ['%s  统计 %s 至 %s，共 %d 天'
         % (title, DATES[sub[0]['day']], DATES[sub[-1]['day']], len(sub)),
         '  总购电费 %.2f 元' % tot,
         '  计划购电费 %.2f 元' % sum(r['cost_plan'] for r in sub),
         '  调整相关费 %.2f 元' % sum(r['cost_dev'] for r in sub),
         '  紧急购电费 %.2f 元' % sum(r['cost_emerg'] for r in sub),
         '  紧急购电量 %.2f kWh，发生在 %d 天'
         % (sum(float(r['e'].sum()) for r in sub),
            sum(1 for r in sub if r['e'].sum() > 1e-6))]
    return '\n'.join(L)


def main():
    load, pv = load_a2()
    fc = load_a3()
    prices = load_a4()

    print('波动电价下重算问题 2，不用附件 3 预报，当天不调整')
    r2 = run_year(prices, load, pv, fc, mode=2, tag='4-2')
    print()
    print(summary(r2, '问题 4 对应问题 2'))
    print('已写出', write_result(r2[REPORT_FROM:], 'result4-2.xlsx', 'result4-2.xlsx', False))

    print()
    print('波动电价下重算问题 3，用附件 3 预报，6:00 12:00 18:00 各调整一次')
    r3 = run_year(prices, load, pv, fc, mode=3, tag='4-3')
    print()
    print(summary(r3, '问题 4 对应问题 3'))
    print('已写出', write_result(r3[REPORT_FROM:], 'result4-3.xlsx', 'result4-3.xlsx', True))


if __name__ == '__main__':
    main()
```

Q4/q4_ 对照.py

```
# -*- coding: utf-8 -*-
""" 问题四的对照：波动电价下不调整、完全不做抵御，以及把电价换成全年逐段均值的退化检验。

直接调用同目录的 q4_ 求解.py，只改它的参数与输入电价，保证与正式结果同一套代码。
"""
import importlib.util
import os
import sys
import time

import numpy as np

HERE = os.path.dirname(os.path.abspath(__file__))
OUT = os.environ.get('C2026_OUT', '')


def load_solver():
    spec = importlib.util.spec_from_file_location('q4_solver', os.path.join(HERE, 'q4_ 求解.py'))
    m = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(m)
    return m


def stat(m, recs):
    sub = recs[m.REPORT_FROM:]
    return dict(total=sum(r['cost'] for r in sub),
                plan=sum(r['cost_plan'] for r in sub),
                dev=sum(r['cost_dev'] for r in sub),
                emerg=sum(r['cost_emerg'] for r in sub),
                kwh=float(sum(r['e'].sum() for r in sub)),
                days=int(sum(1 for r in sub if r['e'].sum() > 1e-6)))
```

```
CASES = [
    ('对应问题二，波动电价', dict(mode=2)),
    ('对应问题三，波动电价（本文）', dict(mode=3)),
    ('对应问题三，只用 0:00 预报', dict(mode=3, adjust=())),
    ('波动电价下不做任何抵御', dict(mode=2, zero_hedge=True)),
    ('退化检验：电价换成全年逐段均值', dict(mode=3, mean_price=True)),
]


def main():
    sel = sys.argv[1:]
    lines = ['问题四的对照，统计 2 月 1 日至 12 月 31 日',
             '  %-30s %14s %13s %12s %12s %9s %5s'
             % ('算例', '总费用', '计划购电费', '调整相关费', '紧急购电费', '紧急电量', '天数')]
    for name, cfg in CASES:
        if sel and not any(s in name for s in sel):
            continue
        t0 = time.time()
        m = load_solver()
        if cfg.get('zero_hedge'):
            m.Q2_MARGIN_P = 0.0
            m.Q2_RESERVE_M = 0.0
            m.Q2_RESERVE_E = 0.0
        load, pv = m.load_a2()
        fc = m.load_a3()
        prices = m.load_a4()
        if cfg.get('mean_price'):
            prices = np.tile(prices.mean(0), (365, 1))
        recs = m.run_year(prices, load, pv, fc, mode=cfg['mode'], adjust=cfg.get('adjust', (1, 2, 3)))
        r = stat(m, recs)
        lines.append('  %-30s %14.2f %13.2f %12.2f %12.2f %9.0f %5d'
                     % (name, r['total'], r['plan'], r['dev'], r['emerg'], r['kwh'], r['days']))
        print(lines[-1], '  用时 %.0f s' % (time.time() - t0), flush=True)
    out = OUT or os.path.join(os.path.dirname(HERE), '结果')
    os.makedirs(out, exist_ok=True)
    open(os.path.join(out, 'q4_ 对照.txt'), 'w', encoding='utf-8').write('\n'.join(lines))


if __name__ == '__main__':
    main()
```